MicroBlaze V Processor Reference Guide

UG1629 (v2025.1) July 9, 2025

Table of Contents

Chapter 1

Introduction

This guide provides information about the 32-bit and 64-bit AMD MicroBlaze™ V soft processor, which is included in the AMD Vivado™ Design Suite. The document is intended as a guide to the processor hardware architecture, accompanied by the RISC-V Instruction Set Manual, Volume I and II.

MicroBlaze V is fully hardware compatible with the classic MicroBlaze processor.

Note: In the Vivado 2025.1 release, certain features are early access, as detailed in Table 1: Configurable Feature Overview. Contact your sales representative for more information.

Guide Contents

This guide contains the following chapters:

- Chapter 2: MicroBlaze V Architecture contains an overview of processor features as well as information on specific custom functionality and cache implementation.
- Chapter 3: MicroBlaze V Signal Interface Description describes the types of signal interfaces that can be used to connect the processor.
- Appendix A: Performance and Resource Utilization contains maximum frequencies and resource utilization numbers for different configurations and devices.
- Appendix B: Additional Resources and Legal Notices provides links to documentation and additional resources.

Chapter 2

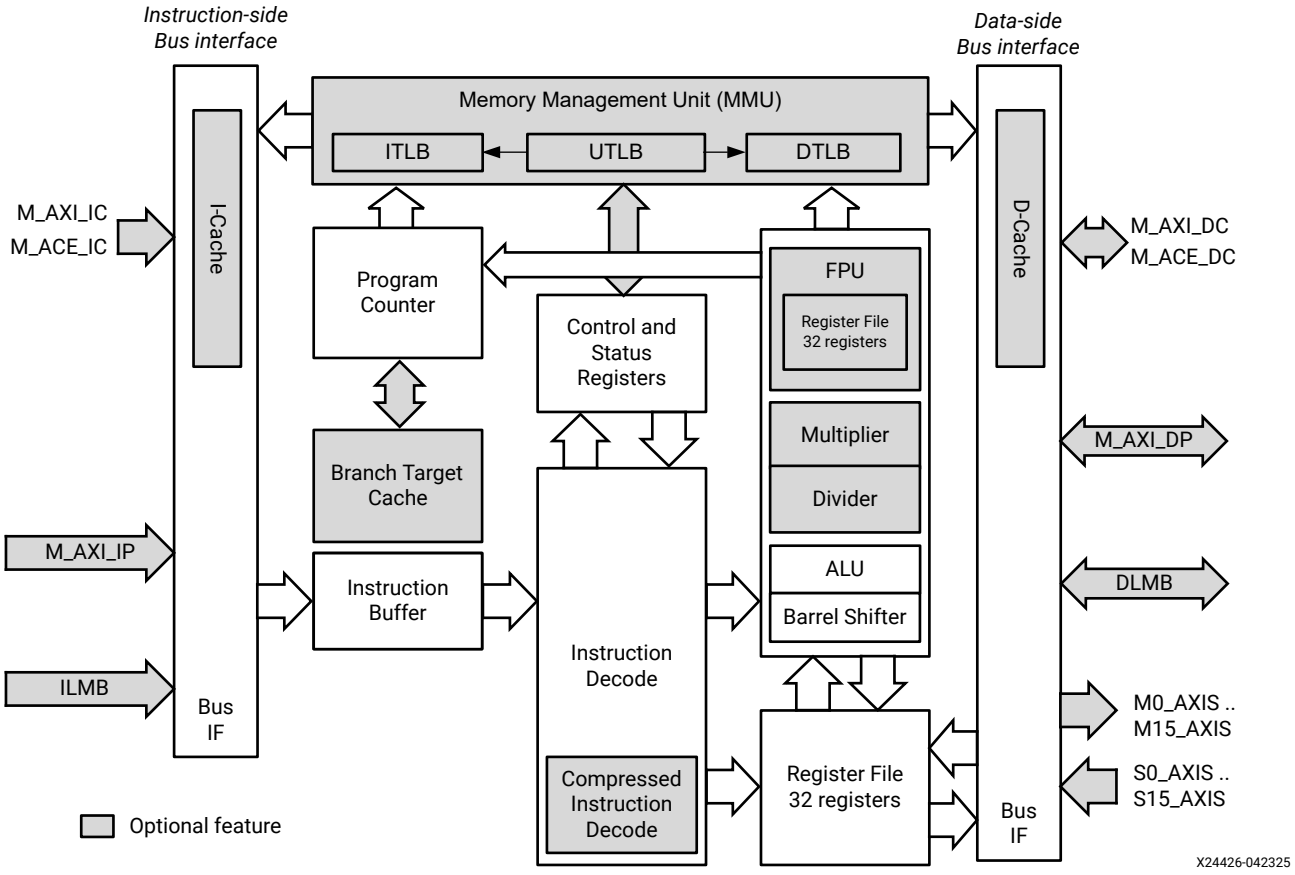
MicroBlaze V Architecture

This chapter contains an overview of MicroBlaze™ V processor features and includes detailed information on MicroBlaze V-specific optional functionality.

Overview

The MicroBlaze V embedded processor soft core is a reduced instruction set computer (RISC) optimized for implementation in AMD field programmable gate arrays (FPGAs). The following figure shows a functional block diagram of the processor.

Figure 1: **MicroBlaze V Processor Block Diagram**



Features

The MicroBlaze V soft core processor is highly configurable, allowing you to select a specific set of features required by your design.

The fixed feature set of the processor implements the RISC-V RV32I Base Integer Instruction Set and the machine-level ISA.

- 32 general purpose registers
- Extensible 32-bit program counter
- Single-issue pipeline
- Arithmetic logic unit (ALU)
- Barrel-shifter
- "Zicsr" control and status register (CSR) instructions
- "Zifencei" instruction-fetch fence

In addition to these fixed features, the MicroBlaze V processor is parameterized to allow selective enabling of additional functionality.

The following table provides an overview of the configurable features.

Table 1: **Configurable Feature Overview**

Feature	Version
	v1.0
RV64I Base Integer Instruction Set	Option
Processor pipeline depth	3, 4, 5, 8
"M" Standard Extension for Integer Multiplier and Divider	Option
"A" Standard Extension for Atomic Instructions	Option
"C" Standard Extension for Compressed Instructions	Option
"F" Standard Extension for Single-Precision Floating-Point	Option
"D" Standard Extension for Double Precision Floating Point	Option
"Zba", "Zbb", "Zbc", "Zbs" Bit-Manipulation ISA Extensions	Option
User-mode	Option
Physical Memory Protection	Option
"Smepmp" PMP Enhancements	Option
Supervisor Level ISA[1]	Option
Page-Based Virtual Memory System[1]	Option
External debug support	Option
Parallel debug module interface	Option
Branch target cache (BTC)	Option
Local memory bus (LMB) data side interface	Option

Table 1: **Configurable Feature Overview** *(cont'd)*

Feature	Version
	v1.0
Local memory bus (LMB) instruction side interface	Option
Instruction and data caches with "Zicbom" Cache Block Management	Option
Cache line word length	4, 8, 16
LUT cache memory	Option
Use write-back caching policy for D-Cache	Option
Streams for I-Cache	Option
Victim handling for I-Cache	0, 2, 4, 8
Victim handling for D-Cache	0, 2, 4, 8
Force distributed RAM for cache tags	Option
Configurable cache data widths	Option
AXI4 (M_AXI_DP) data side interface	Option
AXI4 (M_AXI_IP) instruction side interface	Option
AXI4 (M_AXI_DC) protocol for D-Cache	Option
AXI4 (M_AXI_IC) protocol for I-Cache	Option
ACE (M_ACE_DC) protocol for D-Cache[1]	Option
ACE (M_ACE_IC) protocol for I-Cache[1]	Option
AXI4 non-secure mode	Option
Lockstep support	Option
Configurable use of FPGA primitives	Option
Relocatable base vectors	Option
Pipeline pause functionality	Option

Notes:

1. Early access feature in 2025.1, available through lounge access. Contact your sales representative for more information.

Terminology

RISC-V terminology used in this guide is briefly explained in the following list, as it pertains to the MicroBlaze V processor. Refer to the RISC-V Instruction Set Manual for a complete and comprehensive explanations of the terms.

- **Custom Instruction:** An instruction set category available for vendor-specific non-standard extensions. MicroBlaze V defines GET and PUT custom instructions to support AXI4-Stream interfaces, providing compatibility with classic MicroBlaze.
- **Exception:** An unusual condition occurring at runtime associated with an instruction in the current RISC-V hart.
- **Hart:** Hardware thread. Each MicroBlaze V core only supports one hart.

- **Interrupt:** An external asynchronous event that can cause a RISC-V hart to experience an unexpected transfer of control. MicroBlaze V supports machine external interrupt, non-maskable interrupt, and custom platform interrupt.
- **Retire:** An instruction is said to retire when its execution completes. In MicroBlaze V instructions are retired when they leave the execute (EX) pipeline stage for the 3-stage pipeline, or the writeback (WB) pipeline stage for all other pipelines.
- **Trap:** The transfer of control to a trap handler caused by either an exception or an interrupt.

Instructions

MicroBlaze V implements instructions as defined by the RISC-V Instruction Set Manual.

Instruction latency is listed in the following table where differences due to pipeline optimization are indicated, as well as optional instructions and their controlling parameters.

Table 2: **Instruction Latency**

Instruction Mnemonics	Latency	Remark
RV32I Base Integer Instruction Set		
LUI	1	
AUIPC	1	
JAL, JALR	3	3-, 4-, 5-stage pipeline
	5	8-stage pipeline
BEQ, BNE, BLT, BGE, BLTU, BGEU	1	If branch is not taken
	3	If branch is taken, 3-, 4-, 5-stage pipeline
	5	If branch is taken, 8-stage pipeline
LB, LH, LW, LBU, LHU, SB, SH, SW	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
ADDI, SLTI, SLTIU, XORI, ORI, ANDI	1	
ADD, SUB, SLT, SLTU, XOR, OR, AND	1	
SLLI, SRLI, SRAI, SLL, SRL, SRA[1]	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
FENCE[2]	2 + N	C_INTERCONNECT = 2 (AXI)
	8 + N	C_INTERCONNECT = 3 (ACE)
ECALL	5	
EBREAK	5	
RV64I Base Integer Instruction Set		**C_DATA_SIZE = 64**
ADDIW	1	
ADDW, SUBW	1	

Table 2: **Instruction Latency** *(cont'd)*

Instruction Mnemonics	Latency	Remark
SLLIW, SRLIW, SRAIW, SLLW, SRLW, SRAW	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
LWU	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
LD, SD[3]	1, 2	4-, 5-, 8-stage pipeline
	2, 3	3-stage pipeline
RV32/RV64 Zifencei Standard Extension		
FENCE.I[2]	2 + N	C_INTERCONNECT = 2 (AXI)
	8 + N	C_INTERCONNECT = 3 (ACE)
RV32/RV64 Zicsr Standard Extension		
CSRRW, CSRRS, CSRRC	1	
CSRRWI, CSRRSI, CSRRCI	1	
RV32M Standard Extension		**C_USE_MULDIV > 0, C_DATA_SIZE = 32**
MUL, MULH, MULHSU, MULHU	1	4-, 5-, 8-stage pipeline
	3	3-stage pipeline
DIV, DIVU[4]	30	8-stage pipeline
	34	5-stage pipeline
	35	3-, 4-stage pipeline
REM, REMU[4]	31	8-stage pipeline
	35	5-stage pipeline
	36	3-, 4-stage pipeline
RV32M Standard Extension		**C_USE_MULDIV > 0, C_DATA_SIZE = 64**
MUL, MULH, MULHSU, MULHU	2	4-, 5-, 8-stage pipeline
	4	3-stage pipeline
DIV, DIVU[4]	62	8-stage pipeline
	66	5-stage pipeline
	67	3-, 4-stage pipeline
REM, REMU[4]	63	8-stage pipeline
	67	5-stage pipeline
	68	3-, 4-stage pipeline
RV64M Standard Extension		**C_DATA_SIZE = 64**
MULW	1	4-, 5-, 8-stage pipeline
	3	3-stage pipeline
DIVW, DIVUW[4]	30	8-stage pipeline
	34	5-stage pipeline
	35	3-, 4-stage pipeline
REMW, REMUW[4]	31	8-stage pipeline
	35	5-stage pipeline
	36	3-, 4-stage pipeline

Table 2: **Instruction Latency** *(cont'd)*

Instruction Mnemonics	Latency	Remark
RV32A Standard Extension		**C_USE_ATOMIC = 1**
LR.W	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
SC.W	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
AMO*.W	5	4-, 5-, 8-stage pipeline
	7	3-stage pipeline
RV64A Standard Extension[3]		**C_USE_ATOMIC = 1, C_DATA_SIZE = 64**
LR.D	1, 2	4-, 5-, 8-stage pipeline
	2, 3	3-stage pipeline
SC.D	1, 2	4-, 5-, 8-stage pipeline
	2, 3	3-stage pipeline
AMO*.D	5, 6	4-, 5-, 8-stage pipeline
	7, 8	3-stage pipeline
RV32F Standard Extension		**C_USE_FPU = 1**
FLW, FSW	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
FMADD.S, FMSUB.S, FNMADD.S, FNMSUB.S	2	8-stage pipeline
	5	5-stage pipeline
	6	4-stage pipeline
	7	3-stage pipeline
FADD.S, FSUB.S	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FMUL.S[5]	1	8-stage pipeline
	3	5-stage pipeline
	4	4-stage pipeline
	5	3-stage pipeline
FDIV.S[5]	26	8-stage pipeline
	30	4-, 5-stage pipeline
	32	3-stage pipeline
FSQRT.S[5]	25	8-stage pipeline
	29	4-, 5-stage pipeline
	31	3-stage pipeline
FSGNJ.S, FSGNJN.S, FSGNJX.S, FMIN.S, FMAX.S, FCLASS.S	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline

Table 2: **Instruction Latency** *(cont'd)*

Instruction Mnemonics	Latency	Remark
FCVT.W.S, FCVT.WU.S	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FEQ.S, FLT.S, FLE.S	1	4-, 5-, 8-stage pipeline
	3	3-stage pipeline
FCVT.S.W, FCVT.S.WU	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FMV.X.W, FMV.W.X	1	
RV64F Standard Extension		**C_USE_FPU = 2, C_DATA_SIZE = 64**
FCVT.L.S, FCVT.LU.S	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FCVT.S.L, FCVT.S.LU	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
RV64D Standard Extension		**C_USE_FPU = 2, C_DATA_SIZE = 64**
FLD, FSD	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
FMADD.D, FMSUB.D, FNMADD.D, FNMSUB.D	3	8-stage pipeline
	6	5-stage pipeline
	7	4-stage pipeline
	8	3-stage pipeline
FADD.D, FSUB.D	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FMUL.D[5]	1	8-stage pipeline
	3	5-stage pipeline
	4	4-stage pipeline
	5	3-stage pipeline
FDIV.D[5]	26	8-stage pipeline
	30	4-, 5-stage pipeline
	32	3-stage pipeline

Table 2: **Instruction Latency** *(cont'd)*

Instruction Mnemonics	Latency	Remark
FSQRT.D[5]	25	8-stage pipeline
	29	4-, 5-stage pipeline
	31	3-stage pipeline
FSGNJ.D, FSGNJN.D, FSGNJX.D, FMIN.D, FMAX.D, FCLASS.D	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
FCVT.W.D, FCVT.WU.D, FCVT.L.D, FCVT.LU.D	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FEQ.D, FLT.D, FLE.D	1	4-, 5-, 8-stage pipeline
	3	3-stage pipeline
FCVT.D.W, FCVT.D.WU, FCVT.D.L, FCVT.D.LU	1	8-stage pipeline
	4	5-stage pipeline
	5	4-stage pipeline
	6	3-stage pipeline
FMV.X.D, FMV.D.X	1	
Zba Extension		**C_USE_BITMAN_A = 1**
ADD.UW, SH1ADD, SH1ADD.UW, SH2ADD, SH2ADD.UW, SH3ADD, SH3ADD.UW, SLLI.UW	1	
Zbb Extension		**C_USE_BITMAN_B = 1**
ANDN, CLZ, CLZW, CPOP, CPOPW, CTZ, CTZW, ORC.B, ORN, REV8, ROL, ROLW, ROR, RORL, RORIW, RORW	1	
MAX, MAXU, MIN, MINU	1	4-, 5-, 8-stage pipeline
	2	3-stage pipeline
SEXT.B, SEXT.H, XNOR, ZEXT.H	1	
Zbc Extension		**C_USE_BITMAN_C = 1**
CLMUL, CLMULH, CLMULR	32	
Zbs Extension		**C_USE_BITMAN_S = 1**
BCLR, BCLRI, BEXT, BEXTI, BINV, BINVI, BSET, BSETI	1	
Zicbom Extension		**C_USE_ICACHE = 1 or C_USE_DCACHE = 1**
CBO.INVAL, CBO.CLEAN, CBO.FLUSH[2]	2 + N	C_INTERCONNECT = 2 (AXI)
	8 + N	C_INTERCONNECT = 3 (ACE)

Notes:

1. Latency is increased with the shift amount when C_USE_BARREL = 2 (Area), but only for 3-, 4-, 5-stage pipeline when Zba or Zbs extension is not enabled and 32-bit implementation is selected.
2. N is the number of cycles to wait for memory accesses to complete.
3. Latency depends on the accessed memory interface width.
4. Latency is 1 when the denominator is 0. When `C_USE_MULDIV` = 2, division and remainder are optimized to reduce the latency for byte operands with 24 cycles and for short operands with 16 cycles. For RV64, the optimization is only applied if the 32 most significant bits are zero.
5. If any of the operands is a subnormal floating-point value, the latency is increased by one cycle.

Custom Instructions

MicroBlaze V provides four custom instructions that provide a method to get and put data on AXI4-Stream links. These links can be used as generic input and output interfaces, but can also interface directly with an external accelerator that extends the processor functionality.

Additionally, MicroBlaze V provides eight custom instructions to support extended address load and store. These instructions are only available with the 32-bit implementation (RV32), when physical memory protection or Supervisor mode is not enabled.

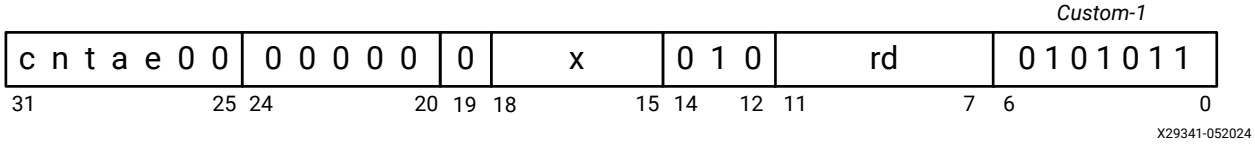
Each custom instruction is described in the following sections, with mnemonics and semantics compatible with the equivalent instructions in the classic MicroBlaze processor.

Software macros compatible with the classic MicroBlaze processor are provided in the standalone BSP to enable you to use the custom instructions.

GET

Figure 2: **Get from Stream Interface**

*tnea*get	rd, x	Get data from link x t = Test only n = Non-blocking e = Exception if control bit set a = Atomic
*tneca*get	rd, x	Get control from link x t = Test only n = Non-blocking e = Exception if control bit not set a = Atomic



Description

MicroBlaze V reads from the link x interface and places the result in register rD.

The GET instruction has 32 variants.

The blocking versions (when the n bit is 0) stall until the data from the interface is valid. The non-blocking versions do not stall, and set the C bit in mstream to 0 if the data is valid and to 1 if the data is invalid. In case of an invalid access, the destination register contents are undefined.

All data GET instructions (when the c bit is 0) expect the control bit from the interface to be 0. If this is not the case, the instruction sets the FSL bit in mstream to 1. All control GET instructions (when the c bit is 1) expect the control bit from the interface to be 1. If this is not the case, the instruction sets the FSL bit in mstream to 1. The FSL bit is only set when valid data is transferred.

The exception versions (when the e bit is 1) generate an exception if there is a control bit mismatch. The destination register is not updated when an exception is generated.

The test versions (when the t bit is 1) are handled as normal, except that the read signal to the link is not asserted.

The atomic versions (when the a bit is 1) cannot be interrupted by a machine external interrupt or external break. Each atomic instruction prevents the subsequent instruction from being interrupted. This means that a sequence of atomic instructions can be grouped together without an interrupt breaking the program flow. However, exceptions might still occur.

If the available number of links set by C_FSL_LINKS is less than or equal to x, an illegal instruction exception occurs when C_ILL_INSTR_EXCEPTION = 2, otherwise the instruction behaves like a NOP.

Pseudocode

```
if x >= C_FSL_LINKS then
   Sx_AXIS_TVALID ← 0
   if C_ILL_INSTR_EXCEPTION = 2 then
      PC ← mtvec
      mcause ← 2
else
   (rD) ← Sx_AXIS_TDATA
   if (n = 1) then _______________
      mstream.C ← Sx_AXIS_TVALID
   if Sx_AXIS_TLAST ≠ c and Sx_AXIS_TVALID then
      mstream.FSL ← 1
      if (e = 1) then
         PC ← mtvec
         mcause ← 24
```

Registers Altered

- rD, unless an exception is generated, in which case the register is unchanged
- mstream
- stream when C_USE_MMU > 0
- PC and mcause, in case a stream or illegal instruction exception is generated

Latency

- One cycle with C_OPTIMIZATION = 0, 2, 3
- Two cycles with C_OPTIMIZATION = 1

The blocking versions of this instruction stall the pipeline until the instruction can be completed. Interrupts are served when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1, and the instruction is not atomic.

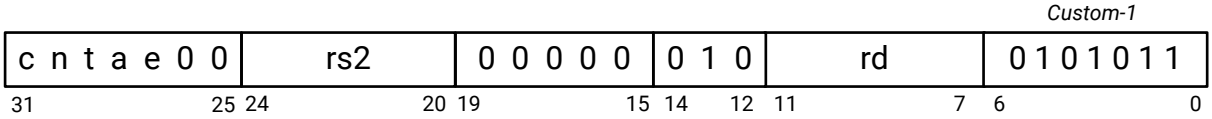
Notes

- With RV64, 32-bit data read from the link is zero/extended.
- The e bit does not have any effect unless C_FSL_EXCEPTION is set to 1.
- These instructions are only available when the parameter C_FSL_LINKS is greater than 0.
- The extended instructions (exception, test, and atomic versions) are only available when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1.
- Instruction bits 19-26 are ignored when decoding the instruction.

GETD

Figure 3: **Get from Stream Interface Dynamic**

*tnea*getd	rd, rs2	Get data from link rs2[3:0] t = Test only n = Non-blocking e = Exception if control bit set a = Atomic
*tneca*getd	rd, rs2	Get control from link rs2[3:0] t = Test only n = Non-blocking e = Exception if control bit not set a = Atomic



Description

MicroBlaze V reads from the link defined by the four least significant bits in rs2 and places the result in register rD.

The GETD instruction has 32 variants.

The blocking versions (when the n bit is 0) stall until the data from the interface is valid. The non-blocking versions do not stall, and set the C bit in mstream to 0 if the data is valid and to 1 if the data is invalid. In case of an invalid access, the destination register contents are undefined.

All data GETD instructions (when the c bit is 0) expect the control bit from the interface to be 0. If this is not the case, the instruction sets the FSL bit in mstream to 1. All control GETD instructions (when the c bit is 1) expect the control bit from the interface to be 1. If this is not the case, the instruction sets the FSL bit in mstream to 1. The FSL bit is only set when valid data is transferred.

The exception versions (when the e bit is 1) generate an exception if there is a control bit mismatch. The destination register is not updated when an exception is generated.

The test versions (when the t bit is 1) are handled as normal, except that the read signal to the link is not asserted.

The atomic versions (when the a bit is 1) cannot be interrupted by a machine external interrupt or external break. Each atomic instruction prevents the subsequent instruction from being interrupted. This means that a sequence of atomic instructions can be grouped together without an interrupt breaking the program flow. However, exceptions might still occur.

If the available number of links set by C_FSL_LINKS is less than or equal to the four least significant bits in rs2, an illegal instruction exception occurs when C_ILL_INSTR_EXCEPTION = 2, otherwise the instruction behaves like a NOP.

Pseudocode

```
x ← rs2[3:0]
if x >= C_FSL_LINKS then
   if C_ILL_INSTR_EXCEPTION = 2 then
      PC ← mtvec
      mcause ← 2
else
   (rD) ← Sx_AXIS_TDATA
   if n = 1 then
      mstream.C ← Sx_AXIS_TVALID
   if Sx_AXIS_TLAST ≠ c and Sx_AXIS_TVALID then
      mstream.FSL ← 1
      if (e = 1) then
         PC ← mtvec
         mcause ← 24
```

Registers Altered

- rD, unless an exception is generated, in which case the register is unchanged.
- mstream
- stream when C_USE_MMU > 0
- PC and mcause, in case a stream or illegal instruction exception is generated.

Latency

- One cycle with C_OPTIMIZATION = 0, 2, 3
- Two cycles with C_OPTIMIZATION = 1

The blocking versions of this instruction stall the pipeline until the instruction can be completed. Interrupts are served when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1, and the instruction is not atomic.

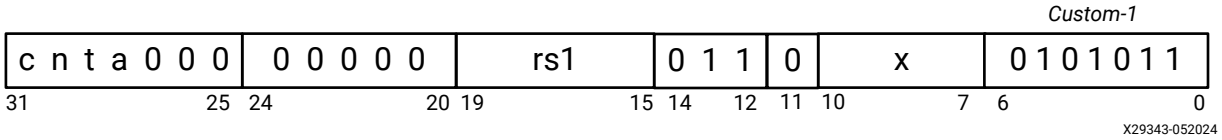
Notes

- With RV64, 32-bit data read from the link is zero/extended.
- The e bit does not have any effect unless C_FSL_EXCEPTION is set to 1.
- These instructions are only available when the parameter C_FSL_LINKS is greater than 0.
- The extended instructions (exception, test, and atomic versions) are only available when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1.
- Instruction bits 15-19 and 25-26 are ignored when decoding the instruction.

PUT

Figure 4: **Put to Stream Interface**

*na*put	rs1, x	Put data to link x n = Non-blocking a = Atomic
t*na*put	x	Put data to link x test only n = Non-blocking a = Atomic
*nc*aput	rs1, x	Put control to link x n = Non-blocking a = Atomic
t*nc*aput	x	Put control to link x test only n = Non-blocking a = Atomic



Description

MicroBlaze V writes the value from register rs1 to the link x interface.

The PUT instruction has 16 variants.

The blocking versions (when n is 0) stall until there is space available in the interface. The non-blocking versions do not stall, and set the C bit in mstream to 0 if space is available, and to 1 if no space is available.

Except for the test versions, all data PUT instructions (when c is 0) set the control bit to the interface to 0. All control PUT instructions (when c is 1) set the control bit to 1.

The test versions (when the t bit is 1) are handled as normal, except that the data and control bit are not output and the write signal to the link is not asserted, and therefore no source register is required, and link data is not assigned. The test version requires that rs1 is x0.

The atomic versions (when the a bit is 1) cannot be interrupted by a machine external interrupt or external break. Each atomic instruction prevents the subsequent instruction from being interrupted. This means that a sequence of atomic instructions can be grouped together without an interrupt breaking the program flow.

If the available number of links set by C_FSL_LINKS is less than or equal to x, an illegal instruction exception occurs when C_ILL_INSTR_EXCEPTION = 2, otherwise the instruction behaves like a NOP.

Pseudocode

```
if x >= C_FSL_LINKS then
   if C_ILL_INSTR_EXCEPTION = 2 then
      PC ← mtvec
      mcause ← 2
else
   if t = 0 then
      Mx_AXIS_TDATA ← (rs1)
   if n = 1 then
      mstream.C ← Mx_AXIS_TVALID ∧ Mx_AXIS_TREADY
   if t = 0 then
      Mx_AXIS_TLAST ← c
```

Registers Altered

- mstream
- stream when C_USE_MMU > 0
- PC and mcause, in case an illegal instruction exception is generated

Latency

- One cycle with C_OPTIMIZATION= 0, 2, 3
- Two cycles with C_OPTIMIZATION= 1

The blocking versions of this instruction stall the pipeline until the instruction can be completed. Interrupts are served when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1, and the instruction is not atomic.

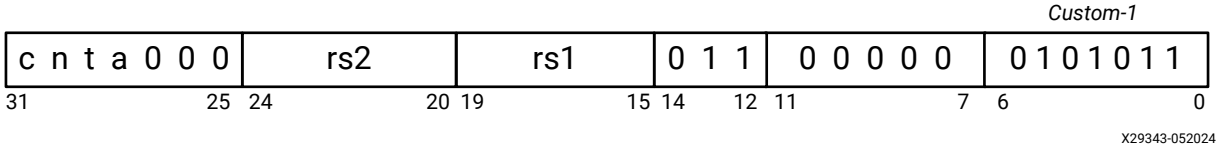
Notes

- These instructions are only available when the parameter C_FSL_LINKS is greater than 0.
- The extended instructions (test and atomic versions) are only available when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1.
- Instruction bits 11 and 20-27 are ignored when decoding the instruction.
- For the test version, rs1 is ignored when decoding the instruction.

PUTD

Figure 5: **Put to Stream Interface Dynamic**

*na*putd	rs1, rs2	Put data to link rs2[3:0] n = Non-blocking a = Atomic
t*na*putd	rs2	Put data to link rs2[3:0] test only n = Non-blocking a = Atomic
*nca*putd	rs1, rs2	Put control to link rs2[3:0] n = Non-blocking a = Atomic
t*nca*putd	rs2	Put control to link rs2[3:0] test only n = Non-blocking a = Atomic



Description

MicroBlaze V writes the value from register rs1 to the link interface defined by the four least significant bits of rs2.

The PUTD instruction has 16 variants.

The blocking versions (when n is 0) stall until there is space available in the interface. The non-blocking versions do not stall, and set the C bit in mstream to 0 if space is available, and to 1 if no space is available.

Except for the test versions, all data PUTD instructions (when c is 0) set the control bit to the interface to 0. All control PUTD instructions (when c is 1) set the control bit to 1.

The test versions (when the t bit is 1) are handled as normal, except that the data and control bit are not output and the write signal to the link is not asserted, and thus no source register is required, and link data is not assigned. The test version requires that rs1 is x0.

The atomic versions (when the a bit is 1) cannot be interrupted by a machine external interrupt or external break. Each atomic instruction prevents the subsequent instruction from being interrupted. This means that a sequence of atomic instructions can be grouped together without an interrupt breaking the program flow.

If the available number of links set by C_FSL_LINKS is less than or equal to the four least significant bits of rs2, an illegal instruction exception occurs when C_ILL_INSTR_EXCEPTION = 2, otherwise the instruction behaves like a NOP.

Pseudocode

```
x ← rs2[3:0]
if x >= C_FSL_LINKS then
   if C_ILL_INSTR_EXCEPTION = 2 then
      PC ← mtvec
      mcause ← 2
else
if t = 0 then
   Mx_AXIS_TDATA ← (rs1)
if n = 1 then                          ____________
   mstream.C ← Mx_AXIS_TVALID ∧ Mx_AXIS_TREADY
if t = 0 then
   Mx_AXIS_TLAST ← c
```

Registers Altered

- mstream
- stream whenC_USE_MMU > 0
- PC and mcause, in case an illegal instruction exception is generated

Latency

- One cycle with C_OPTIMIZATION= 0, 2, 3
- Two cycles with C_OPTIMIZATION = 1

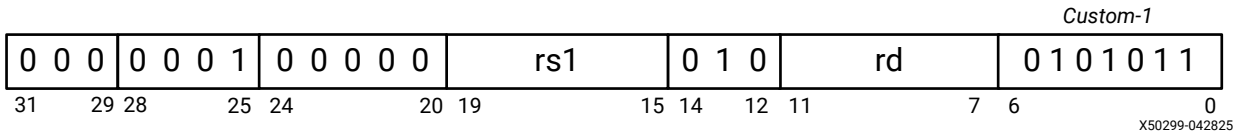
The blocking versions of this instruction stall the pipeline until the instruction can be completed. Interrupts are served when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1, and the instruction is not atomic.

Notes

- These instructions are only available when the parameter C_FSL_LINKS is greater than 0.
- The extended instructions (test and atomic versions) are only available when the parameter C_USE_EXTENDED_FSL_INSTR is set to 1.
- Instruction bits 7-11 and 25-27 are ignored when decoding the instruction.
- For the test version, rs1 is ignored when decoding the instruction.

LBEA

Figure 6: **Load Byte Extended Address**



Description

Loads a byte (8 bits) from the memory location that results from the effective address formed by concatenating the contents of registers rs1 and rs1+1. The sign-extended data is placed in register rd. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A load access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when reading data from memory.

A load page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
(rd)[24:31] ← Mem(Addr)[24:31]
(rd)[0:23] ← Mem(Addr)[24]
```

Registers Altered

rd, unless a trap occurs, in which case, the register is unchanged

Latency

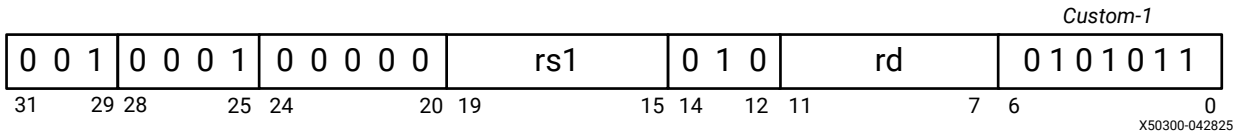
- One cycle with `C_OPTIMIZATION` = 0, 2, 3
- Two cycles with `C_OPTIMIZATION` = 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 20-24 and 26-28 are ignored when decoding the instruction.

LHEA

Figure 7: **Load Halfword Extended Address**



Description

Loads a halfword (16 bits) from the halfword aligned memory location that results from the effective address formed by concatenating the contents of registers rs1 and rs1+1. The sign-extended data is placed in register rd. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A load address misaligned exception occurs when the effective address has the least significant bit set.

A load access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when reading data from memory.

A load page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
(rd)[16:31] ← Mem(Addr)[16:31]
(rd)[0:15] ← Mem(Addr)[16]
```

Registers Altered

rd, unless a trap occurs, in which case the register is unchanged

Latency

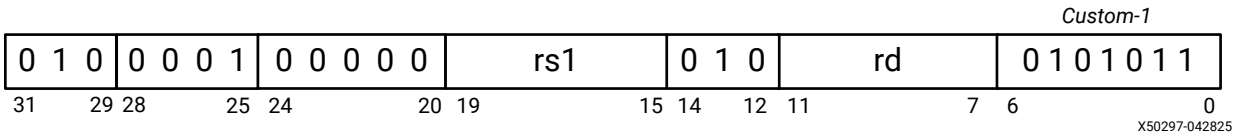
- One cycle with `C_OPTIMIZATION` = 0, 2, 3
- Two cycles with `C_OPTIMIZATION` = 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 20-24 and 26-28 are ignored when decoding the instruction.

LWEA

Figure 8: **Load Word Extended Address**



Description

Loads a word (32 bits) from the word aligned memory location that results from the effective address formed by concatenating the contents of registers rs1 and rs1+1. The data is placed in register rd. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A load address misaligned exception occurs when the effective address has any of the two least significant bits set.

A load access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when reading data from memory.

A load page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
(rd) ← Mem(Addr)
```

Registers Altered

rd, unless a trap occurs, in which case the register is unchanged

Latency

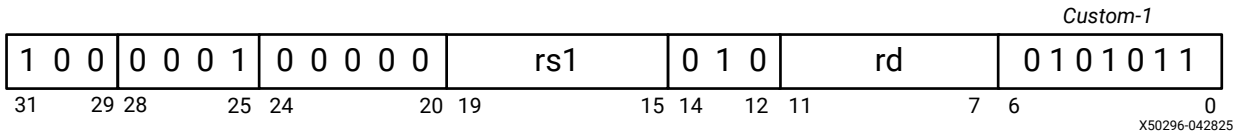
- One cycle with `C_OPTIMIZATION` = 0, 2, 3
- Two cycles with `C_OPTIMIZATION` = 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 20-24 and 26-28 are ignored when decoding the instruction.

LBUEA

Figure 9: **Load Byte Unsigned Extended Address**



Description

Loads a byte (8 bits) from the memory location that results from the effective address formed by concatenating the contents of registers rs1 and rs1+1. The zero-extended data is placed in register rd. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A load access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when reading data from memory.

A load page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
(rd)[24:31] ← Mem(Addr)[24:31]
(rd)[0:23] ← 0
```

Registers Altered

rd, unless a trap occurs, in which case the register is unchanged

Latency

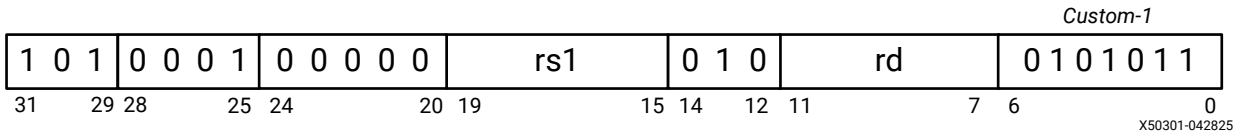
- One cycle with `C_OPTIMIZATION` = 0, 2, 3
- Two cycles with `C_OPTIMIZATION` = 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 20-24 and 26-28 are ignored when decoding the instruction.

LHUEA

Figure 10: **Load Halfword Unsigned Extended Address**



Description

Loads a halfword (16 bits) from the halfword aligned memory location that results from the effective address formed by concatenating the contents of registers rs1 and rs1+1. The zero-extended data placed in register rd. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A load address misaligned exception occurs when the effective address has the least significant bit set.

A load access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when reading data from memory.

A load page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
(rd)[16:31] ← Mem(Addr)[16:31]
(rd)[0:23] ← 0
```

Registers Altered

rd, unless a trap occurs, in which case the register is unchanged

Latency

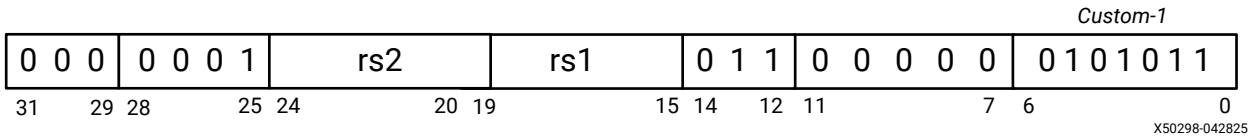
- One cycle with `C_OPTIMIZATION` = 0, 2, 3
- Two cycles with `C_OPTIMIZATION` = 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 20-24 and 26-28 are ignored when decoding the instruction.

SBEA

Figure 11: **Store Byte Extended Address**



Description

Stores the least significant byte of register rs2, into the byte memory location accessed by the effective address formed by concatenating the contents of registers rs1 and rs1+1. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A store/AMO access fault exception occurs in the case of an unsuccessful Physical Memory Access (PMA) check, or in the case of errors when writing data to memory.

A store/AMO page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
Mem(Addr) ← (rs2)[24:31]
```

Registers Altered

None

Latency

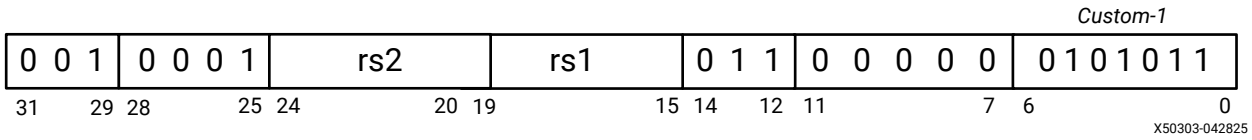
- One cycle with `C_OPTIMIZATION`= 0, 2, 3
- Two cycles with `C_OPTIMIZATION`= 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 7-11, 26-28 and 31 are ignored when decoding the instruction.

SHEA

Figure 12: **Store Halfword Extended Address**



Description

Stores the least significant halfword of register rs2, into the halfword aligned memory location accessed by the effective address formed by concatenating the contents of registers rs1 and rs1+1. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A store/AMO address misaligned exception occurs when the effective address has the least significant bit set.

A store/AMO access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when writing data to memory.

A store/AMO page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
Mem(Addr) ← (rs2)[16:31]
```

Registers Altered

None

Latency

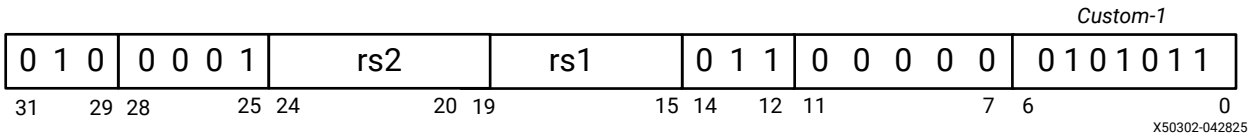
- One cycle with `C_OPTIMIZATION` = 0, 2, 3
- Two cycles with `C_OPTIMIZATION` = 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 7-11, 26-28 and 31 are ignored when decoding the instruction.

SWEA

Figure 13: **Store Word Extended Address**



Description

Stores the contents of register rs2, into the word aligned memory location accessed by the effective address formed by concatenating the contents of registers rs1 and rs1+1. The rs1 register number must be even. Use of a misaligned (odd-numbered) register is reserved.

A store/AMO address misaligned exception occurs when the effective address has any of the two least significant bits set.

A store/AMO access fault exception occurs in case of an unsuccessful Physical Memory Access (PMA) check, or in case of errors when writing data to memory.

A store/AMO page fault exception is caused by an unsuccessful virtual memory effective address translation.

Pseudocode

```
Addr ← (rs1 + 1) (rs1)
Mem(Addr) ← (rs2)
```

Registers Altered

None

Latency

- One cycle with `C_OPTIMIZATION`= 0, 2, 3
- Two cycles with `C_OPTIMIZATION`= 1

Notes

- The instruction is only valid if MicroBlaze V is configured to use extended address with PMP disabled (C_ADDR_SIZE > 32, C_PMP_ENTRIES = 0) and with RV32I Base Integer Instruction Set (C_DATA_SIZE = 32).
- Instruction bits 7-11, 26-28 and 31 are ignored when decoding the instruction.

Registers

MicroBlaze V implements registers according to the RISC-V Instruction Set Manual. All mandatory registers are implemented.

Implemented control and status registers (CSRs) are described here. For information on the general purpose registers and floating-point registers, see the RISC-V Instruction Set Manual.

Table 3: **Control and Status Registers**

Name	Number	Reset Value	Access	Remark
fflags	0x001	0x00000000	RW	Exists if floating-point is enabled.
frm	0x002	0x00000000	RW	Exists if floating-point is enabled.
fcsr	0x003	0x00000000	RW	Exists if floating-point is enabled.
sstatus	0x100	-	RW	Exists if supervisor mode is enabled. Reset value depends on if floating-point is enabled. See Reset for details.
sie	0x104	0x00000000	RO	Exists if supervisor mode is enabled.
stvec	0x105	C_BASE_VECTORS+4	RW	Exists if supervisor mode is enabled.
scounteren	0x106	0x00000000	RW	Exists if supervisor mode is enabled.
senvcfg	0x10a	0x00000000	RW	Exists if supervisor mode is enabled.
sscratch	0x140	0x00000000	RW	Exists if supervisor mode is enabled.
sepc	0x141	0x00000000	RW	Exists if supervisor mode is enabled.
scause	0x142	0x00000000	RW	Exists if supervisor mode is enabled.
stval	0x143	0x00000000	RW	Exists if supervisor mode is enabled.
sip	0x144	0x00000000	RO	Exists if supervisor mode is enabled.
satp	0x180	0x00000000	RW	Exists if supervisor mode is enabled.
mstatus	0x300	-	RW	Reset value depends on if user mode, supervisor mode, and floating-point are enabled. See Reset for details.
misa	0x301	-	RO	Reset value depends on enabled extensions and modes. See Reset for details.
medeleg	0x302	0x00000000	RW	Exists if supervisor mode is enabled.
mideleg	0x303	0x00000000	RO	Exists if supervisor mode is enabled.
mie	0x304	0x00000000	RW	
mtvec	0x305	C_BASE_VECTORS+4	RW	
mcounteren	0x306	0x00000000	RW	Exists if user mode is enabled.
menvcfg	0x30a	0x00000000	RW	Exists if user mode is enabled.
mcountinhibit	0x320	0x00000000	RW	
mhpmevent	0x323 - 0x323+N	0x00000000 or 0x0000000A	RW	Read-only zero unless debug event or latency counters are enabled. Write is ignored when read-only. Reset value is 0x0000000A for debug latency counters. N = 0 to 28.

Table 3: **Control and Status Registers** *(cont'd)*

Name	Number	Reset Value	Access	Remark
mscratch	0x340	0x00000000	RW	
mepc	0x341	0x00000000	RW	
mcause	0x342	0x00000000	RW	
mtval	0x343	0x00000000	RW	
mip	0x344	-	RO	Reset value depends on inputs
pmpcfg	0x3a0 - 0x3a0+N	0x00000000	RW	Exists if PMP entries > 0. N = 0 to 15 for RV32, and N = 0, 2, ..., 14 for RV64.
pmpaddr	0x3b0 - 0x3b0+N	0x00000000	RW	Exists if PMP entries > 0. N = 0 to 63.
mseccfg	0x747	0x00000000	RW	Exists if PMP entries > 0 and PMP Enhancements are enabled.
tselect	0x7a0	0x00000000	RW	Exists if debug is enabled.
tdata1	0x7a1	0x00000000	RW	Exists if debug is enabled.
tdata2	0x7a2	0x00000000	RW	Exists if debug is enabled.
tinfo	0x7a3	0x00000005	RO	Exists if debug is enabled.
dcsr	0x7b0	0x40000603	RW	Exists if debug is enabled, only accessible in debug mode
dpc	0x7b1	C_BASE_VECTORS	RW	Exists if debug is enabled, only accessible in debug mode
mstream	0x7c0	0x00000000	RW	Custom CSR, exists if one or more AXI4-Streams are enabled
mwfi	0x7c4	0x00000000	RW	Custom CSR, exists if more than one of sleep, hibernate, and suspend outputs are enabled
stream	0x8c0	0x00000000	RW	Custom CSR, exists if one or more AXI4-Streams are enabled, user mode is enabled, and stream instructions are allowed
mcycle	0xb00	0x00000000	RW	Read-only zero unless base counters and timers are enabled.
minstret	0xb02	0x00000000	RW	Read-only zero unless base counters and timers are enabled.
mhpmcounter	0xb03 - 0xb03+N	0x00000000 or 0xFFFF0000	RW	Read-only zero unless debug event or latency counters are enabled, and base counters and timers are enabled. Reset value is 0xFFFF0000 for debug latency counter min/max registers. N = 0 to 28.
mcycleh	0xb80	0x00000000	RW	Read-only zero unless base counters and timers are enabled.
minstreth	0xb82	0x00000000	RW	Read-only zero unless base counters and timers are enabled.
mhpmcounterh	0xb83 - 0xb83+N	0x00000000	RW	Read-only zero unless debug event or latency counters are enabled, and base counters and timers are enabled. N = 0 to 28. Debug latency counter min/max registers are read-only zero.
cycle	0xc00	0x00000000	RO	Read-only zero unless base counters and timers are enabled.

Table 3: **Control and Status Registers** *(cont'd)*

Name	Number	Reset Value	Access	Remark
instret	0xc02	0x00000000	RO	Read-only zero unless base counters and timers are enabled.
hpmcounter	0xc03 - 0xc03+N	0x00000000 or 0xFFFF0000	RO	Zero unless debug event or latency counters are enabled, and base counters and timers are enabled. Reset value is 0xFFFF0000 for debug latency counter min/max registers. Write is ignored. N = 0 to 28.
cycleh	0xc80	0x00000000	RO	Read-only zero unless base counters and timers are enabled.
instreth	0xc82	0x00000000	RO	Read-only zero unless base counters and timers are enabled.
hpmcounterh	0xc83 - 0xc83+N	0x00000000	RO	Zero unless debug event or latency counters are enabled, and base counters and timers are enabled. Write is ignored. N = 0 to 28. Debug latency counter min/max registers are read-only zero.
mvendorid	0xF11	0x00000049	RO	
marchid	0xF12	C_ARCHID	RO	
mimpid	0xF13	C_IMPID	RO	
mhartid	0xF14	C_HARTID	RO	
mconfigptr	0xF15	0x00000000	RO	Implemented, but zero

For CSRs with optional, implementation defined, features and for custom CSRs, descriptions are provided. For other registers, see the RISC-V Instruction Set Manual.

Supervisor Status Register (sstatus)

The sstatus register is a read/write register formatted as shown in the following figure for RV32 and RV64. The sstatus register is a subset of the mstatus register. The register only exists when supervisor mode is enabled (`C_USE_MMU` > 2).

Figure 14: **Supervisor Status Register Formatted for RV32**

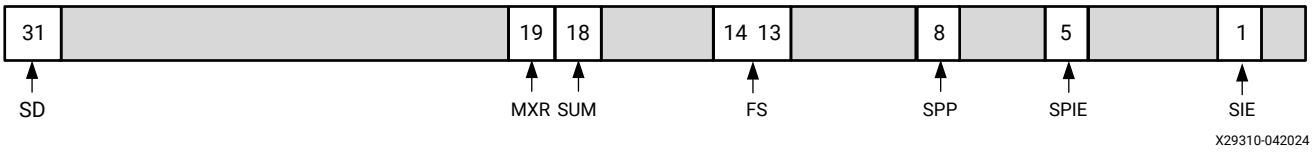
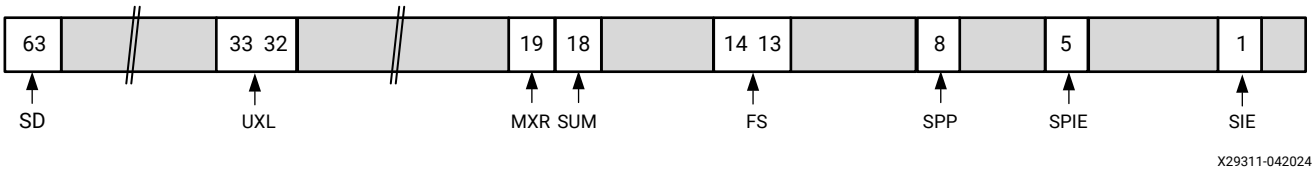


Figure 15: **Supervisor Status Register Formatted for RV64**



Detailed bit field information is listed in the description of the Machine Status register.

Supervisor Cause Register (scause)

The scause register is a read/write register. When a trap is taken into S-mode, scause is written with a code indicating the event that caused the trap. The register only exists when supervisor mode is enabled (C_USE_MMU > 2).

The code uses four bits, unless extended AXI4-Stream custom instructions with exception support are enabled (C_FSL_LINKS > 0, C_EXTENDED_FSL_INSTR > 0, and C_FSL_EXCEPTION > 0), in which case it uses five bits.

Figure 16: **Supervisor Cause Register**

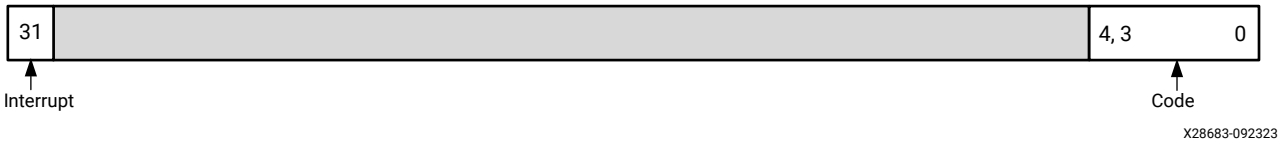


Table 4: **Supervisor Cause Register**

Bits	Name	Description	Reset Value
63, 31	Interrupt	Set if the trap was caused by an interrupt	0
4:0, 3:0	Code	Contains a code identifying the last exception or interrupt	00000, 0000

The processor supported interrupt and exception codes are listed in the description of the Machine Cause register.

Supervisor Trap Vector Base Address Register (stvec)

The stvec register is a read/write register that holds trap vector configuration, consisting of a vector base address (BASE) and a read-only vector mode (MODE). The register only exists when supervisor mode is enabled (C_USE_MMU > 2).

Figure 17: **Supervisor Trap-Vector Base-Address Register**

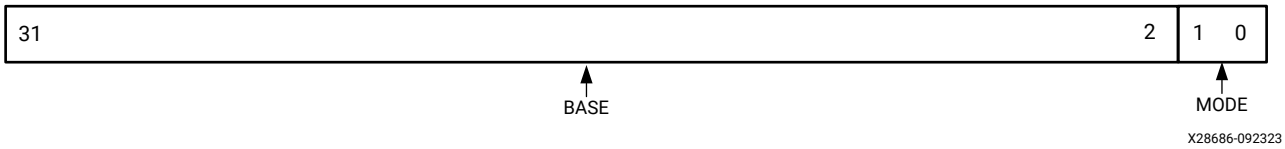


Table 5: **Supervisor Trap-Vector Base-Address Register**

Bits	Name	Description	Reset Value
31:2	BASE	Vector base address	(C_BASE_VECTORS + 4) / 4
1:0	MODE	Direct: All traps set PC to BASE. Read only.	00

Because MODE is read-only zero, the supervisor trap handler code must be aligned on a 32-bit word boundary.

With compressed instructions enabled (`C_USE_COMPRESSION` > 0), software must use an align attribute to ensure this is the case.

With the RISC-V GCC compiler, this can be achieved with the following code:

```
/* Ensure alignment on a word boundary */
void trap_handler(void) __attribute__((aligned(4)));
```

Supervisor Trap Value Register (stval)

The stval register is a read/write register. When a trap is taken into S-mode, stval is either set to zero or written with exception-specific information to assist software in handling the trap. The register only exists when supervisor mode is enabled (`C_USE_MMU` > 2).

The exception-specific information is listed in the description of the Machine Trap Value Register (mtval).

Supervisor Address Translation and Protection Register (satp)

The satp register is a read/write register, formatted as shown in the following figures for RV32 and RV64, which controls supervisor-mode address translation and protection. The register only exists when supervisor mode is enabled (`C_USE_MMU` > 2).

Figure 18: **Supervisor Address Translation and Protection Register Formatted for RV32**

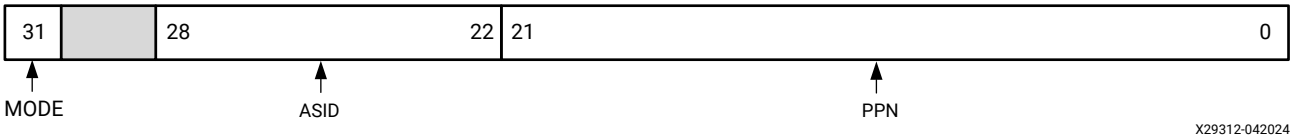


Figure 19: **Supervisor Address Translation and Protection Register Formatted for RV64**

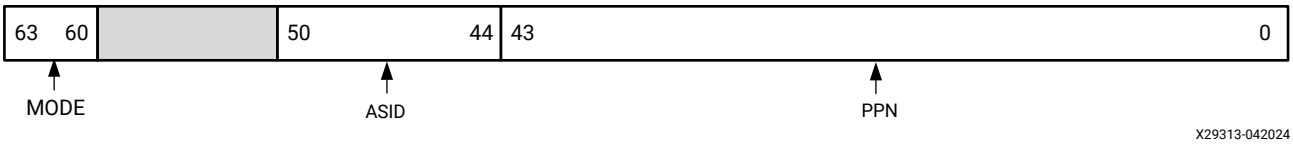


Table 6: **Supervisor Address Translation and Protection Register**

Bits	Name	Description	Reset Value
63:60,31	MODE	Mode: 0 = BARE, 1 = Sv32 (RV32), 8 = Sv39 (RV64)	0
50:44 28:22	ASID	Address´Space Identifier	0
43:0,21:0	PPN	Physical Page Number	0

Machine Status Register (mstatus)

The mstatus register is a read/write register formatted as shown in the following figures for RV32 and RV64. The mstatus register keeps track of and controls the current operating state of the hart.

Figure 20: **Machine Status Register Formatted for RV32**

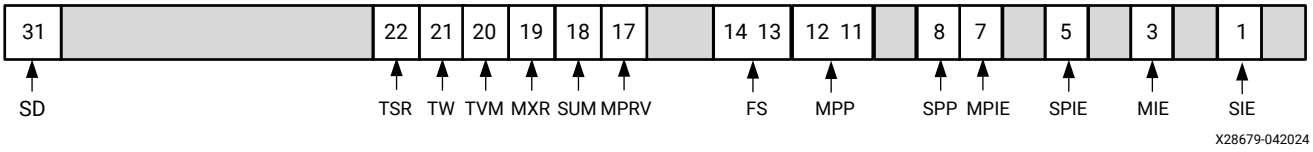


Figure 21: **Machine Status Register Formatted for RV64**

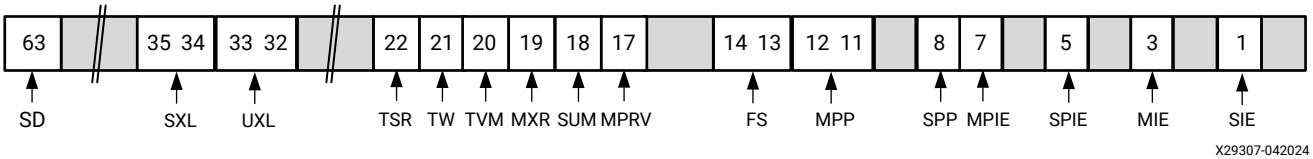


Table 7: **Machine Status Register**

Bits	Name	Description	Reset Value
63,31	SD	The SD bit is a read-only bit that indicates whether the FS field signals the presence of some dirty state that requires saving extended floating-point context to memory. Exists if floating-point is implemented (C_USE_FPU > 0), read-only 0 otherwise.	0: FS ≠ 11 1: FS = 11
35:34	SXL	The SXL field is a read-only field only available for RV64 that indicates that supervisor mode XLEN is 64 bits. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 00 otherwise.	10
33:32	UXL	The UXL field is a read-only field only available for RV64 that indicates that user mode XLEN is 64 bits. Exists if user mode is implemented (C_USE_MMU > 0), read-only 00 otherwise.	10
22	TSR	Trap SRET. When TSR = 1, if SRET is executed in supervisor mode, the instruction causes an illegal instruction exception. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0
21	TW	When TW=0, the WFI instruction might execute in user mode. When TW=1, if WFI is executed in user mode, the instruction causes an illegal instruction exception. Exists if user mode is implemented (C_USE_MMU > 0), read-only 0 otherwise.	0
20	TVM	Trap virtual memory. When TVM = 1, if SFENCE.VMA is executed or SATP is accessed in supervisor mode, an illegal instruction exception occurs. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0
19	MXR	Make eXecutable readable. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0
18	SUM	Permit supervisor user Memory access. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0
17	MPRV	The MPRV (modify privilege) bit modifies the effective privilege mode. Exists if user mode is implemented (C_USE_MMU > 0), read-only 0 otherwise.	0
14:13	FS	FS tracks the current state of the floating-point unit. Exists if floating-point is implemented (C_USE_FPU > 0), read-only 00 otherwise.	00: C_USE_FPU = 0 01: C_USE_FPU > 0
12:11	MPP	Machine previous privilege mode. Exists if user mode is implemented (C_USE_MMU > 0), read-only 11 otherwise.	11: C_USE_MMU = 0 00: C_USE_MMU > 0
8	SPP	Supervisor previous privilege mode. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0
7	MPIE	Previous MIE.	0
5	SPIE	Previous SIE. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0
3	MIE	Global interrupt enable for machine mode.	0

Table 7: **Machine Status Register** *(cont'd)*

Bits	Name	Description	Reset Value
1	SIE	Global interrupt enable for supervisor mode. Exists if supervisor mode is implemented (C_USE_MMU > 2), read-only 0 otherwise.	0

Machine ISA Register (misa)

The misa CSR is a read-only register reporting the ISA supported by the processor.

Figure 22: **Machine ISA Register**

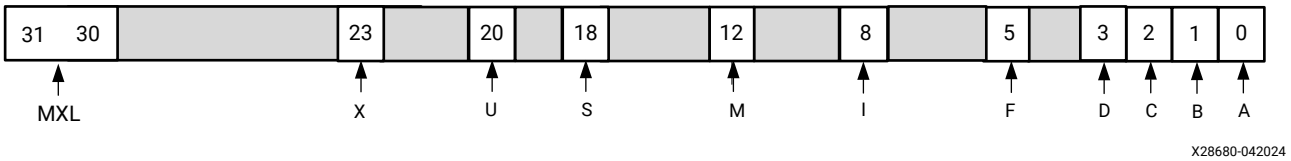


Table 8: **Machine ISA Register**

Bits	Name	Description	Reset Value
31:30	MXL	XLEN = 32 (C_DATA_SIZE = 32) or XLEN = 64 (C_DATA_SIZE = 64)	01 or 10
23	X	Set to 1 if custom instructions are implemented (C_FSL_LINKS > 0) or C_ADDR_SIZE > 32 for RV32	0 or 1
20	U	Set to 1 if user mode is implemented (C_USE_MMU = 1).	0 or 1
18	S	Set to 1 if supervisor mode is implemented (C_USE_MMU = 3).	
12	M	Set to 1 if multiplication and division (M-extension) are implemented (C_USE_MULDIV > 0).	0 or 1
8	I	RV32I or RV64I	1
5	F	Set to 1 if single-precision floating-point (F-extension) is implemented (C_USE_FPU > 0).	0 or 1
3	D	Set to 1 if double-precision floating-point (D-extension) is implemented (C_USE_FPU = 2).	0 or 1
2	C	Set to 1 if compressed instructions (C-extension) are implemented (C_USE_COMPRESSION > 0).	0 or 1
1	B	Set to 1 if the bit manipulation extension (B-extension) is implemented (C_USE_BITMAN_A = 1, C_USE_BITMAN_B = 1, C_USE_BITMAN_S = 1)	0 or 1
0	A	Set to 1 if atomic instructions (A-extension) is implemented (C_USE_ATOMIC > 0).	0 or 1

Machine Interrupt Registers (mip and mie)

The mip register is a read-only register containing information on pending interrupts, while mie is the corresponding read/write register containing interrupt enable bits.

Figure 23: **Machine External Break Pending (MEBP) and Machine External Interrupt Pending (MEIP)**

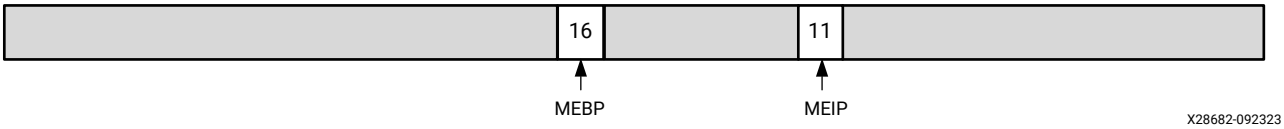


Table 9: **Machine Interrupt Pending Register**

Bits	Name	Description	Reset Value
16	MEBP	Machine external break pending. Custom platform interrupt, using the external break input.	Ext_Brk
11	MEIP	Machine external interrupt pending.	Interrupt

Figure 24: **Machine External Break Enable (MEBE) and Machine External Interrupt Enable (MEIE)**

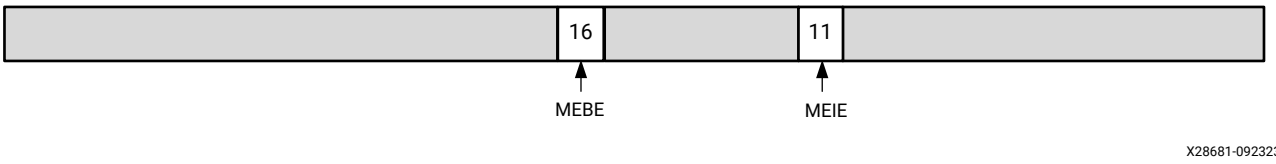


Table 10: **Machine Interrupt Enable Register**

Bits	Name	Description	Reset Value
16	MEBE	Machine external break enable. Custom platform interrupt, using the external break input.	0
11	MEIE	Machine external interrupt enable.	0

Machine Cause Register (mcause)

The mcause register is a read/write register. When a trap is taken into M-mode, mcause is written with a code indicating the event that caused the trap.

The code uses four bits, unless extended AXI4-Stream custom instructions with exception support are enabled (`C_FSL_LINKS` > 0, `C_EXTENDED_FSL_INSTR` > 0, and `C_FSL_EXCEPTION` > 0), in which case it uses five bits.

Figure 25: **Machine Cause Register**

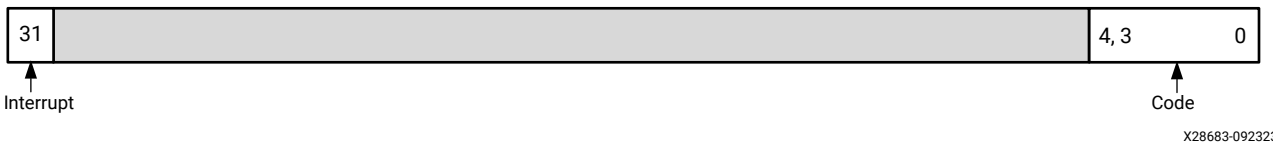


Table 11: **Machine Cause Register**

Bits	Name	Description	Reset Value
31	Interrupt	Set if the trap was caused by an interrupt	0
4:0, 3:0	Code	Contains a code identifying the last exception or interrupt	00000, 0000

The processor supports the following interrupt and exception codes:

Table 12: **Interrupt and Exception Codes**

Interrupt	Code	Description
1	0	Non-maskable interrupt. Occurs when the Ext_NM_BRK signal is pulsed. Exists when C_USE_EXT_NM_BRK > 0.
1	11	Machine external interrupt. Occurs when the Interrupt signal is activated. Exists when C_USE_INTERRUPT > 0.
1	16	Machine external break platform interrupt. Occurs when the Ext_BRK signal is set. Exists when C_USE_EXT_BRK > 0.
0	0	Instruction address misaligned. Exists when compressed instructions are not enabled (C_USE_COMPRESSION = 0).
0	1	Instruction access fault. • Caused by a bus error, either from an LMB uncorrectable error, an LMB correctable error when C_USE_ECC_CE_EXCEPTION > 0, an AXI4 slave error, or an AXI4 decode error. • Caused by an AXI4 OKAY response to a load-reserved (LR) or atomic (AMO) instruction load when exclusive accesses are enabled on the AXI4 interface. • Caused by a physical memory protection (PMP) check when PMP is enabled (C_PMP_ENTRIES > 0). Exists when C_M_AXI_I_BUS_EXCEPTION > 0 or C_PMP_ENTRIES > 0.
0	2	Illegal instruction. Exists when illegal instruction exceptions are enabled (C_ILL_INSTR_EXCEPTION > 0).
0	3	Breakpoint (EBREAK) Exists when debug is enabled (C_DEBUG_ENABLED > 0).
0	4	Load address misaligned
0	5	Load access fault. • Caused by a bus error, either from an LMB uncorrectable error, an LMB correctable error when C_USE_ECC_CE_EXCEPTION > 0, an AXI4 slave error, or an AXI4 decode error. A bus error can also occur during virtual memory translation page table lookup. • Caused by a physical memory protection (PMP) check when PMP is enabled (C_PMP_ENTRIES > 0). Exists when C_M_AXI_D_BUS_EXCEPTION > 0, C_PMP_ENTRIES > 0, or when supervisor mode is enabled (C_USE_MMU > 1).
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault. • Caused by a bus error, either from an LMB uncorrectable error, an LMB correctable error when C_USE_ECC_CE_EXCEPTION > 0, an AXI4 slave error, or an AXI4 decode error. • Caused by a physical memory protection (PMP) check when PMP is enabled (C_PMP_ENTRIES > 0). Exists when C_M_AXI_D_BUS_EXCEPTION > 0 or C_PMP_ENTRIES > 0.
0	8	Environment call from U-mode (ECALL). Exists when user mode is enabled (C_USE_MMU > 0).

Table 12: **Interrupt and Exception Codes** *(cont'd)*

Interrupt	Code	Description
0	9	Environment call from S-mode (ECALL). Exists when supervisor mode is enabled (C_USE_MMU > 1).
0	11	Environment call from M-mode (ECALL)
0	12	Instruction page fault. Exists when supervisor mode is enabled (C_USE_MMU > 1).
0	13	Load page fault. Exists when supervisor mode is enabled (C_USE_MMU > 1).
0	15	Store/AMO page fault. Exists when supervisor mode is enabled (C_USE_MMU > 1).
0	24	AXI4-Stream exception. Exists when extended AXI4-Stream custom instructions with exception support are enabled (C_FSL_LINKS > 0, C_EXTENDED_FSL_INSTR > 0, and C_FSL_EXCEPTION > 0.

Machine Trap-Vector Base-Address Register (mtvec)

The mtvec register is a read/write register that holds trap vector configuration, consisting of a vector base address (BASE) and a read-only vector mode (MODE).

Figure 26: **Machine Trap-Vector Base-Address Register**

Table 13: **Machine Trap-Vector Base-Address Register**

Bits	Name	Description	Reset Value
31:2	BASE	Vector base address	(C_BASE_VECTORS + 4) / 4
1:0	MODE	Direct: All traps set PC to BASE. Read only.	00

Because MODE is read-only zero, the trap handler code must be aligned on a 32-bit word boundary.

With compressed instructions enabled (`C_USE_COMPRESSION` > 0), software must use an align attribute to ensure this is the case.

With the RISC-V GCC compiler, this can be achieved with the following code:

```
/* Ensure alignment on a word boundary */
void trap_handler(void) __attribute__((aligned(4)));
```

Machine HPM Event Register (mhpmevent)

The hardware performance monitor (HPM) includes up to 29 64-bit event counters, mhpmcounter3 – mhpmcounter31. The event selector registers, mhpmevent3 – mhpmevent31, are read/write registers that control which event causes the corresponding counter to increment. If more than one event is enabled in the event register and two or more of the enabled events occur simultaneously, the counter only increments by one.

Each event selector register is divided into a class, and individual events. Within each class, one or more of the events can be counted. Only events that are defined within a class can be set.

The latency class requires using a counter implemented as a latency counter for sum and max/min, but should use an event counter to count the number of events. For this class, it is not recommended to count more than one event with each counter. The class of a latency counter is fixed to 5.

Event 0 is defined to mean “no event.”

If class is set to an undefined value or an individual event bit that is not defined for the class is set, the counter does not increment.

Figure 27: **Machine HPM Event Register**

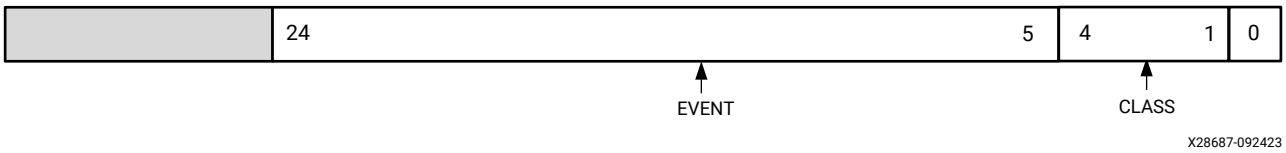


Table 14: **Machine HPM Event Register**

Bits	Name	Description	Reset Value
24:5	Event	Event mask. Defined for each event class.	0
4:1	Class	0 Retired instructions 1 Branches and traps 2 Instruction and data cache 3 Pipeline stalls 4 Miscellaneous 5 Latency	0
0	No Event	Set to represent no event	0

Table 15: **Event Class 0 – Retired Instructions**

Event Bit	Description
5	Integer load instruction retired
6	Integer store instruction retired
7	Atomic instruction retired

Table 15: **Event Class 0 – Retired Instructions** *(cont'd)*

Event Bit	Description
8	System instruction retired, including ECALL and EBREAK
9	Integer arithmetic instruction, including C.NOP retired
10	Integer multiply instruction retired
11	Integer divide/remainder instruction retired
12	Custom instruction retired
13	Bit manipulation instruction retired
14	Compressed instructions retired
15	JAL or C.J instruction retired
16	JALR or C.JR instruction retired
17	Floating point load instruction retired
18	Floating point store instruction retired
19	Floating point add/sub instruction retired
20	Floating point multiply instruction retired
21	Floating point divide instruction retired
22	Floating point fused instruction retired
23	Floating point other instruction retired
24	Cache invalidate or flush retired

Table 16: **Event Class 1 – Branches and Traps**

Event Bit	Description
5	Taken conditional branch
6	Not taken conditional branch
7	Exception taken
8	Interrupt occurred
9	Branch target cache hit
10	Branch target mispredict

Table 17: **Event Class 2 – Instruction and Data Cache**

Event Bit	Description
5	Data request from instruction cache
6	Hit in instruction cache
7	Read data requested from data cache
8	Read data hit in data cache
9	Write data request from data cache
10	Write data hit in data cache

Table 18: **Event Class 3 – Pipeline Stalls**

Event Bit	Description
6	Pipeline stalled due to operand fetch stage (OF)
7	Pipeline stalled due to execute stage (EX)
8	Pipeline stalled due to memory stage: • 5-stage pipeline: MEM • 8-stage pipeline: M0, M1, M2, or M3

Table 19: **Event Class 4 – Miscellaneous**

Event Bit	Description
5	Divide/remainder by zero operation
6	Floating-point subnormal result

Table 20: **Event Class 5 – Latency**

Event Bit	Description
5	Interrupt: total sum Interrupt: max (31:16) and min (15:0)
7	Data cache memory read: total sum Data cache memory read: max (31:16) and min (15:0)
9	Data cache memory write: total sum Data cache memory write:max (31:16) and min (15:0)
11	Instruction cache memory read: total sum Instruction cache memory read: max (31:16) and min (15:0)
13	Peripheral AXI data read: total sum Peripheral AXI data read: max (31:16) and min (15:0)
15	Peripheral AXI data write: total sum Peripheral AXI data write: max (31:16) and min (15:0)

The number of event counters and event selector registers is set by `C_DEBUG_EVENT_COUNTERS` + 2 * `C_DEBUG_LATENCY_COUNTERS`. The lower registers are implemented as event counters, whereas the higher are implemented as pairs of latency counters, consisting of the latency sum and min/max latency. The selected event for a pair of latency counters is set in the corresponding event selector register for the first counter in the pair.

An example with `C_DEBUG_EVENT_COUNTERS` = 5 and `C_DEBUG_LATENCY_COUNTERS` = 2 illustrates how the event counters are allocated.

Table 21: **Event Counter Allocation**

Event Counter	Kind	Description
mhpmcounter3	Event Counters	Used to count events from Class 0 – 5.
mhpmcounter4		
mhpmcounter5		
mhpmcounter6		
mhpmcounter7		
mhpmcounter8	Latency Counter: Total sum	Used to count a latency event from Class 5. Use mhpmevent8 to set events for both counters.
mhpmcounter9	Latency Counter: Min/Max	
mhpmcounter10	Latency Counter: Total sum	Used to count a latency event from Class 5. Use mhpmevent10 to set events for both counters.
mhpmcounter11	Latency Counter: Min/Max	

Machine Trap Value Register (mtval)

The mtval register is a read/write register. When a trap is taken into M-mode, mtval is either set to zero or written with exception-specific information to assist software in handling the trap.

Table 22: **Machine Trap Value Register Exception-specific Information**

Code	Exception	Description
0	Instruction address misaligned	Set to the program counter of the misaligned access
2	Illegal instruction	Set to the instruction causing the exception
3	Breakpoint trap	Set to the program counter of the breakpoint
4	Load address misaligned	Set to the misaligned load address
5	Load access fault	Set to the address causing the access fault
6	Store/AMO address misaligned	Set to the misaligned store address
7	Store/AMO access fault	Set to the address causing the access fault
Other	Other events	Set to 0x00000000

Machine Security Configuration Register (mseccfg)

The mseccfg register is an optional read/write register that controls security features. It only exists when PMP is enabled (`C_PMP_ENTRIES` > 0) and PMP Enhancements for memory access and execution prevention on Machine mode (Smepmp) is enabled (`C_PMP_ENHANCEMENTS` > 0).

Figure 28: **Machine Security Configuration Register**

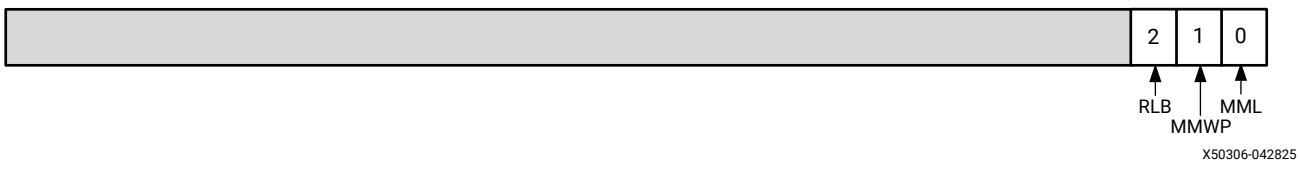


Table 23: **Machine Security Configuration Register**

Bits	Name	Description	Reset Value
2	RLB	Rule Locking Bypass	0
1	MMWP	Machine Mode Whitelist Policy	0
0	MML	Machine Mode Lockdown	0

Machine Stream Register (mstream)

The mstream register is a read/write custom CSR with the number 0x7C0. It holds information related to executed AXI4-Stream custom instructions. It only exists when AXI4-Stream custom instructions are enabled (`C_FSL_LINKS` > 0).

Figure 29: **Machine Stream Register**

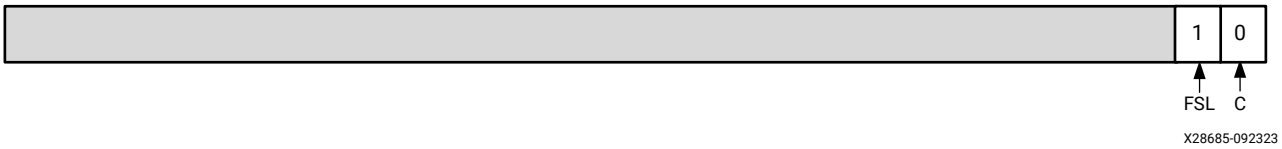


Table 24: **Machine Stream Register**

Bits	Name	Description	Reset Value
1	FSL	GET or GETD custom instruction TLAST control bit mismatch. A mismatch occurs when the custom instruction expects TLAST to be set (the instruction 'c' bit is set) and it is not, or vice versa. This bit is sticky, meaning that it is set by the first GET or GETD custom instruction with a mismatch, and remains set until cleared by software.	0
0	C	A non-blocking GET or GETD custom instruction sets this bit to 1 if no data is available and to 0 if valid data is read. A non-blocking PUT or PUTD custom instruction sets this bit to 1 if data could not be written and to 0 if data is successfully written. This bit represents the result of the last executed non-blocking custom instruction.	0

Machine WFI Register (mwfi)

The mwfi register is a read/write custom CSR with the number 0x7C4. It is used to determine the output signal that is set to indicate that the processor has executed a WFI instruction and is sleeping, which can either be `Sleep`, `Hibernate`, or `Suspend`. It only exists when more than one output signal is enabled (`C_USE_SLEEP` = 3, 5, 6, or 7).

Figure 30: **Machine WFI Register**

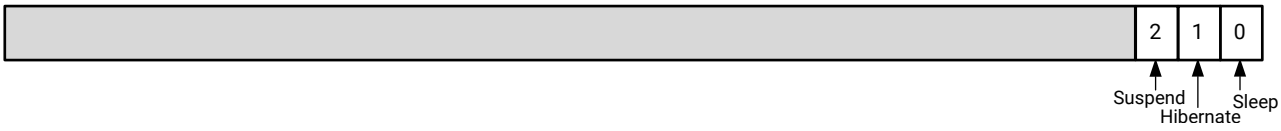


Table 25: **Machine WFI Register**

Bits	Name	Description	Reset Value
2	Suspend	The Suspend signal is set when a WFI instruction is executed.	0
1	Hibernate	The Hibernate signal is set when a WFI instruction is executed.	0
0	Sleep	The Sleep signal is set when a WFI instruction is executed.	0

TIP: *It is recommended to enable only one of the output signals by setting the register to 1, 2, or 4 to be compatible with MicroBlaze (that is, only one signal active at a time).*

Pipeline Architecture

MicroBlaze V instruction execution is pipelined. For most instructions, each stage takes one clock cycle to complete. Consequently, the number of clock cycles necessary for a specific instruction to complete is equal to the number of pipeline stages, and one instruction is completed on every cycle in the absence of data, control or structural hazards.

A data hazard occurs when the result of an instruction is needed by a subsequent instruction. This can result in stalling the pipeline, unless the result can be forwarded to the subsequent instruction.

A control hazard occurs when a branch is taken, and the next instruction is not immediately available. This results in stalling the pipeline. MicroBlaze V provides the optional branch target cache to reduce the number of stall cycles.

A structural hazard occurs for a few instructions that require multiple clock cycles in the execute stage or a later stage to complete. This is achieved by stalling the pipeline.

Load and store instructions accessing slower memory might take multiple cycles. The pipeline is stalled until the access completes. MicroBlaze V provides the optional data cache to improve the average latency of slower memory.

When executing from slower memory, instruction fetches might take multiple cycles. This additional latency directly affects the efficiency of the pipeline. MicroBlaze V implements an instruction prefetch buffer that reduces the impact of such multi-cycle instruction memory latency. While the pipeline is stalled for any other reason, the prefetch buffer continues to load sequential instructions speculatively. When the pipeline resumes execution, the fetch stage can load new instructions directly from the prefetch buffer instead of waiting for the instruction memory access to complete.

The local memory bus together with on-chip memory provides tightly-coupled memory with single cycle access, for maximum performance.

If instructions are modified during execution (for example with self-modifying code), the prefetch buffer should be emptied before executing the modified instructions, to ensure that it does not contain the old unmodified instructions. This can be achieved with the `fence.i` instruction.

MicroBlaze V also provides the optional instruction cache to improve the average instruction fetch latency of slower memory.

All hazards are independent, and can potentially occur simultaneously. In such cases, the number of cycles the pipeline is stalled is defined by the hazard with the longest stall duration.

3-Stage Pipeline

With `C_OPTIMIZATION` set to 1 (area), the pipeline is divided into three stages to minimize hardware cost: Fetch, Decode, and Execute.

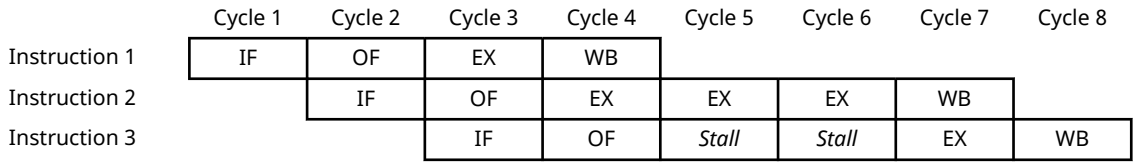
	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7
Instruction 1	Fetch	Decode	Execute				
Instruction 2		Fetch	Decode	Execute	Execute	Execute	
Instruction 3			Fetch	Decode	*Stall*	*Stall*	Execute

The 3-stage pipeline does not have any data hazards. Pipeline stalls are caused by control hazards, structural hazards due to multi-cycle instructions, memory accesses using slower memory, instruction fetch from slower memory, or stream accesses.

The multi-cycle instruction categories are multiply, divide, remainder, and floating-point instructions.

4-Stage Pipeline

With `C_OPTIMIZATION` set to 3 (throughput), the pipeline is divided into four stages to maximize computational throughput: fetch (IF), decode (OF), execute (EX), and writeback (WB).



The 4-stage pipeline has the following data hazard:

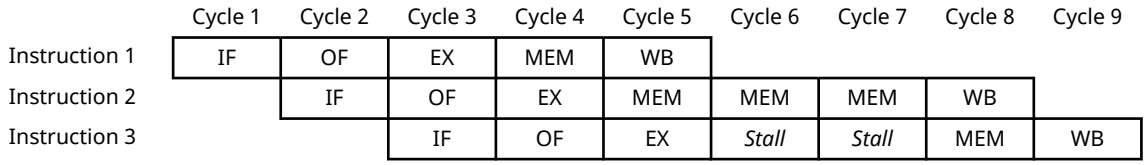
- An instruction in OF needs the result from an instruction in EX as a source operand. In this case, the EX instruction categories are load, store, multiply, divide, remainder, and floating-point instructions. This results in a one cycle stall.

Pipeline stalls are caused by data hazards, control hazards, structural hazards due to multi-cycle instructions, memory accesses using slower memory, instruction fetch from slower memory, or stream accesses.

The multi-cycle instruction categories are divide, remainder and floating-point instructions.

5-Stage Pipeline

With `C_OPTIMIZATION` set to 0 (performance), the pipeline is divided into five stages to maximize performance: fetch (IF), decode (OF), execute (EX), access memory (MEM), and writeback (WB).



The 5-stage pipeline has the following kinds of data hazard:

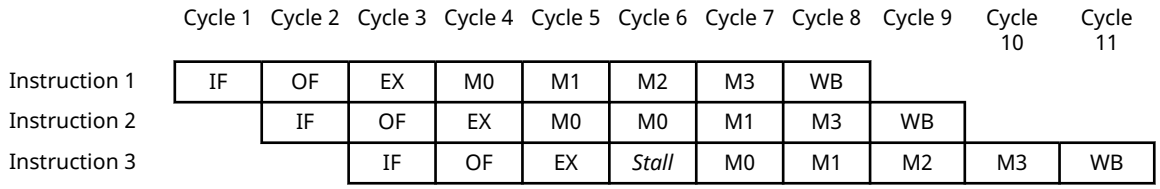
- An instruction in OF needs the result from an instruction in EX as a source operand. In this case, the EX instruction categories are load, store, multiply, divide, remainder, and floating-point instructions. This results in a 1–2 cycle stall.
- An instruction in OF uses the result from an instruction in MEM as a source operand. In this case, the MEM instruction categories are load, multiply, and floating-point instructions. This results in a 1 cycle stall.

Pipeline stalls are caused by data hazards, control hazards, structural hazards due to multi-cycle instructions, memory accesses using slower memory, instruction fetch from slower memory, or stream accesses.

The multi-cycle instruction categories are divide, remainder and floating-point instructions.

8-Stage Pipeline

With `C_OPTIMIZATION` set to 2 (frequency), the pipeline is divided into eight stages to maximize possible frequency: fetch (IF), decode (OF), execute (EX), access memory 0 (M0), access memory 1 (M1), access memory 2 (M2), access memory 3 (M3), and writeback (WB).



The 8-stage pipeline has the following kinds of data hazard:

- An instruction in OF needs the result from an instruction in EX as a source operand. In this case, the EX instruction categories are load, store, multiply, divide, remainder, and floating-point instructions. This results in a 1–5 cycle stall.
- An instruction in OF uses the result from an instruction in M0 as a source operand. In this case, the M0 instruction categories are load, multiply, divide, remainder, and floating-point instructions. This results in a 1–4 cycle stall.
- An instruction in OF uses the result from an instruction in M1 or M2 as a source operand. In this case, the M1 or M2 instruction categories are load, divide, remainder, and floating-point instructions. This results in a 1–3 or 1–2 cycle stall respectively.
- An instruction in OF uses the result from an instruction in M3 as a source operand. In this case, M3 instruction categories are load and floating-point instructions. This results in a 1 cycle stall.

In addition to multi-cycle instructions, there is one other kind of structural hazard: an instruction in M0 is a load or store instruction, and the instruction in M1, M2 or M3 is a load or store instruction that can cause an exception. This results in a 1 cycle stall.

Pipeline stalls are caused by data hazards, control hazards, structural hazards, memory accesses using slower memory, instruction fetch from slower memory, or stream accesses.

The multi-cycle instruction categories are divide and remainder instructions as well as floating-point divide, convert, and square-root instructions.

Branches

The instructions in the fetch and decode stages (as well as the prefetch buffer) are flushed when executing a taken branch. The fetch pipeline stage is then reloaded with a new instruction from the calculated branch address. A taken branch in MicroBlaze V takes a minimum of three clock cycles to execute, two of which are required for refilling the pipeline. To reduce this latency overhead, MicroBlaze V supports the optional branch target cache (BTC).

To improve branch performance, the branch target cache is coupled with a branch prediction scheme. With the BTC enabled, a correctly predicted branch or jump instruction incurs no overhead.

The BTC operates by saving the target address of each immediate branch and return instruction the first time the instruction is encountered. The next time it is encountered, it is usually found in the branch target cache, and the instruction fetch program counter is then changed to the saved target address, in case the branch should be taken. Jump instructions are always taken, whereas branches use branch prediction, to avoid taking a branch that should not have been taken and vice versa.

The BTC is cleared when a `fence.i` instruction is executed.

Branch prediction can cause a mispredict in the following cases:

- A branch that should *not* have been taken is taken.
- A branch that *should* have been taken is not taken.
- The target address of a return instruction is incorrect in the BTC, which might occur when returning from a function called from different places in the code.

All of these cases are detected and corrected when the branch or return instruction reaches the execute stage, and the branch prediction bits or target address are updated in the BTC, to reflect the actual instruction behavior. This correction incurs a penalty of two clock cycles for the 3-stage, 4-stage, and 5-stage pipeline and 7–9 clock cycles for the 8-stage pipeline due to the pipelined instruction fetch.

The size of the BTC can be selected with `C_BRANCH_TARGET_CACHE_SIZE`. The default recommended setting uses two block RAMs and provides 1024 entries. When selecting 64 entries or below, distributed RAM is used to implement the BTC, otherwise block RAM is used.

When the BTC uses block RAM, and `C_FAULT_TOLERANT` is set to 1, block RAMs are protected by parity. In case of a parity error, the branch is not predicted. To prevent accumulating errors in this case, the BTC should be cleared periodically by a `fence.i` instruction.

Pipeline Hazard Example

The effect of a data hazard is illustrated in the following table, using the 5-stage pipeline with the 32-bit implementation (RV32). The example shows a data hazard for a multiplication instruction, where the subsequent add instruction needs the result in register s0 to proceed. This means that the add instruction is stalled in OF during cycle 3 and 4 until the multiplication is complete.

Table 30: **Multiplication Data Hazard Example**

Cycle	IF	OF	EX	MEM	WB
1	Mul *s0*, s1, s2				

Table 30: **Multiplication Data Hazard Example** *(cont'd)*

Cycle	IF	OF	EX	MEM	WB
2	Add s3, *s0*, s1	Mul *s0*, s1, s2			
3		Add s3, *s0*, s1	Mul *s0*, s1, s2		
4		Add s3, *s0*, s1		Mul s0, s1, s2	
5			Add s3, *s0*, s1		Mul s0, s1, s2

Avoiding Data Hazards

In some cases, the compiler cannot optimize code to completely avoid data hazards. However, it is often possible to change the source code to achieve this, mainly by better utilization of the general purpose registers.

The following C code example shows the multiplication of a static array in memory:

```
static int a[4], b[4], c[4];
register int a0, a1, a2, a3, b0, b1, b2, b3, c0, c1, c2, c3;

a0 = a[0]; a1 = a[1]; a2 = a[2]; a3 = a[3];
b0 = b[0]; b1 = b[1]; b2 = b[2]; b3 = b[3];
c0 = a0 * b0; c1 = a1 * b1; c2 = a2 * b2; c3 = a3 * b3;
c[3] = c3; c2 = c[2]; c1 = c[1]; c0 = c[0];
```

This code ensures that load instructions are first executed to load operands into separate registers, which are then multiplied and finally stored. The code can be extended up to eight multiplications without running out of general purpose registers.

Memory Architecture

MicroBlaze V is implemented with a Harvard memory architecture; instruction and data accesses are done in separate address spaces.

With the 32-bit implementation (RV32), the address space has a 32-bit virtual address range (that is, handles up to 4 GB), and a 34-bit physical address range (16 GB) when using the Sv32 page-based virtual memory system.

With the 64-bit implementation (RV64), the address space has a default 32-bit physical address range, which can be extended up to a 64-bit range (that is, handles from 4 GB to 16 EB). When physical memory protection (PMP) is enabled, the maximum address range is limited to 56 bits (64 PB). When using the Sv39 page-based virtual memory system, the virtual address range is 39 bits (512 GB) and the physical address range is 56 bits (64 PB).

The instruction and data memory ranges can be made to overlap by mapping them both to the same physical memory. This is necessary for software debugging.

Both instruction and data interfaces of MicroBlaze V are 32 bits wide and use little endian format. MicroBlaze V supports word, half word, and byte accesses to data memory.

Data accesses must be aligned (word accesses must be on word boundaries, half word accesses on half word boundaries), unless software supports misaligned exception handling. All instruction accesses must be word aligned, unless compressed instructions are enabled (`C_USE_COMPRESSION` > 0), in which case accesses are half-word aligned.

MicroBlaze V prefetches instructions to improve performance, using the instruction prefetch buffer and (if enabled) instruction cache streams. To avoid attempts to prefetch instructions beyond the end of physical memory, which might cause an instruction access error or a processor stall, instructions must not be located too close to the end of physical memory. The instruction prefetch buffer requires 16 bytes margin, and using instruction cache streams adds two additional cache lines (32, 64, or 128 bytes).

MicroBlaze V does not separate data accesses to I/O and memory (it uses memory-mapped I/O). The processor has up to three interfaces for memory accesses:

- Local memory bus (LMB)
- Advanced eXtensible Interface (AXI4) for peripheral access
- Advanced eXtensible Interface (AXI4) or AXI Coherency Extension (ACE) for cache access

The LMB memory address range must not overlap with AXI4 ranges.

MicroBlaze V has a single cycle latency for accesses to local memory (LMB) and for cache read hits, except with `C_OPTIMIZATION` set to 1 (area), when data side accesses and data cache read hits require two clock cycles, and with error correction codes (ECCs) when byte writes and half word writes to LMB normally require one additional clock cycle.

The data cache write latency depends on `C_DCACHE_USE_WRITEBACK`. When `C_DCACHE_USE_WRITEBACK` is set to 1, the write latency normally is one cycle (more if the cache needs to do memory accesses). When `C_DCACHE_USE_WRITEBACK` is cleared to 0, the write latency normally is two cycles (more if the posted-write buffer in the memory controller is full).

The MicroBlaze V instruction and data caches can be configured to use 4, 8, or 16 word cache lines. When using a longer cache line, more bytes are prefetched, which generally improves performance for software with sequential access patterns. However, for software with a more random access pattern the performance can instead decrease for a given cache size. This is caused by a reduced cache hit rate due to fewer available cache lines.

For details on the different memory interfaces, see Chapter 3: MicroBlaze V Signal Interface Description.

Virtual Memory Management

Programs running on MicroBlaze V use effective addresses to access a flat 4 GB address space with the 32-bit implementation (RV32), and up to a 16 EB address space with the 64-bit implementation (RV64) depending on parameter C_ADDR_SIZE.

The processor can interpret this address space in different ways, depending on the virtual memory system as defined in the RISC-V Instruction Set Manual, Volume II:

- In bare mode, effective addresses are used to directly access physical memory.
- In virtual mode, the effective addresses are translated into physical addresses by the virtual-memory management hardware in the processor.

Virtual mode provides system software with the ability to relocate programs and data anywhere in the physical address space. System software can move inactive programs and data out of physical memory when space is required by active programs and data.

Relocation can make it appear to a program that more memory exists than is actually implemented by the system. This frees the programmer from working within the limits imposed by the amount of physical memory present in a system. Programmers do not need to know which physical-memory addresses are assigned to other software processes and hardware devices. The addresses visible to programs are translated into the appropriate physical addresses by the processor.

Virtual mode provides greater control over memory protection. Memory pages can be individually protected from unauthorized access. Protection and relocation enable system software to support multitasking. This capability gives the appearance of simultaneous or near-simultaneous execution of multiple programs.

In MicroBlaze V, virtual mode is implemented by the memory-management unit (MMU), available when C_USE_MMU is set to 3 (Supervisor). The MMU controls physical-address mapping and supports memory protection. Using these capabilities, system software can implement demand-paged virtual memory and other memory management schemes.

MicroBlaze V supports the RISC-V Supervisor Level ISA, with the Sv32 page-based virtual-memory system for the 32-bit implementation, and the Sv39, Sv48 or Sv57 page-based virtual-memory system for the 64-bit implementation. In both cases, the Supervisor Address Translation and Protection (satp) register implements a 7-bit address space identifier (ASID).

Physical Memory Protection is not available when C_USE_MMU is set to 3 (Supervisor).

Translation Look-aside Buffer

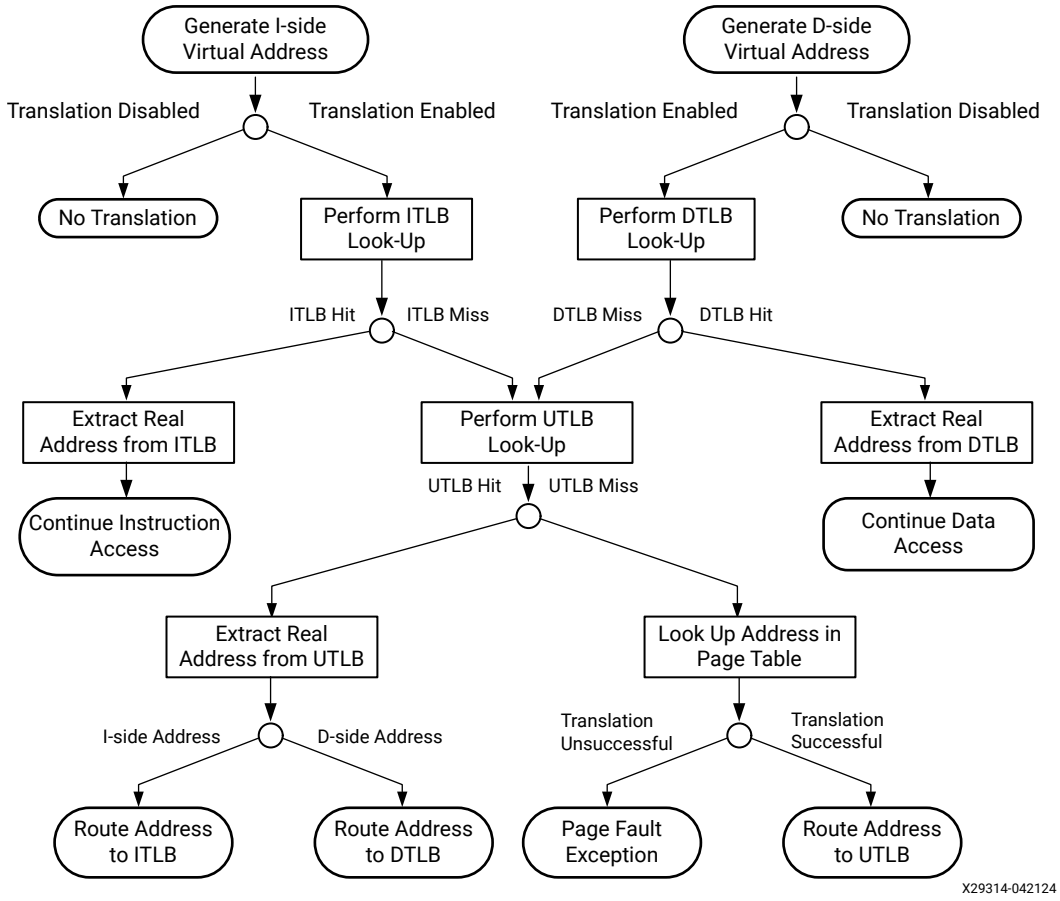
The translation look-aside buffer (TLB) is used by the MicroBlaze V MMU for address translation when the processor is running in virtual mode, memory protection, and storage control. Each entry within the TLB contains the information necessary to identify a virtual page (ASID and virtual page number), specify its translation into a physical page, determine the protection characteristics of the page, and specify the storage attributes associated with the page.

The MicroBlaze V TLB is physically implemented as three separate TLBs:

- **Unified TLB:** The UTLB contains 64 entries and is pseudo-associative. Instruction-page and data-page translation can be stored in any UTLB entry. The initialization and management of the UTLB is controlled by hardware, which reads information from the page tables stored in memory.
- **Instruction Shadow TLB:** The ITLB contains instruction page-translation entries and is fully associative. The page-translation entries stored in the ITLB represent the most recently accessed instruction-page translations from the UTLB. The ITLB is used to minimize contention between instruction translation and UTLB-update operations. The initialization and management of the ITLB are controlled completely by hardware and are transparent to software.
- **Data Shadow TLB:** The DTLB contains data page-translation entries and is fully associative. The page-translation entries stored in the DTLB represent the most-recently accessed data-page translations from the UTLB. The DTLB is used to minimize contention between data translation and UTLB-update operations. The initialization and management of the DTLB is controlled completely by hardware and is transparent to software.

The following figure provides the translation flow for TLB.

Figure 31: **TLB Address Translation Flow**



Two schemes to manage accessed (A) and dirty (D) bits are permitted by the virtual memory system. MicroBlaze V implements the first scheme, raising a page-fault exception when a virtual page is accessed and the A bit is clear, or is written and the D bit is clear.

MicroBlaze V implements the Supervisor Memory-Management Fence Instruction (SFENCE.VMA) to ensure ordering and invalidate entries in the TLB.

Reset, Interrupts, and Exceptions

MicroBlaze V supports reset, interrupt, and hardware exceptions according to the RISC-V Instruction Set Manual. Non-maskable interrupt is supported using the external input signal `Ext_NM_BRK`. A platform interrupt is defined using the external input signal `Ext_BRK`. The following section describes the execution flow associated with each of these events.

Normally, the reset vector is located at address 0, but the parameter `C_BASE_VECTORS` can be used to locate it anywhere in memory.

The memory address location of the trap vector is determined by the mtvec or stvec registers, which are initialized to C_BASE_VECTORS + 4 with the two least significant bits set to 00, but can be changed to any address by software.

Table 31: **Vector Addresses**

Type	Vector Address	Description
Reset	C_BASE_VECTORS + 0x0	Execution starts from here after reset is released.
Trap	C_BASE_VECTORS + 0x4	Default address for exceptions and interrupts

The relative priority, starting with the highest, is:

1. Reset
2. Non-maskable interrupt
3. Platform interrupt
4. External interrupt
5. Breakpoint (instruction) exception
6. Instruction page fault exception
7. Instruction access fault exception
8. Illegal instruction, instruction address misaligned, environment call, breakpoint (data) exceptions
9. Load address misaligned, store/AMO address misaligned exceptions
10. Load page fault, store/AMO page fault exception
11. Load access fault, store/AMO access fault, AXI4-Stream exceptions

The priority between exceptions is compliant with the RISC-V Instruction Set Manual, Volume II.

Reset

When a Reset or Debug_Rst occurs, MicroBlaze V flushes the pipeline and immediately starts fetching instructions from the reset vector (address C_BASE_VECTORS). Both external reset signals are active-High, and it is recommended to assert the signals for at least 16 cycles.

Reset does not clear the general purpose registers (x1 – x31), the floating-point registers (f0 – f31), or the PMP address registers (pmpaddr0 – pmpaddr63) . To ensure that stale data is not used, software should not assume that the general purpose registers or floating-point registers are zero.

MicroBlaze V does not wait for outstanding AXI or LMB transactions to complete before it begins fetching instructions from the reset vector. When only resetting the processor, all external accesses must be completed before asserting `Reset`. This can be achieved with a `wfi` instruction to enter sleep mode or the `Pause` signal. See Sleep and Pause Functionality for details.

Equivalent Pseudocode

```
pc ← C_BASE_VECTORS
fcsr ← 0
mstatus ← 0, 00020000, 00018000, 00038000
mie ← 0; mtvec[31:2] ← (C_BASE_VECTORS + 4) / 4; mtvec[1:0] ← 00
mcounteren ← 0; mscratch ← 0; mepc ← 0; mcause ← 0; mtval ← 0
mstream ← 0; mwfi ← 0
mcycle  ← 0; minstret  ← 0;
mcycleh ← 0; minstreth ← 0; mhpmcounterh3 - mhpmcounterh31 ← 0
mcountinhibit ← 0;
cycle   ← 0; instret   ← 0;
cycleh  ← 0; instreth  ← 0; hpmcounterh3 - hpmcounterh31  ← 0
Reservation ← 0
```

The actual mstatus reset value depends on parameters. See the Machine Status Register (mstatus) description for details.

The actual mhpmevent, mhpmcounter, and hpmcounter reset values depend on the configured debug event counters and debug latency counters. For details, see the description in Table 3: Control and Status Registers.

Interrupts and Exceptions

MicroBlaze V can be configured to trap the following internal error conditions: illegal instruction, instruction and data access error, and misaligned access.

A hardware exception causes MicroBlaze V to flush the pipeline and branch to the hardware trap vector (mtvec). The execution stage instruction in the exception cycle is not executed. The registers mcause, mepc, and mtval are updated according to the RISC-V Instruction Set Manual, Volume II.

External Interrupt

MicroBlaze V supports one external interrupt source (connected to the `Interrupt` input port). The processor only reacts to interrupts if both the global interrupt enable bit MIE in mstatus and the external interrupt enable bit MEIE in mie are set to 1. On an interrupt, the instruction in the execution stage completes while the instruction in the decode stage is replaced by a branch to the trap-vector base-address in mtvec, or with low-latency interrupt mode, the address supplied by the Interrupt Controller.

The interrupt return address (the PC associated with the instruction in the decode stage at the time of the interrupt) is automatically loaded into mepc. In addition, the processor also disables future interrupts by clearing the global interrupt enable bit MIE in mstatus to 0. The MIE bit is automatically set again when executing the MRET instruction.

By using the parameter `C_INTERRUPT_IS_EDGE`, the external interrupt can either be set to level-sensitive or edge-triggered:

- When using level-sensitive interrupts, the `Interrupt` input must remain set until MicroBlaze V has taken the interrupt, and jumped to the trap vector. Software must acknowledge the interrupt at the source to clear it before returning from the interrupt handler. If not, the interrupt is taken again, as soon as interrupts are enabled when returning from the trap handler.
- When using edge-triggered interrupts, MicroBlaze V detects and latches the `Interrupt` input edge, which means that the input only needs to be asserted one clock cycle. The interrupt input can remain asserted, but must be deasserted at least one clock cycle before a new interrupt can be detected. The latching of an edge-triggered interrupt is independent of the global interrupt enable bit MIE in mstatus. If an interrupt occurs while the MIE bit is 0, it is immediately serviced when the MIE bit is set to 1 (and the external interrupt enable bit MEIE in mie is set to 1). The MEIP bit in MIP is set when the interrupt is latched and cleared when returning from the trap handler with the MRET instruction.

With periodic interrupt sources, such as the FIT Timer IP core, that do not have a method to clear the interrupt from software, it is recommended to use edge-triggered interrupts.

Low-Latency Vectored Interrupt Mode

A low-latency vectored interrupt mode is available, which allows the Interrupt Controller to directly supply the interrupt vector for each individual interrupt (using the input port `Interrupt_Address`). The address of all interrupt handlers, including interrupts that are using the trap-vector base-address in mtvec, must be passed to the Interrupt Controller when initializing the interrupt system. When a particular interrupt occurs, this address is supplied by the Interrupt Controller, which allows MicroBlaze V to directly jump to the handler code. The interrupt address must remain valid until MicroBlaze V has taken the interrupt and jumped to the address.

With this mode, MicroBlaze V also directly sends the appropriate interrupt acknowledge to the Interrupt Controller (using the `Interrupt_Ack` output port), although it is still the responsibility of the interrupt service routine to acknowledge level sensitive interrupts at the source.

This information allows the Interrupt Controller to acknowledge interrupts appropriately, both for level-sensitive and edge-triggered interrupts.

To inform the Interrupt Controller of the interrupt handling events, `Interrupt_Ack` is set to:

- **01:** When MicroBlaze V jumps to the interrupt handler code.
- **10:** When the MRET instruction is executed.

- **11:** When the global interrupt enable bit MIE in mstatus is changed from 0 to 1, which enables interrupts again.

The `Interrupt_Ack` output port is active during one clock cycle, and is then reset to 00.

With low-latency interrupt mode, control is directly passed to the interrupt handler for each individual interrupt utilizing this mode. In this case, it is the responsibility of each handler to save and restore used registers. This can be achieved with the RISC-V GCC compiler using the following code:

```
/* Declare a low-latency interrupt handler */
void interrupt_handler_name() __attribute__((aligned(4), interrupt));
```

Equivalent Pseudocode

```
mepc ← PC
if C_USE_INTERRUPT = 2
   PC ← Interrupt_Address
   Interrupt_Ack ← 01
else
   PC ← mtvec
mcause ← 0x8000000B
mstatus.mpie ← mstatus.mie
mstatus.mie ← 0
if C_USE_MMU > 0
   mstatus.mpp ← privilege mode
else
   mstatus.mpp ← 11
mtval ← 0
Reservation ← 0
```

Non-Maskable Interrupt

A non-maskable interrupt is performed by asserting the external non-maskable break signal (that is, the `Ext_NM_BRK` input port). On a break, the instruction in the execution stage completes while the instruction in the decode stage is replaced by a branch to the trap-vector base-address in mtvec.

The return address (the PC associated with the instruction in the decode stage at the time of the break) is automatically loaded into mepc. The mcause interrupt bit is set to 1 and the exception code is set to 0.

A non-maskable interrupt is always handled immediately.

The `Ext_NM_BRK` signal must only be asserted one clock cycle.

Equivalent Pseudocode

```
mepc ← PC
PC ← mtvec
mcause ← 0x80000000
mstatus.mpie ← mstatus.mie
mstatus.mie ← 0
if C_USE_MMU > 0
   mstatus.mpp ← privilege mode
else
   mstatus.mpp ← 11
mtval ← 0
Reservation ← 0
```

Platform Interrupt

A machine external break platform interrupt is performed by asserting the external break signal (that is, the `Ext_BRK` input port). On a break, the instruction in the execution stage completes while the instruction in the decode stage is replaced by a branch to the trap-vector base-address in mtvec.

The return address (the PC associated with the instruction in the decode stage at the time of the break) is automatically loaded into mepc. The mcause interrupt bit is set to 1 and the exception code is set to 16.

A platform interrupt is only handled when the global interrupt enable bit MIE in mstatus is set to 1 (that is, there is no other interrupt in progress).

The `Ext_BRK` signal must be kept asserted until the trap has occurred, and deasserted before the MRET instruction is executed.

Equivalent Pseudocode

```
mepc ← PC
PC ← mtvec
mcause ← 0x80000010
mstatus.mpie ← mstatus.mie
mstatus.mie ← 0
if C_USE_MMU > 0
   mstatus.mpp ← privilege mode
else
   mstatus.mpp ← 11
mtval ← 0
Reservation ← 0
```

Exceptions

Exception Causes

- **Instruction address misaligned:** When compressed instructions are not enabled, the misaligned exception is caused by an instruction access where the address to the instruction bus has the second least significant bit set.

When compressed instructions are enabled, the exception cannot occur.

- **Instruction access fault:** The instruction access fault exception is caused by errors when reading data from memory.
  - The instruction peripheral AXI4 interface (M_AXI_IP) exception is caused by an error response on `M_AXI_IP_RRESP`.
  - The instruction side local memory (ILMB) can only cause an instruction access fault when an uncorrectable error occurs in the LMB memory (as indicated by the `IUE` signal), or when `C_ECC_USE_CE_EXCEPTION` is set to 1 and a correctable error occurs in the LMB memory, as indicated by the `ICE` signal.
- **Illegal instruction:** When complete illegal instruction decoding is enabled (`C_ILL_INSTR_EXCEPTION` = 2) the illegal instruction exception occurs for all unimplemented or undefined instructions, including access to all unimplemented or undefined control and status registers. Decoding takes into account which extensions and modes are enabled.

  When basic illegal instruction decoding is enabled (`C_ILL_INSTR_EXCEPTION` = 1) the illegal instruction exception only occurs for unimplemented opcodes. Decoding takes into account which extensions are enabled.

  Otherwise, the illegal instruction exception cannot occur.
- **Load address misaligned:** The misaligned exception is caused by a word access where the address to the data bus has any of the two least significant bits set, or a half word access with the least significant bit set.For the 64-bit implementation (RV64), a misaligned exception is also caused by a double access where the address to the data bus has any of the three least significant bits set.
- **Load access fault:** The load access fault exception is caused by errors when reading data from memory. This can also occur for a virtual memory translation page table access.
  - The data peripheral AXI4 interface (M_AXI_DP) exception is caused by an error response on `M_AXI_DP_RRESP` or an OKAY response in case of an exclusive access using a load-reserved (LR) or atomic (AMO) instruction load.
  - The data cache AXI4 interface (M_AXI_DC) exception is caused by an OKAY response on `M_AXI_DC_RRESP` in case of an exclusive access using a load-reserved (LR) or atomic (AMO) instruction load. The exception can only occur when an exclusive access using LR is performed. In all other cases the response is ignored.
  - The data side local memory (DLMB) can only cause a load access fault exception when either an uncorrectable error occurs in the LMB memory, as indicated by the `DUE` signal, or `C_ECC_USE_CE_EXCEPTION` is set to 1 and a correctable error occurs in the LMB memory, as indicated by the `DCE` signal.

  The load access fault exception is also caused by an unsuccessful physical memory protection (PMP) check.

- **Store/AMO address misaligned:** The misaligned exception is caused by a word access where the address to the data bus has any of the two least significant bits set, or a half word access with the least significant bit set.For the 64-bit implementation (RV64), a misaligned exception is also caused by a double access where the address to the data bus has any of the three least significant bits set.
- **Store/AMO access fault:** The store/AMO access fault exception is caused by errors when writing data to memory, or when an AMO instruction accesses memory.
  - The data peripheral AXI4 interface (M_AXI_DP) exception is caused by an error response on `M_AXI_DP_BRESP`.
  - The data cache AXI4 interface (M_AXI_DC) exception is caused by an error response on `M_AXI_DC_BRESP`. The exception can only occur when an exclusive access using SC is performed. In all other cases, the response is ignored.
  - The data side local memory (DLMB) can only cause a load access fault exception when either an uncorrectable error occurs in the LMB memory, as indicated by the `DUE` signal, or `C_ECC_USE_CE_EXCEPTION` is set to 1 and a correctable error occurs in the LMB memory, as indicated by the `DCE` signal. An error can only occur for byte and half word write accesses.

  The store/AMO access fault exception is also caused by an unsuccessful physical memory protection (PMP) check.
- **Instruction page fault:** The instruction page fault is caused by an unsuccessful virtual memory instruction address translation.
- **Load page fault:** The load page fault is caused by an unsuccessful virtual memory data address translation for a load instruction.
- **Store/AMO page fault:** The store/AMO page fault is caused by an unsuccessful virtual memory data address translation for a store or AMO instruction.
- **AXI4-Stream exception:** The AXI4-Stream exception is caused by a GET or GETD custom instruction `TLAST` control bit mismatch when the instruction 'e' bit is set. A mismatch either occurs when the custom instruction expects `TLAST` to be set (the instruction 'c' bit is set) and it is not, or vice versa.

Imprecise Exceptions

Normally all exceptions in MicroBlaze V are precise, meaning that any instructions in the pipeline after the instruction causing an exception are invalidated, and have no effect.

When `C_IMPRECISE_EXCEPTIONS` is set to 1 an Instruction Access Fault, Load Access Fault, or Store/AMO Access Fault exception is not precise, meaning that a subsequent memory access instruction in the pipeline might be executed. If this behavior is acceptable, the maximum frequency or performance can be improved by setting this parameter to 1.

When using the 3-stage, 4-stage or 5-stage pipelines (`C_OPTIMIZATION ≠ 2`) imprecise exceptions can only be caused by ECC errors in LMB memory. The advantage of enabling imprecise exceptions is improved maximum frequency.

When using the 8-stage pipeline (`C_OPTIMIZATION = 2`) imprecise exceptions can be caused by any erroneous memory access. The advantage of enabling imprecise exceptions is improved performance.

Equivalent Pseudocode

```
if C_USE_MMU > 1 and medeleg(exception cause) = 1
   sepc ← PC
   PC ← stvec
   scause ← exception cause
   if privilege mode = 00
      sstatus.sie ← 0
   else
      sstatus.spp ← 1
   stval ← exception specific value
else
   mepc ← PC
   PC ← mtvec
   mcause ← exception cause
   mstatus.mpie ← mstatus.mie
   mstatus.mie ← 0
   if C_USE_MMU > 0
      mstatus.mpp ← privilege mode
   else
      mstatus.mpp ← 11
   mtval ← exception specific value
Reservation ← 0
```

The actual scause, stval, mcause, and mtval values depend on the exception. See the description of the registers for details.

Instruction Cache

The MicroBlaze V processor can be used with an optional instruction cache for improved performance when executing code that resides outside the LMB address range.

The instruction cache has the following features:

- Direct mapped (one-way associative)
- Cleared by hardware after reset
- Cleared by the FENCE.I instruction
- User selectable cacheable memory address range
- Configurable cache and tag size

- Caching over AXI4 interface (M_AXI_IC)
- Option to use 4, 8, or 16 word cache-line
- Optional stream buffers to improve performance by speculatively prefetching instructions
- Optional victim cache to improve performance by saving evicted cache lines
- Optional parity protection that invalidates cache lines if a block RAM bit error is detected
- Optional data width selection to either use 32 bits, an entire cache line, or 512 bits

General Instruction Cache Functionality

When the instruction cache is used, the memory address space is split into two segments: a cacheable segment and a non-cacheable segment. The cacheable segment is determined by two parameters: `C_ICACHE_BASEADDR` and `C_ICACHE_HIGHADDR`. All addresses within this range correspond to the cacheable address segment. All other addresses are non-cacheable.

The cacheable segment size must be 2^N, where N is a positive integer. The range specified by `C_ICACHE_BASEADDR` and `C_ICACHE_HIGHADDR` must comprise a complete power-of-two range, such that range = 2^N and the N least significant bits of `C_ICACHE_BASEADDR` must be zero.

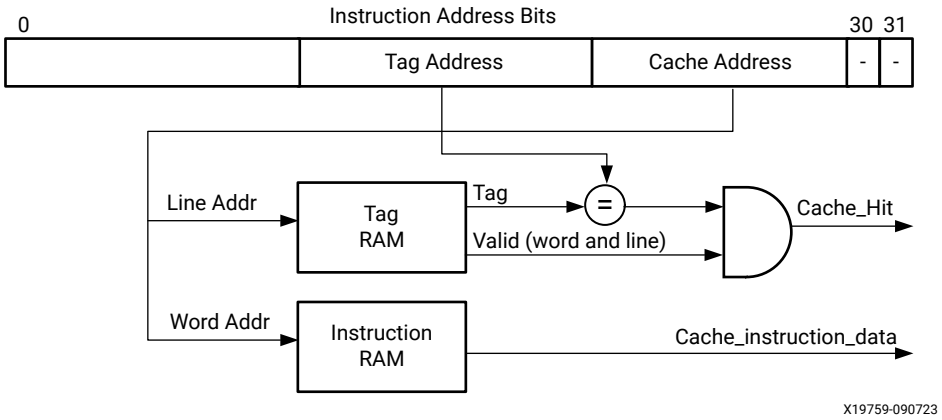
The cacheable instruction address consists of two parts: the cache address, and the tag address. The instruction cache can be configured from 64 bytes to 64 kB. This corresponds to a cache address of between 6 and 16 bits. The tag address together with the cache address should match the full address of cacheable memory.

When selecting cache sizes below 2 kB, distributed RAM is used to implement the tag RAM and instruction RAM. Distributed RAM is always used to implement the tag RAM, when setting the parameter `C_ICACHE_FORCE_TAG_LUTRAM` to 1. This parameter is only available with cache size 8 kB and less for 4 word cache lines, with 16 kB and less for 8 word cache lines, and with 32 kB and less for 16 word cache-lines.

For example: in a 32-bit MicroBlaze V configured with `C_ICACHE_BASEADDR`= 0x0030_0000, `C_ICACHE_HIGHADDR`=0x0030_ffff, `C_ICACHE_BYTE_SIZE`= 4096, `C_ICACHE_LINE_LEN`= 8, and `C_ICACHE_FORCE_TAG_LUTRAM`= 0; the cacheable memory of 64 kB uses 16 bits of byte address, and the 4 kB cache uses 12 bits of byte address. The required address tag width is therefore 16-12 = 4 bits. The total number of block RAM primitives required in this configuration is: 2 RAMB16 for storing the 1024 instruction words, and 1 RAMB16 for 128 cache line entries, each consisting of: 4 bits of tag, 8 word-valid bits, 1 line-valid bit. In total 3 RAMB16 primitives.

The following figure shows the organization of the instruction cache.

Figure 32: **Instruction Cache Organization**



Instruction Cache Operation

After reset is released, the instruction cache is initialized by hardware invalidating all cache lines individually. The initialization requires `C_ICACHE_BYTE_SIZE` / (`C_ICACHE_LINE_LEN` * 4) clock cycles. While it is in progress, reads within the cacheable range directly access memory, bypassing the cache.

For every instruction fetched, the instruction cache detects if the instruction address belongs to the cacheable segment. If the address is non-cacheable, the cache controller ignores the instruction and lets the M_AXI_IP or ILMB complete the request. If the address is cacheable, a lookup is performed on the tag memory to check if the requested address is currently cached. The lookup is successful if: the word and line valid bits are set, and the tag address matches the instruction address tag segment. On a cache miss, the cache controller requests the new instruction over the instruction AXI4 interface (M_AXI_IC), and waits for the memory controller to return the associated cache line.

Executing a FENCE.I instruction invalidates all locations in the instruction cache.

`C_ICACHE_DATA_WIDTH` determines the bus data width, either 32 bits, an entire cache line (128, 256, or 512 bits), or 512 bits.

When `C_FAULT_TOLERANT` is set to 1, a cache miss also occurs if a parity error is detected in a tag or instruction block RAM.

The instruction cache issues burst accesses for the AXI4 interface when 32-bit data width is used, otherwise single accesses are used.

Stream Buffers

When stream buffers are enabled, by setting the parameter `C_ICACHE_STREAMS` to 1, the cache speculatively fetches cache lines in advance in sequence following the last requested address, until the stream buffer is full.

The stream buffer can hold up to two cache lines. Should the processor subsequently request instructions from a cache line prefetched by the stream buffer, which occurs in linear code, they are immediately available.

The stream buffer often improves performance, because the processor generally spends less time waiting for instructions to be fetched from memory.

`C_ICACHE_DATA_WIDTH` determines the amount of data transferred from the stream buffer each clock cycle: either 32 bits or an entire cache line.

Victim Cache

The victim cache is enabled by setting the parameter `C_ICACHE_VICTIMS` to 2, 4, or 8. This value defines the number of cache lines that can be stored in the victim cache. Whenever a cache line is evicted from the cache, it is saved in the victim cache. Saving the most recent lines means that they can be fetched more quickly when the processor requests them, thereby improving performance. If the victim cache is not used, all evicted cache lines must be read from memory again when they are needed.

`C_ICACHE_DATA_WIDTH` determines the amount of data transferred from/to the victim cache each clock cycle, either 32 bits or an entire cache line.

Data Cache

The MicroBlaze V processor can be used with an optional data cache for improved performance. The cached memory range must not include addresses in the LMB address range. The data cache has the following features:

- Direct mapped (one-way associative)
- Cleared by hardware after reset
- Cleared by the FENCE.I instruction
- Write-through or write-back
- User selectable cacheable memory address range
- Configurable cache size and tag size
- Caching over AXI4 interface (M_AXI_DC)

- Option to use 4, 8, or 16 word cache lines
- Optional victim cache with write-back to improve performance by saving clean copies of evicted cache lines
- Optional parity protection for write-through cache that invalidates cache lines if a block RAM bit error is detected
- Optional data width selection to either use 32 bits, an entire cache line, or 512 bits

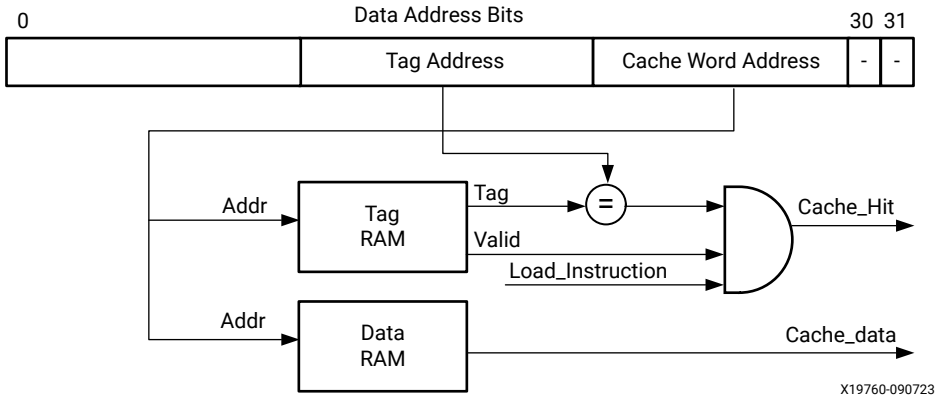
General Data Cache Functionality

When the data cache is used, the memory address space is split into two segments: a cacheable segment and a non-cacheable segment. The cacheable area is determined by two parameters: `C_DCACHE_BASEADDR` and `C_DCACHE_HIGHADDR`. All addresses within this range correspond to the cacheable address space. All other addresses are non-cacheable.

The cacheable segment size must be 2^N, where N is a positive integer. The range specified by `C_DCACHE_BASEADDR` and `C_DCACHE_HIGHADDR` must comprise a complete power-of-two range, such that range = 2^N and the N least significant bits of `C_DCACHE_BASEADDR` must be zero.

The following figure shows the data cache organization.

Figure 33: **Data Cache Organization**



The cacheable data address consists of two parts: the cache address, and the tag address. The data cache can be configured from 64 bytes to 64 kB. This corresponds to a cache address of between 6 and 16 bits. The tag address together with the cache address should match the full address of cacheable memory. When selecting cache sizes below 2 kB, distributed RAM is used to implement the tag RAM and data RAM. Distributed RAM is always used to implement the tag RAM, when setting the parameter `C_DCACHE_FORCE_TAG_LUTRAM` to 1. This parameter is only available with cache size 8 kB and less for 4 word cache-lines, with 16 kB and less for 8 word cache-lines, and with 32 kB and less for 16 word cache-lines.

For example, in a 32-bit MicroBlaze V configured with C_DCACHE_BASEADDR= 0x0040_0000, C_DCACHE_HIGHADDR= 0x0040_3fff, C_DCACHE_BYTE_SIZE= 2048, C_DCACHE_LINE_LEN= 4, and C_DCACHE_FORCE_TAG_LUTRAM= 0; the cacheable memory of 16 kB uses 14 bits of byte address, and the 2 kB cache uses 11 bits of byte address, thus the required address tag width is 14-11=3 bits. The total number of block RAM primitives required in this configuration is 1 RAMB16 for storing the 512 data words, and 1 RAMB16 for 128 cache line entries, each consisting of 3 bits of tag, 4 word-valid bits, 1 line-valid bit. The total is 2 RAMB16 primitives.

Data Cache Operation

After reset is released, the data cache is initialized by hardware by invalidating all cache lines individually. The initialization requires C_DCACHE_BYTE_SIZE / (C_DCACHE_LINE_LEN * 4) clock cycles. While it is in progress, accesses within the cacheable range directly access memory, bypassing the cache.

The caching policy used by the MicroBlaze V data cache, write-back or write-through, is determined by the parameter C_DCACHE_USE_WRITEBACK. When this parameter is set, a write-back protocol is implemented; otherwise, write-through is implemented.

With the write-back protocol, a store to an address within the cacheable range always updates the cached data. If the target address word is not in the cache (that is, the access is a cache miss), and the location in the cache contains data that has not yet been written to memory (the cache location is dirty), the old data is written over the data AXI4 interface (M_AXI_DC) to external memory before updating the cache with the new data. If only a single word needs to be written, a single word write is used. Otherwise, a burst write is used. For byte or half word stores, in case of a cache miss, the address is first requested over the data AXI4 interface, while a word store only updates the cache.

With the write-through protocol, a store to an address within the cacheable range generates an equivalent byte, half word, or word write over the data AXI4 interface to external memory. The write also updates the cached data if the target address word is in the cache (that is, the write is a cache hit). A write cache-miss does not load the associated cache line into the cache.

With the write-back protocol, a FENCE.I instruction flushes all dirty cache locations and invalidates all other locations, whereas with the write-through protocol, all locations are invalidated.

A load from an address within the cacheable range triggers a check to determine if the requested data is currently cached. If it is (that is, on a cache hit) the requested data is retrieved from the cache. If it is not (that is, on a cache miss), the address is requested over the data AXI4 interface using a burst read, and the processor pipeline stalls until the cache line associated to the requested address is returned from the external memory controller.

If the address is non-cacheable, the cache controller ignores the instruction and lets the M_AXI_DP or DLMB complete the request.

The parameter C_DCACHE_DATA_WIDTH determines the bus data width, either 32 bits, an entire cache line (128, 256, or 512 bits), or 512 bits.

When C_FAULT_TOLERANT is set to 1 and write-through protocol is used, a cache miss also occurs if a parity error is detected in the tag or data block RAM.

The following table summarizes all types of accesses issued by the data cache AXI4 interface.

Table 32: **Data Cache Interface Accesses**

Policy	State	Direction	Access Type
Write-through	Cache Enable	Read	Burst for 32-bit interface non-exclusive access and exclusive access with ACE enabled, single access otherwise
		Write	Single access
Write-back	Cache Enable	Read	Burst for 32-bit interface, single access otherwise
		Write	Burst for 32-bit interface cache lines with more than one valid word, a single access otherwise

Victim Cache

The victim cache is enabled by setting the parameter C_DCACHE_VICTIMS to 2, 4, or 8. This defines the number of cache lines that can be stored in the victim cache. Whenever a complete cache line is evicted from the cache, it is saved in the victim cache. By saving the most recent lines they can be fetched much faster, should the processor request them, thereby improving performance. If the victim cache is not used, all evicted cache lines must be read from memory again when they are needed.

With the AXI4 interface, C_DCACHE_DATA_WIDTH determines the amount of data transferred from or to the victim cache each clock cycle, either 32 bits or an entire cache line.

Debug and Trace

Debug

MicroBlaze V features a debug interface to support software debugging tools (commonly known as background debug mode (BDM) debuggers) such as the AMD Vitis™ System Debugger tool.

The debug interface is designed to be connected to the MicroBlaze Debug Module (MDM) V core, which interfaces with the JTAG port of AMD Adaptive Computing FPGAs or with a memory-mapped AXI4-Lite interface.

The debug functionality is implemented according to the definition in RISC-V External Debug Support, Version 1.0, and is compatible with any third-party tools following this standard.

Multiple MicroBlaze V processors can be interfaced with a single MDM V core to enable multiprocessor debugging. Each processor represents a single RISC-V hardware thread (hart).

To download programs, set software breakpoints and disassemble code, ensure that the instruction and data memory ranges overlap, and use the same physical memory.

Debug registers are accessed using the debug interface, and are not directly visible to software running on the processor. The debug interface can either use JTAG serial access or AXI4-Lite parallel access, controlled by the parameter `C_DEBUG_INTERFACE`.

See the *MicroBlaze Debug Module V (MDM V) LogiCORE IP Product Guide* (PG428) for a detailed description of the MDM V features.

The debugging features enabled by setting `C_DEBUG_ENABLED` to 1 include:

- A configurable number of hardware triggers and unlimited software breakpoints
- External control that enables debug tools to stop, reset, and single step the processor
- Ability to read from and write to: memory, general purpose registers, floating-point registers, and control and status registers
- Support for multiple processors (RISC-V harts)
- Configurable number of hardware performance counters (event and latency counters)
- RISC-V N-Trace based program trace with configurable embedded trace buffer size
- RISC-V N-Trace based external trace with Trace Funnel and Trace PIB sink provided by the MDM V
- Non-intrusive profiling support with configurable profiling buffer size

***Note*:** Vivado provides a design rule check that prevents you from inadvertently using the classic MicroBlaze MDM instead of MDM V, and vice versa.

When the debugger attempts to stop the processor with a halt request and the processor is waiting for an ongoing LMB, AXI4, or AXI4-Stream access to complete, the access is forced to complete after 8192 clock cycles. This ensures that the debugger can always stop the processor. Read accesses update the destination register with the current interface data value in this case.

Trace and Profiling Register Access

Both program trace and non-intrusive profiling registers are memory-mapped, and can be accessed through either of two interfaces:

- The Debug Interface with access from MDM V, using the RISC-V System Bus.

  In this case, MDM V determines the System Bus base address for each connected processor, with the first processor starting at 0x8000, the second at 0x10000, and so on, giving each processor a 32 KB area. With external trace, the MDM V uses the first 32 KB area for the Trace Funnel and Trace PIB Sink registers.

- The Slave AXI Interface (S_AXI) with access from any AXI4 Master, but normally from the MDM V AXI4 master port, using the RISC-V System Bus.

  In this case, the base address is assigned in the AMD Vivado™ Block Design, typically to 0x44A00000 with automatic address assignment. It is recommended to set the base addresses for subsequent processors in a multiprocessor design to 0x44A08000, and so on, to give each processor a 32 KB area.

To use the Debug Interface, the Slave AXI Interface should not be enabled, and the Embedded Trace Interface should be selected in the MDM V.

It is recommended to use the Debug Interface, which requires no additional connections between the MDM V and the processor, unless the MDM V AXI4 master port is used for memory access.

Program Trace

With debugging enabled, MicroBlaze V provides program trace, storing information in the Embedded Trace Buffer, to enable program execution tracing. Program trace is implemented according to the RISC-V N-Trace (Nexus-based Trace) Specification. The Embedded Trace Buffer is provided by a Trace RAM Sink, as specified in the RISC-V Trace Control Interface Specification.

The size of the Embedded Trace Buffer can be configured from 4 KB to 128 KB using the parameter `C_DEBUG_TRACE_SIZE`. By setting `C_DEBUG_TRACE_SIZE` to 0 (None), program trace is disabled.

RISC-V N-Trace uses compression to reduce the amount of trace data, while still allowing reconstruction of the program execution flow. The Trace Encoder supports both Branch Trace Messaging (BTM) and History Trace Messaging (HTM). In general, HTM is preferred, as it usually gives better compression.

The memory/mapped registers used to configure and control trace, and to read the Trace Buffer, are listed in the following table.

Table 33: **MicroBlaze V Program Trace Registers**

Register Name	Offset	Reset Value	R/W	Description
Trace Encoder				
trTeControl	0x2000	0x1000000	R/W	Trace Encoder control register
trTeImpl	0x2004	0x15	R	Trace Encoder implementation information
trTeInstFeatures	0x2008	0x0	R/W	Extra instruction trace encoder features and trace source IDs
trTsControl	0x2040	0x30	R/W	Timestamp control register
trTsCounterLow	0x2048	0x0	R	Lower 32 bits of timestamp counter
trTsCounterHigh	0x204C	0x0	R	Upper bits of timestamp counter

Table 33: **MicroBlaze V Program Trace Registers** *(cont'd)*

Register Name	Offset	Reset Value	R/W	Description
Trace RAM Sink				
trRamControl	0x3000	0x0	R/W	RAM Sink control register
trRamImpl	0x3004	0x15	R	RAM Sink Implementation information
trRamStartLow	0x3014	0x0	R	Lower 32 bits of start address of circular trace buffer
trRamLimitLow	0x3018	C_DEBUG_TRACE_SIZE/4-1	R/W	Lower 32 bits of end address of circular trace buffer
rtRamWPLow	0x3020	0x0	R/W	Lower 32 bits of current write location for trace data in circular buffer
trRamRPLow	0x3028	0x0	R/W	Lower 32 bits of access pointer for trace readback
trRamData	0x3040	-	R	Read access to SRAM trace memory (32-bit data)

For registers with optional, implementation defined, features, descriptions are provided. For other registers and additional details, see the RISC-V Trace Control Interface Specification.

Trace Encoder Control Register (trTeControl)

The Trace Encode Control Register implemented fields are listed below. The optional fields trTeInstTrigEnable and TrTeInstStallEna are not implemented.

Figure 34: **Trace Encoder Control Register**

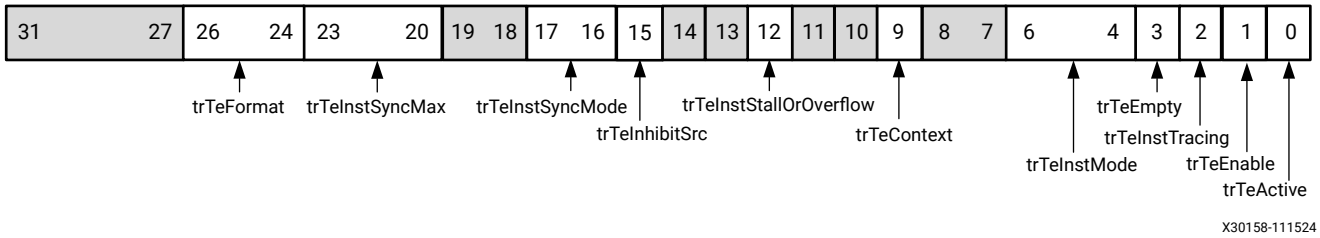


Table 34: **Trace Encoder Control Register**

Bits	Name	Description	Reset Value
26:24	trTeFormat	1: Format defined by RISC-V N-Trace (Nexus-based Trace) Specification. Read only.	001
23:20	trTeInstSyncMax	The maximum interval between instruction trace synchronization messages, with unit trace messages.	0000
17:16	trTeInstSyncMode	Select the periodic instruction trace synchronization message generation mechanism. 0: Off 1: Count trace messages. Other generation mechanisms are not supported.	00
15	trTeInhibitSrc	1: Disable inclusion of source field in trace messages.	0

Table 34: **Trace Encoder Control Register** *(cont'd)*

Bits	Name	Description	Reset Value
12	trTeInstStallOrOverflow	Set to 1 by hardware when trace buffer overflow occurs. Clears to 0 at reset or when the trace is enabled. Write 1 to clear.	0
9	trTeContext	Enable sending trace messages/fields with privilege levels	0
6:4	trTeInstMode	Instruction trace generation mode 0: Full Instruction trace is disabled. 3: Baseline instruction trace (Branch Trace Messaging). 6: Optimized instruction trace (History Trace Messaging). Other generation modes are not supported.	000
3	trTeEmpty	Reads as 1 when all generated trace have been emitted. Read only.	0
2	trTeInstTracing	1: Instruction trace is being generated.	0
1	trTeEnable	1: Trace Encoder is enabled.	0
0	trTeActive	Primary activate/reset bit for the TE.	0

Trace Encoder Instruction Features Register (trTeInstFeatures)

The Trace Encode Instruction Features Register implemented fields are listed below. The optional fields trTeInstNoAddrDiff, trTeInstNoTrapAddr, trTeInstEnSequentialJump, trTeInstEnImplicitReturn, trTeInstEnBranchPrediction, trTeInstEnJumpTargetCache, trTeInstImplicitReturnMode, trTeInstEnRepeatedHistory, trTeInstEnAllJumps, and trTeInstExtendAddrMSB are not implemented.

Figure 35: **Trace Encoder Instruction Features Register**

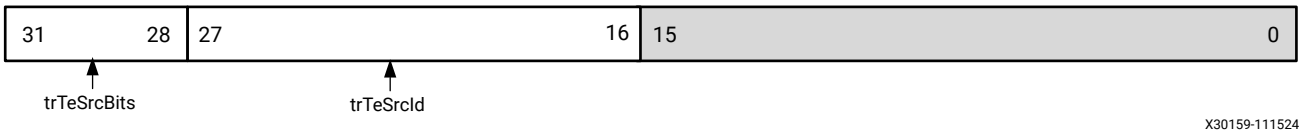


Table 35: **Trace Encoder Instruction Features Register**

Bits	Name	Description	Reset Value
31:28	trTeSrcBits	The number of bits in the trace source field (0..12),	0010
27:16	trTeSrcId	Trace source ID assigned to this trace encoder.	000000000000

Timestamp Control Register (trTsControl)

The Timestamp Control Register implemented fields are listed below. Only the Internal Core mode is implemented.

Figure 36: **Timestamp Control Register**

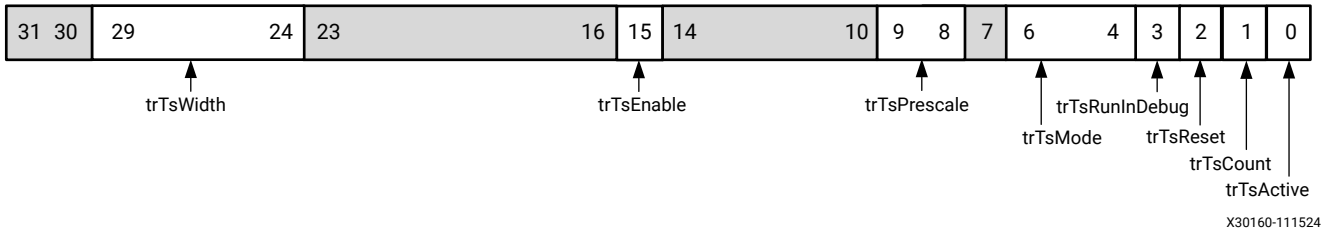


Table 36: **Timestamp Control Register**

Bits	Name	Description	Reset Value
29:24	trTsWidth	Width of timestamp in bits (0..63).	000000
15	trTsEnable	Enable for timestamp field in trace messages.	0
9:8	trTsPrescale	Prescale timestamp input clock by dividing by 1, 4, 16, or 64.	00
6:4	trTsMode	Mode used by Timestamp unit: Read only. 3: Internal Core	011
3	trTeRunInDebug	1: counter runs when hart is halted (in debug mode), 0: stopped	0
2	trTsReset	Write 1 to reset the timestamp counter. Write only.	0
1	trTsCount	1: counter runs, 0: counter stopped.	0
0	trTsActive	Primary activate/reset bit for timestamp unit.	0

RAM Sink Control Register (trRamControl)

The RAM Sink Control Register implemented fields are listed below. The optional field trRamAsyncFreq is not implemented.

Figure 37: **RAM Sink Control Register**

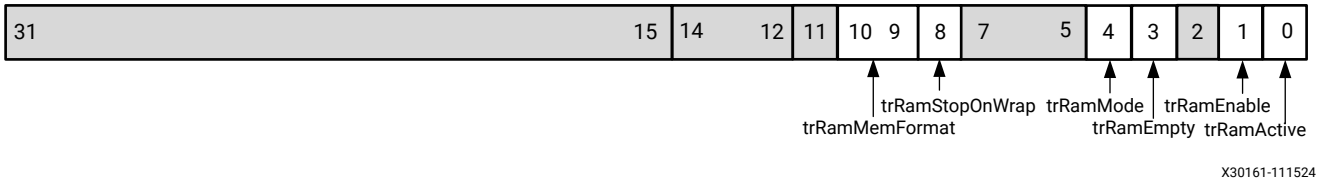


Table 37: **RAM Sink Control Register**

Bits	Name	Description	Reset Value
10:9	trRamMemFormat	00: Memory is formatted as plain bytes. Read only.	00
8	trRamStopOnWrap	1: Disable storing trace when the circular buffer gets full.	0
4	trRamMode	0: This RAM Sink operates in SRAM mode. Read only. SMEM mode is not supported.	0
3	trRamEmpty	Reads 1 when Trace RAM Sink internal buffers are empty. Read only.	0

Table 37: **RAM Sink Control Register** *(cont'd)*

Bits	Name	Description	Reset Value
1	trRamEnable	1: Trace RAM Sink enabled.	0
0	trRamActive	Primary activate/reset bit for Trace RAM Sink.	0

Optional Features

Optional Features

The following RISC-V N-Trace optional features are not supported:

- Virtual Address Optimization
- Sequential Jump Optimization
- Implicit Return Optimization
- Repeated History Optimization
- Trace Encoder Triggers
- System Memory Mode (SMEM) Trace RAM Sink
- Optional registers and register fields not listed above

Non-intrusive Profiling

With debugging enabled, non-intrusive profiling is provided, which uses a Profiling Buffer to store program execution statistics. The size of the profiling buffer can be configured from 4 KB to 128 KB using the parameter `C_DEBUG_PROFILE_SIZE`. By setting `C_DEBUG_PROFILE_SIZE` to 0 (None), non-intrusive profiling is disabled.

The Profiling Buffer is divided into a number of bins, each counting the number of executed instructions or clock cycles within a certain address range. Each bin counts up to 2^{36} - 1 = 68,719,476,735 instructions or cycles.

The address range of each bin is determined by the buffer size and the profiled address range defined using the Profiling Low Address Register and Profiling High Address Register.

Profiling can be started or stopped using the Profiling Control Register.

The memory/mapped registers used to configure and control profiling, and to read or write the Profiling Buffer, are listed in the following table.

Table 38: **MicroBlaze V Profiling Registers**

Register Name	Size (bits)	Offset	R/W	Description
Profiling Control	8	0x1040	W	Enable or disable profiling, configure counting method and bin usage
Profiling Low Address	32	0x1080	W	Defines the low address of the profiled address range
	C_ADDR_SIZE - 32	0x1084		
Profiling High Address	32	0x10C0	W	Defines the high address of the profiled address range
	C_ADDR_SIZE - 32	0x10C4		
Profiling Buffer Address	9 - 14	0x1100	W	Sets the address (bin) in the Profiling Buffer to read or write
Profiling Data Read	32	0x1180	R	Read data from the Profiling Buffer
	4	0x1184		
Profiling Data Write	32	0x11C0	W	Write data to the Profiling Buffer

Profiling Control Register

The Profiling Control Register (PCTRLR) is used to enable (start) profiling and disable (stop) profiling. It is also used to configure whether to count the number of executed instructions or the number of executed clock cycles, as well as define the Profiling Buffer bin usage.

This register is a write-only register. Issuing a read request has no effect, and undefined data is read. See the following figure and table.

The Bin Control value (B) can be calculated by the following formula:

Equation 1: **Bin Control Value (B)**

$$B = \left\lceil log_2 \frac{H - L + S \cdot 4}{S \cdot 4} \right\rceil$$

where:

- *L* is the Profiling Low Register
- *H* is the Profiling High Register
- *S* is the parameter `C_DEBUG_PROFILE_SIZE`.

Figure 38: **Profiling Control Register**

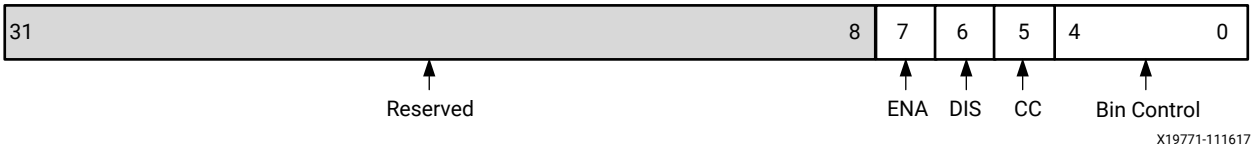


Table 39: **Profiling Control Register**

Bits	Name	Description	Reset Value
7	Enable	Enable and start profiling	0
6	Disable	Disable and stop profiling	0
5	Enable Cycle Count	Enable cycle count to count clock cycles of executed instruction: 0 = Disabled, number of executed instructions counted 1 = Enabled, clock cycles of executed instructions counted	0
4:0	Bin Control	The number of addresses counted by each bin in the Profiling Buffer	00000

Profiling Low Address Register

The Profiling Low Address Register (PLAR) is used to define the low word address of the profiled area. This register is a write-only register. Issuing a read request has no effect, and undefined data is read. See the following figure and table.

Figure 39: **Profiling Low Address Register**

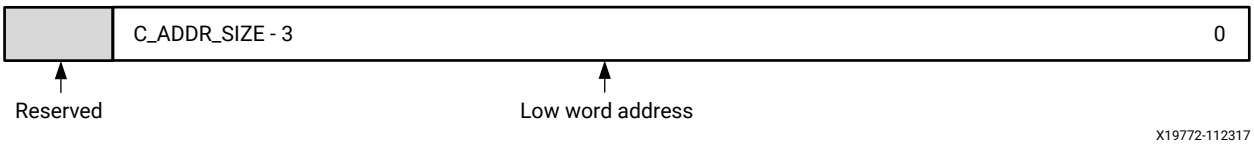


Table 40: **Profiling Low Address Register**

Bits	Name	Description	Reset Value
C_ADDR_SIZE-3:0	Low word	Low word address of the profiled area	0

Profiling High Address Register

The Profiling High Address Register (PHAR) is used to define the high word address of the profiled area. This register is a write-only register. Issuing a read request has no effect, and undefined data is read. See the following figure and table.

Figure 40: **Profiling High Address Register**

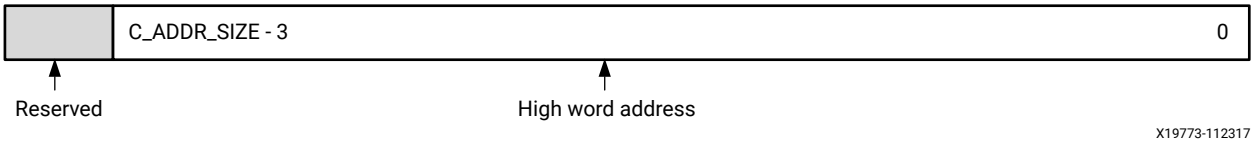


Table 41: **Profiling High Address Register**

Bits	Name	Description	Reset Value
C_ADDR_SIZE-3:0	High word	High word address of the profiled area	0

Profiling Buffer Address Register

he Profiling Buffer Address Register (PBAR) is used to define the bin in the Profiling Buffer to be read or written. This register has variable number of bits, depending on the parameter C_DEBUG_PROFILE_SIZE.

This register is a write-only register. Issuing a read request has no effect, and undefined data is read. See the following figure and table.

Figure 41: **Profiling Buffer Address Register**

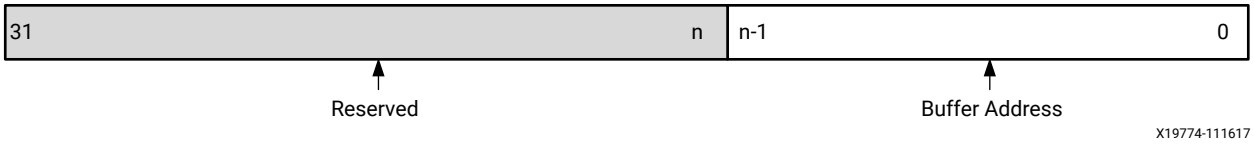


Table 42: **Profiling Buffer Address Register**

Bits	Name	Description	Reset Value
n-1:0	Buffer Address	Bin in the Profiling Buffer to read or write. The number of bits (n) is 10 for a 4 KB buffer, 11 for a 8 KB buffer, ..., 15 for a 128 KB buffer.	0

Profiling Data Read Register

The Profiling Data Read Register (PDRR) reads the bin value indicated by the Profiling Buffer Address Register and increments the Profiling Buffer Address Register. This register is a read-only register. Issuing a write request to the register does nothing. See the following figure and table.

When reading this register with MDM software access to debug registers, data is read with two consecutive accesses.

Figure 42: **Profiling Data Read Register**

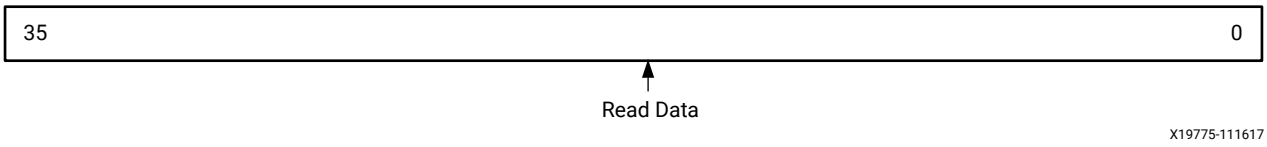


Table 43: **Profiling Data Read Register**

Bits	Name	Description	Reset Value
35:0	Read Data	Number of executed instructions or executed clock cycles in the bin	0

Profiling Data Write Register

The Profiling Data Write Register (PDWR) writes a new value to the bin indicated by the Profiling Buffer Address Register and increments the Profiling Buffer Address Register. This register is a write-only register. Issuing a read request has no effect, and undefined data is read.

This register can be used to clear the Profiling Buffer before enabling profiling.

The four most significant bits in the Profiling Buffer bin are set to zero when writing the new value. See the following figure and table.

Figure 43: **Profiling Data Write Register**

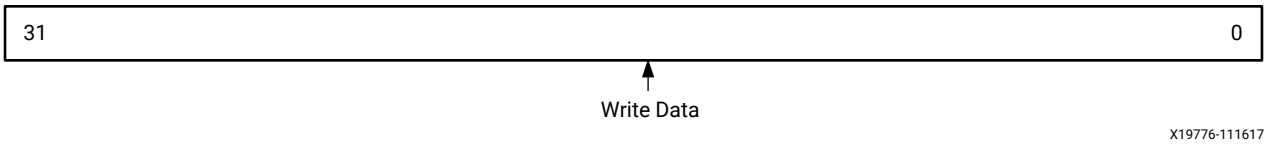


Table 44: **Profiling Data Write Register**

Bits	Name	Description	Reset Value
31:0	Write Data	Data to write to a bin.	0

Fault Tolerance

The fault tolerance features included in MicroBlaze V, enabled with `C_FAULT_TOLERANT`, provide error detection for internal block RAMs (in the instruction cache, data cache, branch target cache, and MMU), and support for error detection and correction (ECC) in LMB block RAMs. When fault tolerance is enabled, all soft errors in block RAMs are detected and corrected, which significantly reduces overall failure intensity.

In addition to protecting block RAM, the FPGA configuration memory also generally needs to be protected. A detailed explanation of this topic, and further references, can be found in *Soft Error Mitigation Controller LogiCORE IP Product Guide* (PG036) and *UltraScale Architecture Soft Error Mitigation Controller LogiCORE IP Product Guide* (PG187).

To further increase fault tolerance, a complete triple modular redundancy (TMR) solution is provided for MicroBlaze V processors, using additional cores to handle majority voting and fault detection. See the *Triple Modular Redundancy (TMR) LogiCORE IP Product Guide* (PG268) for a complete description and implementation details.

Configuration

Using MicroBlaze V Configuration

You can enable fault tolerance on the General page of the MicroBlaze V configuration dialog box.

After enabling fault tolerance in MicroBlaze V, ECC is automatically enabled in the connected LMB block RAM interface controllers by the tools when the system is generated. Consequently, nothing else needs to be configured to enable fault tolerance and minimal ECC support.

It is possible (albeit not recommended) to manually override ECC support, leaving the LMB block RAM unprotected, by disabling C_ECC in the configuration dialogs of all connected LMB block RAM interface controllers.

In this case, the internal MicroBlaze V block RAM protection is still enabled, because fault tolerance is enabled.

Using LMB Block RAM Interface Controller Configuration

As an alternative to the method described above, it is also possible to enable ECC in the configuration dialogs of all connected LMB block RAM interface controllers.

In this case, fault tolerance is automatically enabled in MicroBlaze V by the tools, when the system is generated. This means that nothing else needs to be configured to enable ECC support and MicroBlaze V fault tolerance.

ECC must either be enabled or disabled in all controllers, which is enforced by a DRC.

It is possible to manually override fault tolerance support in MicroBlaze V by explicitly disabling `C_FAULT_TOLERANT` in the MicroBlaze V configuration dialog. Doing this is not recommended, unless no block RAM is used in MicroBlaze V, and there is no need to handle bus exceptions from uncorrectable ECC errors.

Fault Tolerance Features

An overview of all MicroBlaze V fault tolerance features is given here. Further details on each feature can be found in the following sections:

- Instruction Cache
- Data Cache
- Exception Causes

The LMB BRAM Interface Controller v4.0 or later provides the LMB ECC implementation. For details, including performance and resource utilization, see the *LMB BRAM Interface Controller LogiCORE IP Product Guide* (PG112).

Instruction and Data Cache Protection

To protect the block RAM in the instruction and data cache, parity is used. When a parity error is detected, the corresponding cache line is invalidated. This forces the cache to reload the correct value from external memory. Parity is checked whenever a cache hit occurs.

***Note*:** This scheme only works for write-through, and thus write-back data cache is not available when fault tolerance is enabled. This is enforced by a DRC.

When new values are written to a block RAM in the cache, parity is also calculated and written. One parity bit is used for the tag, one parity bit for the instruction cache data, and one parity bit for each word in a data cache line.

In many cases, enabling fault tolerance does not increase the required number of cache block RAMs, since spare bits can be used for the parity. Any increase in resource utilization, in particular number of block RAMs, can easily be seen in the MicroBlaze V configuration dialog when enabling fault tolerance.

Branch Target Cache Protection

To protect block RAM in the branch target cache, parity is used. When a parity error is detected when looking up a branch target address, the address is ignored, forcing a normal branch.

When a new branch address is written to the branch target cache, parity is calculated. One parity bit is used for each address.

Enabling fault tolerance does not increase the branch target cache block RAM size, because a spare bit is available for the parity.

Exception Handling

With fault tolerance enabled, if an error occurs in LMB block RAM, the LMB BRAM Interface Controller generates error signals on the LMB interface.

The uncorrectable error signal either generates an instruction access fault exception, load access fault exception, or store/AMO access fault exception, depending on the affected interface.

Software Support

Scrubbing

To ensure that bit errors are not accumulated in block RAMs, they must be periodically scrubbed.

The standalone BSP provides the function `riscv_scrub()` to perform scrubbing of the entire LMB block RAM and all MicroBlaze internal block RAMs used in a particular configuration. This function is intended to be called periodically from a timer interrupt routine. One location of each block RAM is scrubbed every time it is called, using persistent data to track the current locations.

The following example code illustrates how this can be done.

```
#include "xparameters.h"
#include "xtmrctr.h"
#include "xintc.h"
#include "riscv_interface.h"

#define SCRUB_PERIOD ...

XIntc InterruptController; /* The Interrupt Controller instance */
XTmrCtr TimerCounterInst;/* The Timer Counter instance */

void ScrubHandler(void *CallBackRef, u8 TmrCtrNumber)
{
    /* Perform other timer interrupt processing here */
    riscv_scrub();
}

int main (void)
{
    int Status;

    /*
     * Initialize the timer counter so that it's ready to use,
     * specify the device ID that is generated in xparameters.h
     */
    Status = XTmrCtr_Initialize(&TimerCounterInst, TMRCTR_DEVICE_ID);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Connect the timer counter to the interrupt subsystem such that
     * interrupts can occur.
     */
    Status = XIntc_Initialize(&InterruptController, INTC_DEVICE_ID);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Connect a device driver handler that will be called when an
     * interrupt for the device occurs, the device driver handler performs
     * the specific interrupt processing for the device
     */
    Status = XIntc_Connect(&InterruptController, TMRCTR_DEVICE_ID,
        (XInterruptHandler)XTmrCtr_InterruptHandler,
        (void *) &TimerCounterInst);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Start the interrupt controller such that interrupts are enabled for
```

```
     * all devices that cause interrupts, specifying real mode so that the
     * timer counter can cause interrupts thru the interrupt controller.
     */
    Status = XIntc_Start(&InterruptController, XIN_REAL_MODE);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Setup the handler for the timer counter that will be called from the
     * interrupt context when the timer expires, specify a pointer to the
     * timer counter driver instance as the callback reference so the
     * handler is able to access the instance data
     */
    XTmrCtr_SetHandler(&TimerCounterInst, ScrubHandler,
        &TimerCounterInst);

    /*
     * Enable the interrupt of the timer counter so interrupts will occur
     * and use auto reload mode such that the timer counter will reload
     * itself automatically and continue repeatedly, without this option
     * it would expire once only
     */
    XTmrCtr_SetOptions(&TimerCounterInst, TIMER_CNTR_0,
        XTC_INT_MODE_OPTION | XTC_AUTO_RELOAD_OPTION);

    /*
     * Set a reset value for the timer counter such that it will expire
     * earlier than letting it roll over from 0, the reset value is loaded
     * into the timer counter when it is started
     */
    XTmrCtr_SetResetValue(TmrCtrInstancePtr,TmrCtrNumber,SCRUB_PERIOD);

    /*
     * Start the timer counter such that it's incrementing by default,
     * then wait for it to timeout a number of times
     */
    XTmrCtr_Start(&TimerCounterInst, TIMER_CNTR_0);

    ...
}
```

See Scrubbing for further details on how scrubbing is implemented, including how to calculate the scrubbing rate.

Block RAM Driver

The standalone BSP block RAM driver is used to access the ECC registers in the LMB BRAM Interface Controller, and also provides a comprehensive self test.

By implementing the Vitis platform C project Peripheral Tests, a self-test example including the block RAM self test for each LMB BRAM Interface Controller in the system is generated. Depending on the ECC features enabled in the LMB BRAM Interface Controller, this code performs all possible tests of the ECC function. See the *Vitis Unified Software Platform Documentation Landing Page* (UG1416) for more information.

The self-test example can be found in the standalone BSP block RAM driver source code, typically in the sub-directory `microblaze_riscv_0/libsrc/bram_v4_7/src/xbram_selftest.c`.

Scrubbing

Scrubbing Methods

Scrubbing is performed using specific methods for the different block RAMs:

- **Branch target cache:** The entire BTC is invalidated by doing a `fence.i` instruction.
- **LMB block RAM:** All addresses in the memory are cyclically read and written, thus correcting any single bit errors on each address.

It is also possible to add interrupts for correctable errors from the LMB BRAM Interface Controllers, and immediately scrub this address in the interrupt handler, although in most cases it only improves reliability slightly.

The failing address can be determined by reading the Correctable Error First Failing Address register in each of the LMB BRAM Interface Controllers.

To generate an interrupt, `C_ECC_STATUS_REGISTERS` must be set to 1 in the connected LMB BRAM Interface Controllers. To read the failing address, `C_CE_FAILING_REGISTERS` must be set to 1.

Calculating Scrubbing Rate

The scrubbing rate depends on failure intensity and desired reliability. The approximate equation to determine the LMB memory scrubbing rate in this case is given by:

$$P_W = 760 \left(BER^2 / SR^2 \right)$$

Where P_W is the probability of an uncorrectable error in a memory word, BER is the soft error rate for a single memory bit, and SR is the scrubbing rate.

The soft error rates affecting block RAM for each product family can be found in the *Device Reliability Report* (UG116).

Use Cases

Several common use cases are described in this section. These use cases are derived from the *LMB BRAM Interface Controller LogiCORE IP Product Guide* (PG112).

Minimal

This system is obtained when enabling fault tolerance in MicroBlaze V, without doing any other configuration.

The system is suitable when area constraints are high, and there is no need for testing of the ECC function, or for analysis of error frequency and location. No ECC registers are implemented. Single bit errors are corrected by the ECC logic before being passed to MicroBlaze V. Uncorrectable errors set an error signal, which generates an exception in MicroBlaze V.

Small

This system should be used when it is necessary to monitor error frequency, but there is no need for testing of the ECC function. It is a minimal system with a correctable error counter register added to monitor single bit error rates. If the error rate is too high, the scrubbing rate should be increased to minimize the risk of a single bit error becoming an uncorrectable double bit error. Parameters set are `C_ECC` = 1 and `C_CE_COUNTER_WIDTH` = 10.

Typical

This system represents a typical use case in which the system is required to monitor error frequency, and to generate an interrupt to immediately correct a single bit error through software. It does not provide support for testing of the ECC function.

It is a small system with Correctable Error First Failing registers and a Status register added. A single bit error latches the address for access into the Correctable Error First Failing Address register, and sets the CE_STATUS bit in the ECC Status register. An interrupt is generated, triggering MicroBlaze to read the failing address and then perform a read followed by a write on the failing address. This removes the single bit error from the block RAM, thus reducing the risk of the single bit error becoming a uncorrectable double bit error. The parameters set are as follows:

- `C_ECC` = 1
- `C_CE_COUNTER_WIDTH` = 10
- `C_ECC_STATUS_REGISTER` = 1
- `C_CE_FAILING_REGISTERS` = 1

Full

This system uses all of the features provided by the LMB BRAM Interface Controller to enable full error injection capability, error monitoring, and interrupt generation. It is a typical system with Uncorrectable Error First Failing registers and Fault Injection registers added. All features are switched on for full control of ECC functionality for system debug or systems with high fault tolerance requirements. The parameters are set as follows:

- C_ECC = 1
- C_CE_COUNTER_WIDTH = 10
- C_ECC_STATUS_REGISTER = 1
- C_CE_FAILING_REGISTERS = 1
- C_UE_FAILING_REGISTERS = 1
- C_FAULT_INJECT = 1

Lockstep Operation

MicroBlaze V is able to operate in a lockstep configuration, where two or more identical MicroBlaze V cores execute the same program. By comparing the outputs of the cores, any tampering attempts, transient faults or permanent hardware faults can be detected.

System Configuration

The parameter C_LOCKSTEP_SLAVE is set to 1 on all slave MicroBlaze V cores in the system, except the master (or primary) core. The master core drives all the output signals, and handles the debug functionality. The port Lockstep_Master_Out on the master is connected to the port Lockstep_Slave_In on the slaves to handle debugging.

The slave cores should not drive any output signals, and should only receive input signals. This constraint must be ensured by only connecting signals to the input ports of the slaves. For buses, this means that each individual input port must be explicitly connected.

The port Lockstep_Out on the master and slave cores provide all output signals for comparison. Unless an error occurs, individual signals from each of the cores are identical every clock cycle.

To ensure that lockstep operation works properly, all input signals to the cores must be synchronous. Input signals that could require external synchronization are Interrupt, Ext_Brk, Ext_NM_Brk, and Reset.

Use Cases

Common use cases are described in this section. In addition, lockstep operation provides the basis for implementing triple modular redundancy on the MicroBlaze V core level.

Tamper Protection

This application represents a high assurance use case, where it is required that the system is tamper-proof. A typically example is a cryptographic application.

The approach involves having two redundant MicroBlaze V processors with dedicated local memory and redundant comparators, each in a protected area. The outputs from each processor feed two comparators and each processor receive copies of every input signal.

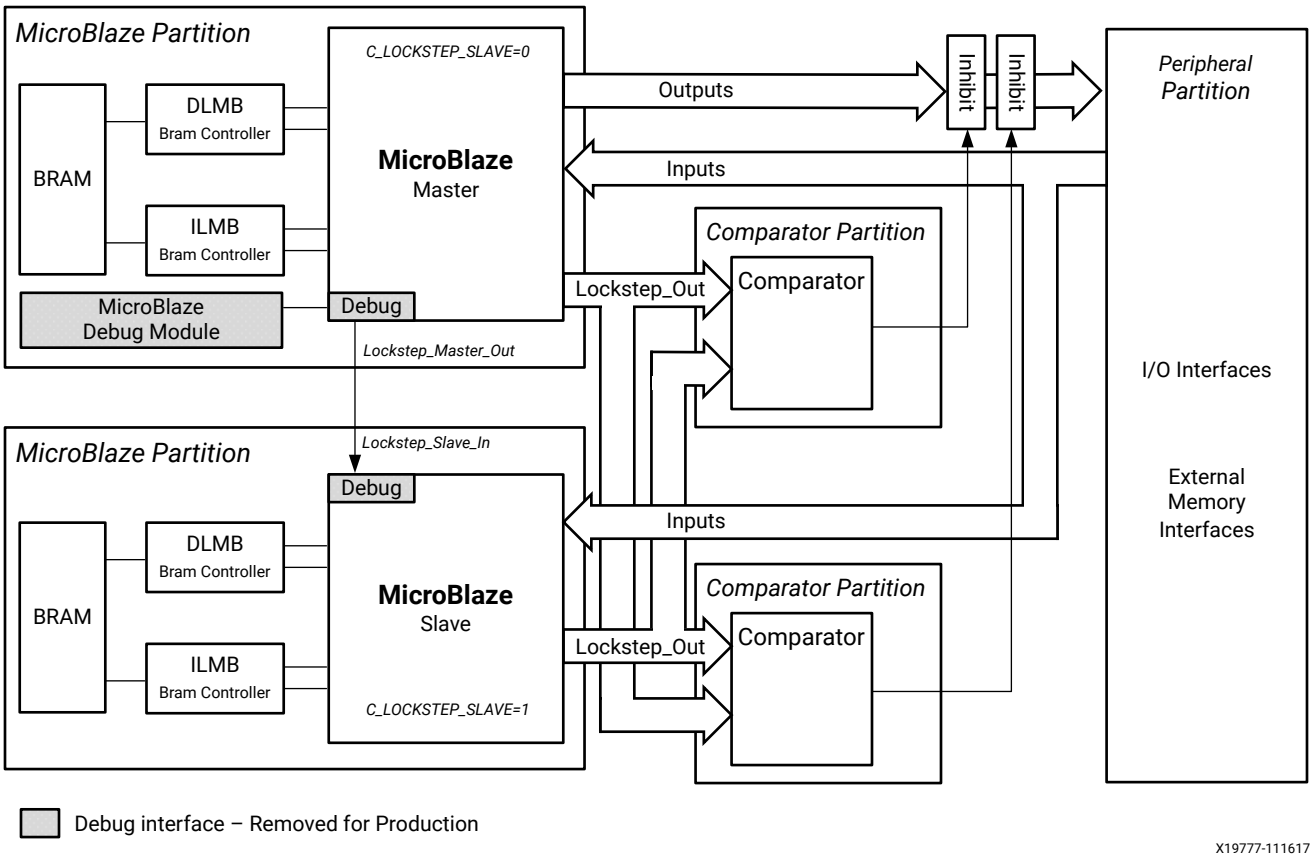
The redundant MicroBlaze V processors are functionally identical and completely independent of each other, without any connecting signals. The only exception is debug logic and associated signals, because it is assumed that debugging is disabled before any productization and certification of the system.

The outputs from the master MicroBlaze V core drive the peripherals in the system. All data leaving the protected area pass through inhibitors. Each inhibitor is controlled from its associated comparator.

Each protected area of the design must be implemented in its own partition, using a hierarchical single chip cryptography (SCC) flow. A detailed explanation of this flow, and further references, can be found in the *Hierarchical Design Methodology Guide* (UG748).

A block diagram of the system is shown in the following figure. The diagram is applicable for both MicroBlaze V and the classic MicroBlaze processor.

Figure 44: **System Block Diagram**



Error Detection

The error detection use case requires that all transient and permanent faults are detected. This is essential in fail safe and fault tolerant applications, where redundancy is utilized to improve system availability.

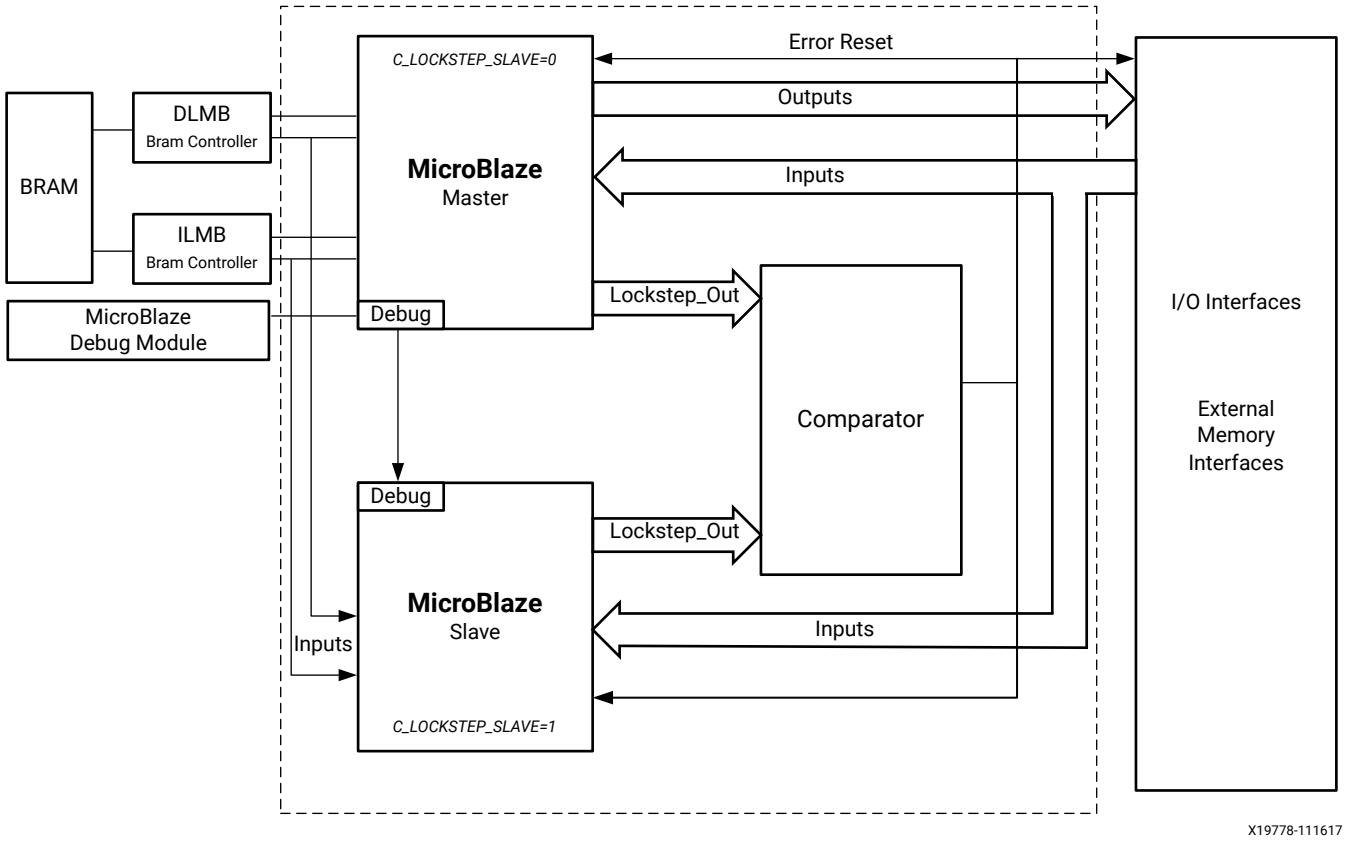
In this system two redundant MicroBlaze V processors run in lockstep. A comparator is used to signal an error when a mis-match is detected on the outputs of the two processors. Any error immediately causes both processors to halt, preventing further error propagation.

The redundant MicroBlaze V processors are functionally identical, except for debug logic and associated signals. The outputs from the master MicroBlaze V drive the peripherals in the system. The slave MicroBlaze V only has inputs connected; all outputs are left open.

The system contains the basic building block for designing a complete fault tolerant application, where one or more additional blocks must be added to provide redundancy.

This use case is illustrated in the following figure. The diagram is applicable for both MicroBlaze V and the classic MicroBlaze processor.

Figure 45: **Error Detection Use Case**



Coherency

MicroBlaze V supports cache coherency, as well as invalidation of caches and translation look-aside buffers, in hardware using the AXI Coherency Extension (ACE) defined in the *AMBA AXI and ACE Protocol Specification* (Arm IHI 0022H). The coherency support is enabled when the parameter `C_INTERCONNECT` is set to 3 (ACE).

Using ACE ensures coherency between the caches of all MicroBlaze V processors in the coherency domain. The peripheral ports (AXI_IP, AXI_DP) and local memory (ILMB, DLMB) are outside the coherency domain.

Coherency is not supported with write-back data cache, wide cache interfaces (more than 32-bit data), instruction cache streams, or instruction cache victims.

Invalidation

The coherency hardware in the caches handles invalidation in the following cases:

- **Data cache invalidation:** When a MicroBlaze V core in the coherency domain invalidates a data cache line with an external cache invalidation instruction, hardware messages ensure that all other cores in the coherency domain do the same. The physical address is always used.
- **Instruction cache invalidation:** When a MicroBlaze V core in the coherency domain invalidates an instruction cache line, hardware messages ensure that all other cores in the coherency domain do the same.
- **Branch target cache invalidation:** When a MicroBlaze V core in the coherency domain invalidates the branch target cache, either with a memory barrier instruction or with a synchronizing branch, hardware messages ensure that all other cores in the coherency domain do the same.

In particular, this means that self-modifying code can be used transparently within the coherency domain in a multi-processor system, provided that the guidelines regarding self-modifying code are followed.

Protocol Compliance

The MicroBlaze V instruction cache interface issues the following subset of the possible ACE transactions.

- **ReadClean:** Issued when a cache line is allocated.
- **ReadOnce:** Issued when the cache is off.

The MicroBlaze data cache interface issues the following subset of the possible ACE transactions:

- **ReadClean:** Issued when a cache line is allocated.
- **CleanUnique:** Issued when an SC instruction is executed as part of an exclusive access sequence.
- **ReadOnce:** Issued when the cache is off.
- **WriteUnique:** Issued whenever a store instruction performs a write.
- **CleanInvalid:** Issued when an external cache flush is executed.
- **MakeInvalid:** Issued when an external cache invalidate is executed.

Both interfaces issue the following subset of the possible distributed virtual memory (DVM) transactions:

- **DVM Operation:**
  - **Branch Predictor Invalidate:** L branch predictor invalidate all.

  - **Physical Instruction Cache Invalidate:** Non-secure physical instruction cache invalidate by PA without virtual index.
  - **Virtual Instruction Cache Invalidate:** Hypervisor invalidate by VA.
- **DVM Sync:** Synchronization.
- **DVM Complete:** Completion.

In addition to the DVM transactions above, the interfaces only accept the CleanInvalid and MakeInvalid transactions. These transactions have no effect in the instruction cache, and invalidate the indicated data cache lines. If any other transactions are received, the behavior is undefined. Only a subset of AXI4 transactions are used by the interfaces, as described in Cache Interfaces.

Data and Instruction Address Extension

MicroBlaze V has the ability to address up to 16 EB of data controlled by the parameter C_ADDR_SIZE.

With the 32-bit implementation (RV32), this is limited to 4 GB when physical memory protection (PMP) is enabled, and to 16 GB when Supervisor mode is enabled. Otherwise extended addressing up to 16 EB is provided by load and store custom instructions.

With the 64-bit implementation (RV64), this is limited to 64 PB when physical memory protection or Supervisor mode is enabled.

Table 45: **Address Extensions**

Addressable Size	Addressable Bytes	Address Bits	Configuration
4 GB	$4 * 1024^3$ bytes	32 bits	RV32 and RV64
16 GB	$16 * 1024^3$ bytes	34 bits	RV32, only supervisor mode Sv32 physical address
64 GB	$64 * 1024^3$ bytes	36 bits	RV64
1 TB	1024^4 bytes	40 bits	RV64
16 TB	$16 * 1024^4$ bytes	44 bits	RV64
256 TB	$256 * 1024^4$ bytes	48 bits	RV64
4 PB	$4 * 1024^5$ bytes	52 bits	RV64
64 PB	$64 * 1024^5$ bytes	56 bits	RV64, supervisor mode Sv39 physical address
16 EB	$16 * 1024^6$ bytes	64 bits	RV64, not available with PMP

There are a number of software limitations with extended addressing when using the 32-bit implementation:

- The GNU tools only generate ELF files with 32-bit addresses with RV32, which means that program instruction and data memory must be located in the first 4 GB of the address space. This is also the reason the instruction address space does not provide an extended address.
- Because all software drivers use address pointers that are 32-bit unsigned integers, it is not possible to access physical extended addresses above 4 GB without modifying the driver code, and consequently all AXI peripherals should be located in the first 4 GB of the address space.
- The extended address is only treated as a physical address, and is not available when enabling Supervisor Mode.
- The GNU compiler does not handle 64-bit address pointers, which means that the only way to access an extended address is using the custom extended addressing instructions, available as macros.

The following C code exemplifies how an extended address can be used to access data:

```
#include "xil_types.h"
#include "riscv_interface.h"

int main()
{
    u64 Addr = 0x000000FF00000000LL; /* Extended address */
    u32 Word;
    u16 Halfword;
    u8 Byte;

    Word = lwea(Addr);       /* Load word from extended address */
    swea(Addr, Word);        /* Store word to extended address */

    Halfword = lhuea(Addr); /* Load unsigned byte from extended address */
    shea(Addr, Halfword);    /* Store byte to extended address */

    Byte = lbuea(Addr);      /* Load unsigned byte from extended address */
    sbea(Addr, Byte);        /* Store byte to extended address */
}
```

Chapter 3

MicroBlaze V Signal Interface Description

This chapter describes the types of signal interfaces that can be used to connect a MicroBlaze™ V processor.

Overview

The MicroBlaze V core is organized as a Harvard architecture with separate bus interface units for data and instruction accesses. The following three types of memory interfaces are supported: Local Memory Bus (LMB), and the AMBA® AXI4 interface (AXI4) and ACE interface (ACE).

The LMB provides single-cycle access to on-chip dual-port block RAM or UltraRAM. The AXI4 interfaces provide a connection to both on-chip and off-chip peripherals and memory. The ACE interfaces provide cache-coherent connections to memory.

Features

MicroBlaze V can be configured with the following bus interfaces:

- The AMBA AXI4 interface for peripheral interfaces and the AMBA AXI4 or AXI Coherency Extension (ACE) interface for cache interfaces (see *AMBA AXI and ACE Protocol Specification* (Arm IHI 0022H)
- LMB provides a simple synchronous protocol for efficient block RAM transfers
- AXI4-Stream provides a fast non-arbitrated streaming communication mechanism
- Debug interface for use with the MicroBlaze Debug Module (MDM) V core
- Trace interface for performance analysis and low-level hardware debug

MicroBlaze V I/O Overview

The core interfaces shown in the following figure and table are defined as follows:

- **M_AXI_DP:** Peripheral data interface, AXI4-Lite or AXI4 interface
- **DLMB:** Data interface, local memory bus (block RAM only)
- **M_AXI_IP:** Peripheral instruction interface, AXI4-Lite interface
- **ILMB:** Instruction interface, local memory bus (block RAM only)
- **M0_AXIS..M15_AXIS:** AXI4-Stream interface master direct connection interfaces
- **S0_AXIS..S15_AXIS:** AXI4-Stream interface slave direct connection interfaces
- **M_AXI_DC:** Data-side cache AXI4 interface
- **M_ACE_DC:** Data-side cache AXI Coherency Extension (ACE) interface
- **M_AXI_IC:** Instruction-side cache AXI4 interface
- **M_ACE_IC:** Instruction-side cache AXI Coherency Extension (ACE) interface
- **S_AXI:** Slave AXI4-Lite interface to access core trace and profiling registers
- **Core:** Miscellaneous signals for clock, reset, interrupt, debug, and trace

Figure 46: **Core Interfaces**

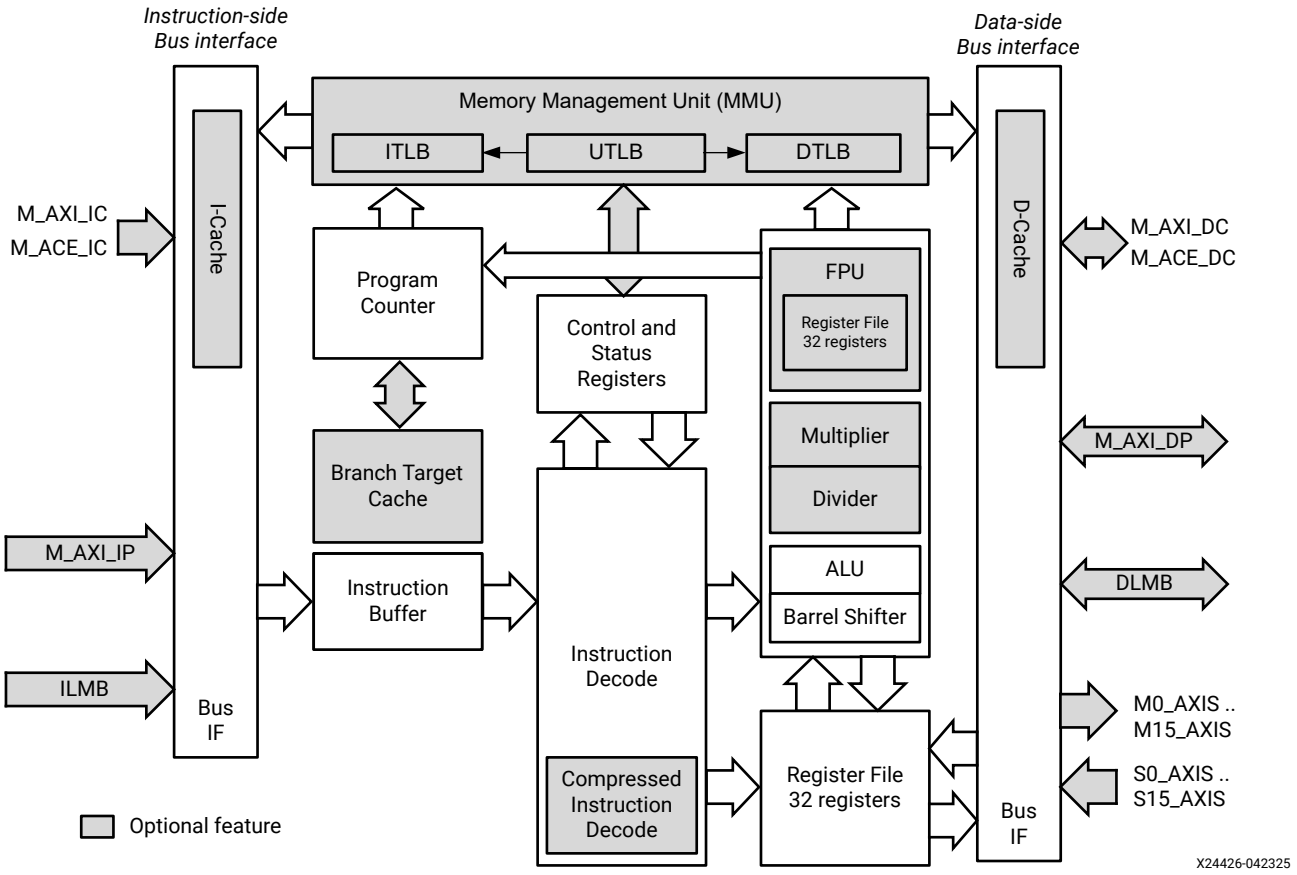


Table 46: **Summary of MicroBlaze V Core I/O**

Signal	Interface	I/O	Description
M_AXI_DP_AWID	M_AXI_DP	O	Master Write address ID
M_AXI_DP_AWADDR	M_AXI_DP	O	Master Write address
M_AXI_DP_AWLEN	M_AXI_DP	O	Master Burst length
M_AXI_DP_AWSIZE	M_AXI_DP	O	Master Burst size
M_AXI_DP_AWBURST	M_AXI_DP	O	Master Burst type
M_AXI_DP_AWLOCK	M_AXI_DP	O	Master Lock type
M_AXI_DP_AWCACHE	M_AXI_DP	O	Master Cache type
M_AXI_DP_AWPROT	M_AXI_DP	O	Master Protection type
M_AXI_DP_AWQOS	M_AXI_DP	O	Master Quality of Service
M_AXI_DP_AWVALID	M_AXI_DP	O	Master Write address valid
M_AXI_DP_AWREADY	M_AXI_DP	I	Slave Write address ready
M_AXI_DP_WDATA	M_AXI_DP	O	Master Write data
M_AXI_DP_WSTRB	M_AXI_DP	O	Master Write strobes
M_AXI_DP_WLAST	M_AXI_DP	O	Master Write last
M_AXI_DP_WVALID	M_AXI_DP	O	Master Write valid
M_AXI_DP_WREADY	M_AXI_DP	I	Slave Write ready
M_AXI_DP_BID	M_AXI_DP	I	Slave Response ID
M_AXI_DP_BRESP	M_AXI_DP	I	Slave Write response
M_AXI_DP_BVALID	M_AXI_DP	I	Slave Write response valid
M_AXI_DP_BREADY	M_AXI_DP	O	Master Response ready
M_AXI_DP_ARID	M_AXI_DP	O	Master Read address ID
M_AXI_DP_ARADDR	M_AXI_DP	O	Master Read address
M_AXI_DP_ARLEN	M_AXI_DP	O	Master Burst length
M_AXI_DP_ARSIZE	M_AXI_DP	O	Master Burst size
M_AXI_DP_ARBURST	M_AXI_DP	O	Master Burst type
M_AXI_DP_ARLOCK	M_AXI_DP	O	Master Lock type
M_AXI_DP_ARCACHE	M_AXI_DP	O	Master Cache type
M_AXI_DP_ARPROT	M_AXI_DP	O	Master Protection type
M_AXI_DP_ARQOS	M_AXI_DP	O	Master Quality of Service
M_AXI_DP_ARVALID	M_AXI_DP	O	Master Read address valid
M_AXI_DP_ARREADY	M_AXI_DP	I	Slave Read address ready
M_AXI_DP_RID	M_AXI_DP	I	Slave Read ID tag
M_AXI_DP_RDATA	M_AXI_DP	I	Slave Read data
M_AXI_DP_RRESP	M_AXI_DP	I	Slave Read response
M_AXI_DP_RLAST	M_AXI_DP	I	Slave Read last
M_AXI_DP_RVALID	M_AXI_DP	I	Slave Read valid
M_AXI_DP_RREADY	M_AXI_DP	O	Master Read ready
M_AXI_IP_AWID	M_AXI_IP	O	Master Write address ID
M_AXI_IP_AWADDR	M_AXI_IP	O	Master Write address

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
M_AXI_IP_AWLEN	M_AXI_IP	O	Master Burst length
M_AXI_IP_AWSIZE	M_AXI_IP	O	Master Burst size
M_AXI_IP_AWBURST	M_AXI_IP	O	Master Burst type
M_AXI_IP_AWLOCK	M_AXI_IP	O	Master Lock type
M_AXI_IP_AWCACHE	M_AXI_IP	O	Master Cache type
M_AXI_IP_AWPROT	M_AXI_IP	O	Master Protection type
M_AXI_IP_AWQOS	M_AXI_IP	O	Master Quality of Service
M_AXI_IP_AWVALID	M_AXI_IP	O	Master Write address valid
M_AXI_IP_AWREADY	M_AXI_IP	I	Slave Write address ready
M_AXI_IP_WDATA	M_AXI_IP	O	Master Write data
M_AXI_IP_WSTRB	M_AXI_IP	O	Master Write strobes
M_AXI_IP_WLAST	M_AXI_IP	O	Master Write last
M_AXI_IP_WVALID	M_AXI_IP	O	Master Write valid
M_AXI_IP_WREADY	M_AXI_IP	I	Slave Write ready
M_AXI_IP_BID	M_AXI_IP	I	Slave Response ID
M_AXI_IP_BRESP	M_AXI_IP	I	Slave Write response
M_AXI_IP_BVALID	M_AXI_IP	I	Slave Write response valid
M_AXI_IP_BREADY	M_AXI_IP	O	Master Response ready
M_AXI_IP_ARID	M_AXI_IP	O	Master Read address ID
M_AXI_IP_ARADDR	M_AXI_IP	O	Master Read address
M_AXI_IP_ARLEN	M_AXI_IP	O	Master Burst length
M_AXI_IP_ARSIZE	M_AXI_IP	O	Master Burst size
M_AXI_IP_ARBURST	M_AXI_IP	O	Master Burst type
M_AXI_IP_ARLOCK	M_AXI_IP	O	Master Lock type
M_AXI_IP_ARCACHE	M_AXI_IP	O	Master Cache type
M_AXI_IP_ARPROT	M_AXI_IP	O	Master Protection type
M_AXI_IP_ARQOS	M_AXI_IP	O	Master Quality of Service
M_AXI_IP_ARVALID	M_AXI_IP	O	Master Read address valid
M_AXI_IP_ARREADY	M_AXI_IP	I	Slave Read address ready
M_AXI_IP_RID	M_AXI_IP	I	Slave Read ID tag
M_AXI_IP_RDATA	M_AXI_IP	I	Slave Read data
M_AXI_IP_RRESP	M_AXI_IP	I	Slave Read response
M_AXI_IP_RLAST	M_AXI_IP	I	Slave Read last
M_AXI_IP_RVALID	M_AXI_IP	I	Slave Read valid
M_AXI_IP_RREADY	M_AXI_IP	O	Master Read ready
M_AXI_DC_AWADDR	M_AXI_DC	O	Master Write address
M_AXI_DC_AWLEN	M_AXI_DC	O	Master Burst length
M_AXI_DC_AWSIZE	M_AXI_DC	O	Master Burst size
M_AXI_DC_AWBURST	M_AXI_DC	O	Master Burst type

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
M_AXI_DC_AWLOCK	M_AXI_DC	O	Master Lock type
M_AXI_DC_AWCACHE	M_AXI_DC	O	Master Cache type
M_AXI_DC_AWPROT	M_AXI_DC	O	Master Protection type
M_AXI_DC_AWQOS	M_AXI_DC	O	Master Quality of Service
M_AXI_DC_AWVALID	M_AXI_DC	O	Master Write address valid
M_AXI_DC_AWREADY	M_AXI_DC	I	Slave Write address ready
M_AXI_DC_AWUSER	M_AXI_DC	O	Master Write address user signals
M_AXI_DC_AWDOMAIN	M_ACE_DC	O	Master Write address domain
M_AXI_DC_AWSNOOP	M_ACE_DC	O	Master Write address snoop
M_AXI_DC_AWBAR	M_ACE_DC	O	Master Write address barrier
M_AXI_DC_WDATA	M_AXI_DC	O	Master Write data
M_AXI_DC_WSTRB	M_AXI_DC	O	Master Write strobes
M_AXI_DC_WLAST	M_AXI_DC	O	Master Write last
M_AXI_DC_WVALID	M_AXI_DC	O	Master Write valid
M_AXI_DC_WREADY	M_AXI_DC	I	Slave Write ready
M_AXI_DC_WUSER	M_AXI_DC	O	Master Write user signals
M_AXI_DC_BRESP	M_AXI_DC	I	Slave Write response
M_AXI_DC_BID	M_AXI_DC	I	Slave Response ID
M_AXI_DC_BVALID	M_AXI_DC	I	Slave Write response valid
M_AXI_DC_BREADY	M_AXI_DC	O	Master Response ready
M_AXI_DC_BUSER	M_AXI_DC	I	Slave Write response user signals
M_AXI_DC_WACK	M_ACE_DC	O	Slave Write acknowledge
M_AXI_DC_ARID	M_AXI_DC	O	Master Read address ID
M_AXI_DC_ARADDR	M_AXI_DC	O	Master Read address
M_AXI_DC_ARLEN	M_AXI_DC	O	Master Burst length
M_AXI_DC_ARSIZE	M_AXI_DC	O	Master Burst size
M_AXI_DC_ARBURST	M_AXI_DC	O	Master Burst type
M_AXI_DC_ARLOCK	M_AXI_DC	O	Master Lock type
M_AXI_DC_ARCACHE	M_AXI_DC	O	Master Cache type
M_AXI_DC_ARPROT	M_AXI_DC	O	Master Protection type
M_AXI_DC_ARQOS	M_AXI_DC	O	Master Quality of Service
M_AXI_DC_ARVALID	M_AXI_DC	O	Master Read address valid
M_AXI_DC_ARREADY	M_AXI_DC	I	Slave Read address ready
M_AXI_DC_ARUSER	M_AXI_DC	O	Master Read address user signals
M_AXI_DC_ARDOMAIN	M_ACE_DC	O	Master Read address domain
M_AXI_DC_ARSNOOP	M_ACE_DC	O	Master Read address snoop
M_AXI_DC_ARBAR	M_ACE_DC	O	Master Read address barrier
M_AXI_DC_RID	M_AXI_DC	I	Slave Read ID tag
M_AXI_DC_RDATA	M_AXI_DC	I	Slave Read data

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
M_AXI_DC_RRESP	M_AXI_DC	I	Slave Read response
M_AXI_DC_RLAST	M_AXI_DC	I	Slave Read last
M_AXI_DC_RVALID	M_AXI_DC	I	Slave Read valid
M_AXI_DC_RREADY	M_AXI_DC	O	Master Read ready
M_AXI_DC_RUSER	M_AXI_DC	I	Slave Read user signals
M_AXI_DC_RACK	M_ACE_DC	O	Master Read acknowledge
M_AXI_DC_ACVALID	M_ACE_DC	I	Slave Snoop address valid
M_AXI_DC_ACADDR	M_ACE_DC	I	Slave Snoop address
M_AXI_DC_ACSNOOP	M_ACE_DC	I	Slave Snoop address snoop
M_AXI_DC_ACPROT	M_ACE_DC	I	Slave Snoop address protection type
M_AXI_DC_ACREADY	M_ACE_DC	O	Master Snoop ready
M_AXI_DC_CRREADY	M_ACE_DC	I	Slave Snoop response ready
M_AXI_DC_CRVALID	M_ACE_DC	O	Master Snoop response valid
M_AXI_DC_CRRESP	M_ACE_DC	O	Master Snoop response
M_AXI_DC_CDVALID	M_ACE_DC	O	Master Snoop data valid
M_AXI_DC_CDREADY	M_ACE_DC	I	Slave Snoop data ready
M_AXI_DC_CDDATA	M_ACE_DC	O	Master Snoop data
M_AXI_DC_CDLAST	M_ACE_DC	O	Master Snoop data last
M_AXI_IC_AWID	M_AXI_IC	O	Master Write address ID
M_AXI_IC_AWADDR	M_AXI_IC	O	Master Write address
M_AXI_IC_AWLEN	M_AXI_IC	O	Master Burst length
M_AXI_IC_AWSIZE	M_AXI_IC	O	Master Burst size
M_AXI_IC_AWBURST	M_AXI_IC	O	Master Burst type
M_AXI_IC_AWLOCK	M_AXI_IC	O	Master Lock type
M_AXI_IC_AWCACHE	M_AXI_IC	O	Master Cache type
M_AXI_IC_AWPROT	M_AXI_IC	O	Master Protection type
M_AXI_IC_AWQOS	M_AXI_IC	O	Master Quality of Service
M_AXI_IC_AWVALID	M_AXI_IC	O	Master Write address valid
M_AXI_IC_AWREADY	M_AXI_IC	I	Slave Write address ready
M_AXI_IC_AWUSER	M_AXI_IC	O	Master Write address user signals
M_AXI_IC_AWDOMAIN	M_ACE_IC	O	Master Write address domain
M_AXI_IC_AWSNOOP	M_ACE_IC	O	Master Write address snoop
M_AXI_IC_AWBAR	M_ACE_IC	O	Master Write address barrier
M_AXI_IC_WDATA	M_AXI_IC	O	Master Write data
M_AXI_IC_WSTRB	M_AXI_IC	O	Master Write strobes
M_AXI_IC_WLAST	M_AXI_IC	O	Master Write last
M_AXI_IC_WVALID	M_AXI_IC	O	Master Write valid
M_AXI_IC_WREADY	M_AXI_IC	I	Slave Write ready
M_AXI_IC_WUSER	M_AXI_IC	O	Master Write user signals

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
M_AXI_IC_BID	M_AXI_IC	I	Slave Response ID
M_AXI_IC_BRESP	M_AXI_IC	I	Slave Write response
M_AXI_IC_BVALID	M_AXI_IC	I	Slave Write response valid
M_AXI_IC_BREADY	M_AXI_IC	O	Master Response ready
M_AXI_IC_BUSER	M_AXI_IC	I	Slave Write response user signals
M_AXI_IC_WACK	M_ACE_IC	O	Slave Write acknowledge
M_AXI_IC_ARID	M_AXI_IC	O	Master Read address ID
M_AXI_IC_ARADDR	M_AXI_IC	O	Master Read address
M_AXI_IC_ARLEN	M_AXI_IC	O	Master Burst length
M_AXI_IC_ARSIZE	M_AXI_IC	O	Master Burst size
M_AXI_IC_ARBURST	M_AXI_IC	O	Master Burst type
M_AXI_IC_ARLOCK	M_AXI_IC	O	Master Lock type
M_AXI_IC_ARCACHE	M_AXI_IC	O	Master Cache type
M_AXI_IC_ARPROT	M_AXI_IC	O	Master Protection type
M_AXI_IC_ARQOS	M_AXI_IC	O	Master Quality of Service
M_AXI_IC_ARVALID	M_AXI_IC	O	Master Read address valid
M_AXI_IC_ARREADY	M_AXI_IC	I	Slave Read address ready
M_AXI_IC_ARUSER	M_AXI_IC	O	Master Read address user signals
M_AXI_IC_ARDOMAIN	M_ACE_IC	O	Master Read address domain
M_AXI_IC_ARSNOOP	M_ACE_IC	O	Master Read address snoop
M_AXI_IC_ARBAR	M_ACE_IC	O	Master Read address barrier
M_AXI_IC_RID	M_AXI_IC	I	Slave Read ID tag
M_AXI_IC_RDATA	M_AXI_IC	I	Slave Read data
M_AXI_IC_RRESP	M_AXI_IC	I	Slave Read response
M_AXI_IC_RLAST	M_AXI_IC	I	Slave Read last
M_AXI_IC_RVALID	M_AXI_IC	I	Slave Read valid
M_AXI_IC_RREADY	M_AXI_IC	O	Master Read ready
M_AXI_IC_RUSER	M_AXI_IC	I	Slave Read user signals
M_AXI_IC_RACK	M_ACE_IC	O	Master Read acknowledge
M_AXI_IC_ACVALID	M_ACE_IC	I	Slave Snoop address valid
M_AXI_IC_ACADDR	M_ACE_IC	I	Slave Snoop address
M_AXI_IC_ACSNOOP	M_ACE_IC	I	Slave Snoop address snoop
M_AXI_IC_ACPROT	M_ACE_IC	I	Slave Snoop address protection type
M_AXI_IC_ACREADY	M_ACE_IC	O	Master Snoop ready
M_AXI_IC_CRREADY	M_ACE_IC	I	Slave Snoop response ready
M_AXI_IC_CRVALID	M_ACE_IC	O	Master Snoop response valid
M_AXI_IC_CRRESP	M_ACE_IC	O	Master Snoop response
M_AXI_IC_CDVALID	M_ACE_IC	O	Master Snoop data valid
M_AXI_IC_CDREADY	M_ACE_IC	I	Slave Snoop data ready

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
M_AXI_IC_CDDATA	M_ACE_IC	O	Master Snoop data
M_AXI_IC_CDLAST	M_ACE_IC	O	Master Snoop data last
Data_Addr[0:N-1]	DLMB	O	Data interface LMB address bus, N = 32 - 64
Byte_Enable[0:3]	DLMB	O	Data interface LMB byte enables
Data_Write[0:31]	DLMB	O	Data interface LMB write data bus
D_AS	DLMB	O	Data interface LMB address strobe
Read_Strobe	DLMB	O	Data interface LMB read strobe
Write_Strobe	DLMB	O	Data interface LMB write strobe
Data_Read[0:31]	DLMB	I	Data interface LMB read data bus
DReady	DLMB	I	Data interface LMB data ready
DWait	DLMB	I	Data interface LMB data wait
DCE	DLMB	I	Data interface LMB correctable error
DUE	DLMB	I	Data interface LMB uncorrectable error
Instr_Addr[0:N-1]	ILMB	O	Instruction interface LMB address bus, N = 32 - 64
I_AS	ILMB	O	Instruction interface LMB address strobe
IFetch	ILMB	O	Instruction interface LMB instruction fetch
Instr[0:31]	ILMB	I	Instruction interface LMB read data bus
IReady	ILMB	I	Instruction interface LMB data ready
IWait	ILMB	I	Instruction interface LMB data wait
ICE	ILMB	I	Instruction interface LMB correctable error
IUE	ILMB	I	Instruction interface LMB uncorrectable error
Mn_AXIS_TLAST	M0_AXIS.. M15_AXIS	O	Master interface output AXI4 channels write last
Mn_AXIS_TDATA	M0_AXIS.. M15_AXIS	O	Master interface output AXI4 channels write data
Mn_AXIS_TVALID	M0_AXIS.. M15_AXIS	O	Master interface output AXI4 channels write valid
Mn_AXIS_TREADY	M0_AXIS.. M15_AXIS	I	Master interface input AXI4 channels write ready
Sn_AXIS_TLAST	S0_AXIS.. S15_AXIS	I	Slave interface input AXI4 channels write last
Sn_AXIS_TDATA	S0_AXIS.. S15_AXIS	I	Slave interface input AXI4 channels write data
Sn_AXIS_TVALID	S0_AXIS.. S15_AXIS	I	Slave interface input AXI4 channels write valid
Sn_AXIS_TREADY	S0_AXIS.. S15_AXIS	O	Slave interface output AXI4 channels write ready
S_AXI_AWADDR	S_AXI	I	Slave Write address
S_AXI_AWVALID	S_AXI	I	Slaver Write address valid
S_AXI_AWREADY	S_AXI	O	Master Write address ready

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
S_AXI_WDATA	S_AXI	I	Slave Write data
S_AXI_WVALID	S_AXI	I	Slave Write valid
S_AXI_WREADY	S_AXI	O	Master Write ready
S_AXI_BRESP	S_AXI	O	Master Write response
S_AXI_BVALID	S_AXI	O	Master Write response valid
S_AXI_BREADY	S_AXI	I	Slave Response ready
S_AXI_ARADDR	S_AXI	I	Slave Read address
S_AXI_ARVALID	S_AXI	I	Slave Read address valid
S_AXI_ARREADY	S_AXI	O	Master Read address ready
S_AXI_RDATA	S_AXI	O	Master Read data
S_AXI_RRESP	S_AXI	O	Master Read response
S_AXI_RVALID	S_AXI	O	Master Read valid
S_AXI_RREADY	S_AXI	I	Slave Read ready
Interrupt	Core	I	Interrupt. The signal is synchronized to Clk if the parameter C_ASYNC_INTERRUPT is set.
Interrupt_Address[1]	Core	I	Interrupt vector address
Interrupt_Ack[1]	Core	O	Interrupt acknowledge
Reset	Core	I	Core reset, active-High. Must be asserted one Clk clock cycle, but it is recommended to keep it asserted for at least 16 clock cycles.
Reset_Mode[0:1][3]	Core	I	Reset mode. Sampled when reset is active.
Clk	Core	I	Clock[2]
Ext_BRK[3]	Core	I	Break signal, implemented as RISC-V platform interrupt
Ext_NM_BRK[3]	Core	I	Non-maskable break, implemented as RISC-V non-maskable interrupt
Halted[3]	Core	O	Pipeline is halted, either using the Debug interface, by setting Dbg_Stop, or by setting Reset_Mode[0:1] to 10.
Dbg_Stop[3]	Core	I	Unconditionally force pipeline to halt as soon as possible. Rising-edge detected pulse that should be held for at least one Clk clock cycle. The signal only has any effect when C_DEBUG_ENABLED is greater than 0.
Sleep[3]	Core	O	MicroBlaze V is in sleep mode after executing a wfi instruction or by setting Reset_Mode[0:1] to 10. All external accesses are completed, and the pipeline is halted. Set when mwfi = 1 or C_USE_SLEEP = 1.
Hibernate[3]	Core	O	MicroBlaze V is in sleep mode after executing a wfi instruction or by setting Reset_Mode[0:1] to 10. All external accesses are completed, and the pipeline is halted. Set when mwfi = 2 or C_USE_SLEEP = 2.
Suspend[3]	Core	O	MicroBlaze V is in sleep mode after executing a wfi instruction or by setting Reset_Mode[0:1] to 10. All external accesses are completed, and the pipeline is halted. Set when mwfi = 4 or C_USE_SLEEP = 4.

Table 46: **Summary of MicroBlaze V Core I/O** *(cont'd)*

Signal	Interface	I/O	Description
Wakeup[0:1][3]	Core	I	Wake MicroBlaze V from sleep mode when either or both bits are set to 1. Ignored if not in sleep mode. The signals are individually synchronized to Clk according to the parameter C_ASYNC_WAKEUP[0:1].
Dbg_Wakeup[3]	Core	O	Debug request that external logic should wake MicroBlaze V from sleep mode with the Wakeup signal, to allow debug access. Synchronous to Dbg_Update.
Pause[3]	Core	I	When this signal is set, the MicroBlaze V pipeline is paused after completing all ongoing bus accesses, and the Pause_Ack signal is set. When this signal is cleared again MicroBlaze V continues normal execution where it was paused.
Pause_Ack[3]	Core	O	MicroBlaze V is in pause mode after the Pause input signal has been set.
Dbg_Continue[3]	Core	O	Debug request that external logic should clear the Pause signal, to allow debug access.
Non_Secure[0:3][3]	Core	I	Determines whether AXI accesses are non-secure or secure. The default value is binary 0000, setting all interfaces to be secure. Bit 0 = M_AXI_DP Bit 1 = M_AXI_IP Bit 2 = M_AXI_DC Bit 3 = M_AXI_IC
Lockstep_...	Core	IO	Lockstep signals for high integrity applications
Dbg_...	Core	IO	Debug signals from MDM V
Trace_...	Core	O	Trace signals for real time hardware analysis

Notes:

1. Only used with `C_USE_INTERRUPT` = 2, for low-latency interrupt support.
2. MicroBlaze V is a synchronous design clocked with the `Clk` signal, except for serial hardware debug logic, which is clocked with the `Dbg_Clk` signal. If serial hardware debug logic is not used, there is no minimum frequency limit for `Clk`. However, if serial hardware debug logic is used, there are signals transferred between the two clock regions. In this case `Clk` must have a higher frequency than `Dbg_Clk`.
3. Only visible when `C_ENABLE_DISCRETE_PORTS` = 1.

Table 47: **Effect of Reset Mode Inputs**

Reset_Mode[0:1]	Description
00	MicroBlaze V starts executing at the reset vector, defined by C_BASE_VECTORS. This is the nominal default behavior.
01	MicroBlaze V immediately enters sleep mode without performing any bus access, as if a wfi instruction has been executed. The sleep/hibernate/suspend output is set to 1. When any of the Wakeup[0:1] signals is set, MicroBlaze V starts executing at the reset vector, defined by C_BASE_VECTORS. This functionality can be useful in a multiprocessor configuration.
10	If C_DEBUG_ENABLED is 0, the behavior is the same as if Reset_Mode[0:1] = 00. If C_DEBUG_ENABLED is greater than 0, MicroBlaze V immediately enters debug halt without performing any bus access, the DCSR cause is set to 5, and the Halted output is set to 1. When execution is continued via the debug interface, MicroBlaze V starts executing at the reset vector, defined by C_BASE_VECTORS.

Table 47: **Effect of Reset Mode Inputs** *(cont'd)*

Reset_Mode[0:1]	Description
11	Reserved

In general, MicroBlaze V signals are synchronous to the `Clk` input signal. However, there are some exceptions controlled by parameters as described in the following table.

Table 48: **Parameter Controlled Asynchronous Signals**

Signal	Parameter	Default	Description
Interrupt	C_ASYNC_INTERRUPT	Tool controlled	Parameter set from connected signal
Reset	C_NUM_SYNC_FF_CLK	2	Parameter can be manually set to 0 for synchronous reset
Wakeup[0:1]	C_ASYNC_WAKEUP C_NUM_SYNC_FF_CLK	Tool controlled 2	Set from connected signals. Can be manually set to 0 to override tool.
Dbg_Wakeup	C_DEBUG_INTERFACE	0 (serial)	0: Clocked by Dbg_Update. 1: Clocked by DEBUG_ACLK, synchronous to Clk.

Sleep and Pause Functionality

There are two distinct ways of halting MicroBlaze V execution in a controlled manner:

- Software controlled by executing a `wfi` instruction to enter sleep mode.
- Hardware controlled by setting the input signal `Pause` to pause the pipeline.

Software Controlled

When a `wfi` instruction is executed to enter sleep mode and MicroBlaze V has completed all external accesses, the pipeline is halted and the `sleep`, `hibernate`, or `suspend` output signal is set.

This indicates to external hardware that it is safe to perform actions such as stopping the clock, resetting the processor or other IP cores. Different actions can be performed depending on which output signal is set. To wake up MicroBlaze V when in sleep mode, one (or both) of the `Wakeup` input signals must be set to one. In this case MicroBlaze V continues execution after the `wfi` instruction.

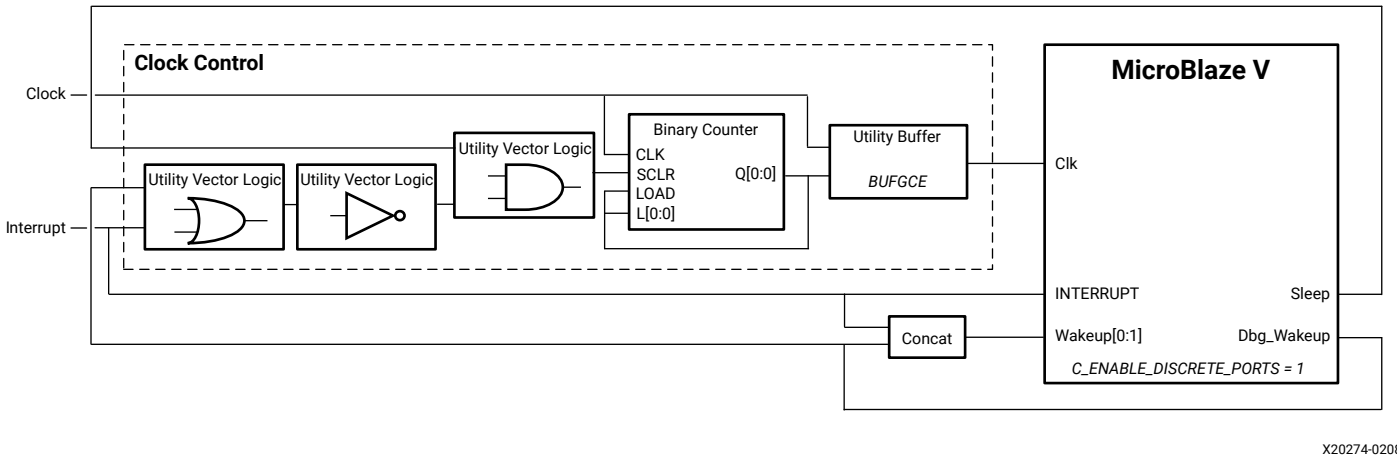
The `Dbg_Wakeup` output signal from MicroBlaze V indicates that the debugger requests a wake up. External hardware should handle this signal and wake up the processor, after performing any other necessary hardware actions such as starting the clock. If debug wake up is used, the software must be aware that this could be the reason for waking up, and go to sleep again if no other action is required.

In the simplest case, where no additional actions are needed before waking up the processor, one of the `Wakeup` inputs can be connected to the same signal as the MicroBlaze V `Interrupt` input, and the other to the MicroBlaze `Dbg_Wakeup` output. This allows MicroBlaze V to wake up when an interrupt occurs, or when the debugger requests it.

For full compliance with the RISC-V Instruction Set Manual, the external interrupt input should be connected to one of the `Wakeup` signals.

The block diagram in the following figure illustrates how to use the sleep functionality to implement clock control. In this example, the clock is stopped when sleep is executed and any interrupt or debug command enables the clock and wakes the processor.

Figure 47: **Clock Control Using the Sleep Functionality**



Instead of implementing the clock control with IP cores, an RTL Module can be used. A possible VHDL implementation corresponding to Clock Control in the block diagram in the figure is given here. See the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) for more information on RTL modules.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

library UNISIM;
use UNISIM.VComponents.all;

entity clock_control is
    port (
        clkin      : in std_logic;
        reset      : in std_logic;
        sleep      : in std_logic;
        interrupt  : in std_logic;
        dbg_wakeup : in std_logic;
        clkout     : out std_logic
    );
end clock_control;

architecture Behavioral of clock_control is
    attribute X_INTERFACE_INFO : string;
```

```
    attribute X_INTERFACE_INFO of clkin : signal
        is "xilinx.com:signal:clock:1.0 clk CLK";
    attribute X_INTERFACE_INFO of reset : signal
        is "xilinx.com:signal:reset:1.0 reset RST";
    attribute X_INTERFACE_INFO of interrupt : signal
        is "xilinx.com:signal:interrupt:1.0 interrupt INTERRUPT";
    attribute X_INTERFACE_INFO of clkout : signal
        is "xilinx.com:signal:clock:1.0 clk_out CLK";

    attribute X_INTERFACE_PARAMETER : string;
    attribute X_INTERFACE_PARAMETER of reset : signal
        is "POLARITY ACTIVE_HIGH";
    attribute X_INTERFACE_PARAMETER of interrupt : signal
        is "SENSITIVITY LEVEL_HIGH";
    attribute X_INTERFACE_PARAMETER of clkout : signal
        is "FREQ_HZ 100000000";

    signal clk_enable : std_logic := '1';
begin

    clock_enable_dff : process (clkin) is
    begin
        if clkin'event and clkin = '1' then
            if reset = '1' then
                clk_enable <= '1';
            elsif sleep = '1' and interrupt = '0' and dbg_wakeup = '0' then
                clk_enable <= '0';
            elsif clk_enable = '0' then
                clk_enable <= '1';
            end if;
        end if;
    end process clock_enable_dff;

    clock_enable : component BUFGCE
        port map (
            O  => clkout,
            CE => clk_enable,
            I  => clkin
        );

end Behavioral;
```

Hardware Controlled

When the `Pause` input signal is set to one and MicroBlaze V has completed all external accesses, the pipeline is halted and the `Pause_Ack` output signal is set. This indicates to external hardware that it is safe to perform actions such as stopping the clock, resetting the processor or other IP cores. To continue from pause, the input signal `Pause` must be cleared to zero. In this case MicroBlaze V continues instruction execution where it was previously paused.

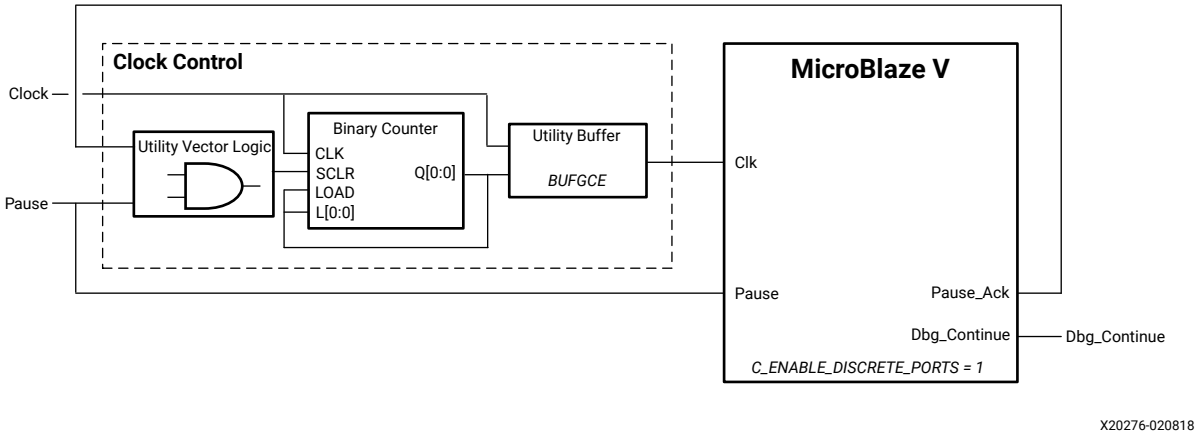
The `Dbg_Continue` output signal from MicroBlaze V indicates that the debugger requests the processor to continue from pause. External hardware should handle this signal and clear pause after performing any other necessary hardware actions such as starting the clock.

After external hardware has set or cleared `Pause`, it is recommended to wait until `Pause_Ack` is set or cleared before `Pause` is changed again, to avoid any issues due to incorrectly detected pause acknowledge.

All signals used for hardware control (`Pause`, `Pause_Ack`, and `Dbg_Continue`) are synchronous to the MicroBlaze V clock.

The block diagram in the following figure illustrates how to use the pause functionality to halt the processor and how to implement clock control. In this example, `Pause` is an external hardware signal that pauses processor execution and stops the clock. When `Pause` is cleared to zero, the clock is enabled and execution resumes. This example assumes that the external logic monitors `Dbg_Continue`, and clears `Pause` to allow debugging.

Figure 48: **Processor Halt and Clock Control Using the Pause Functionality**



AXI4 and ACE Interface Description

Memory Mapped Interfaces

Peripheral Interfaces

The MicroBlaze V AXI4 memory mapped peripheral interfaces are implemented as 32-bit masters. Each of these interfaces only have a single outstanding transaction at any time, and all transactions are completed in order.

- The instruction peripheral interface (M_AXI_IP) only performs single word read accesses, and is always set to use the AXI4-Lite subset.
- The data peripheral interface (M_AXI_DP) performs single word accesses, and is set to use the AXI4-Lite subset as default, but is set to use AXI4 when enabling exclusive access for `lr` and `sc` instructions. Halfword and byte writes are performed by setting the appropriate byte strobes. Each write transaction waits for `M_AXI_DP_BVALID` before the store instruction is completed.

The instruction peripheral interface (M_AXI_IP) address width is set according to the value of the parameter `C_ADDR_SIZE`.

The data peripheral interface (M_AXI_DP) address width is set according to the value of the parameter `C_ADDR_SIZE`.

Cache Interfaces

The AXI4 cache interfaces are implemented either as 32-bit, 128-bit, 256-bit, or 512-bit masters, depending on cache line length and data width parameters, whereas the AXI Coherency Extension (ACE) interfaces are implemented as 32-bit masters.

- With a 32-bit master, the instruction cache interface (M_AXI_IC or M_ACE_IC) performs 4 word, 8 word, or 16 word burst read accesses, depending on cache line length. With 128-bit, 256-bit, or 512-bit masters, only single read accesses are performed.

  With a 32-bit master, this interface can have multiple outstanding transactions, issuing up to two transactions or up to five transactions when stream cache is enabled. The stream cache can request two cache lines in advance, which means that in some cases five outstanding transactions can occur. In this case the number of outstanding reads is set to 8, because this must be a power of two. With 128-bit, 256-bit, or 512-bit masters, the interface only has a single outstanding transaction.

  The cached memory range is always accessed using the AXI4 or ACE cache interface.

- With a 32-bit master, the data cache interface (M_AXI_DC or M_ACE_DC) performs single word accesses, as well as 4 word, 8 word, or 16 word burst accesses, depending on cache line length. Burst write accesses are only performed when using write-back cache with AXI4. With 128-bit, 256-bit, or 512-bit AXI4 masters, only single accesses are performed.

  This interface can have multiple outstanding transactions, either issuing up to two transactions when reading, or up to 32 transactions when writing. MicroBlaze V ensures that all outstanding writes are completed before a read is issued, because the processor must maintain an ordered memory model. However, AXI4 or ACE has separate read/write channels without any ordering. Using up to 32 outstanding write transactions improves performance, because it allows multiple writes to proceed without stalling the pipeline.

  Word, half word and byte writes are performed by setting the appropriate byte strobes.

  Exclusive accesses can be enabled for `lr` and `sc` instructions.

  The cached memory range is always accessed using the AXI4 or ACE cache interface.

Interface Parameters and Signals

The relationship between MicroBlaze V parameter settings and AXI4 interface behavior for tool-assigned parameters is summarized in the following table.

Table 49: **AXI Memory Mapped Interface Parameters**

Interface	Parameter	Description
M_AXI_DP	C_M_AXI_DP_PROTOCOL	AXI4-Lite: Default. AXI4: Used to allow exclusive access when C_M_AXI_DP_EXCLUSIVE_ACCESS is 1.
M_AXI_IC M_ACE_IC	C_M_AXI_IC_DATA_WIDTH	32: Default, single word accesses and burst accesses with C_ICACHE_LINE_LEN word busts used with AXI4 and ACE. 128: Used when C_ICACHE_DATA_WIDTH is set to 1 and C_ICACHE_LINE_LEN is set to 4 with AXI4. Only single accesses can occur. 256: Used when C_ICACHE_DATA_WIDTH is set to 1 and C_ICACHE_LINE_LEN is set to 8 with AXI4. Only single accesses can occur. 512: Used when C_ICACHE_DATA_WIDTH is set to 2, or when it is set to 1 and C_ICACHE_LINE_LEN is set to 16 with AXI4. Only single accesses can occur.
M_AXI_DC M_ACE_DC	C_M_AXI_DC_DATA_WIDTH	32: Default, single word accesses and burst accesses with C_DCACHE_LINE_LEN word busts used with AXI4 and ACE. Write bursts are only used with AXI4 when C_DCACHE_USE_WRITEBACK is set to 1. 128: Used when C_DCACHE_DATA_WIDTH is set to 1 and C_DCACHE_LINE_LEN is set to 4 with AXI4. Only single accesses can occur. 256: Used when C_DCACHE_DATA_WIDTH is set to 1 and C_DCACHE_LINE_LEN is set to 8 with AXI4. Only single accesses can occur. 512: Used when C_DCACHE_DATA_WIDTH is set to 2, or when it is set to 1 and C_DCACHE_LINE_LEN is set to 16 with AXI4. Only single accesses can occur.
M_AXI_IC M_ACE_IC	NUM_READ_OUTSTANDING	1: Default for 128-bit, 256-bit and 512-bit masters, a single outstanding read. 2: Default for 32-bit masters, 2 simultaneous outstanding reads. 8: Used for 32-bit masters when C_ICACHE_STREAMS is set to 1, allowing 8 simultaneous outstanding reads. Can be set to 1, 2, or 8.
M_AXI_DC M_ACE_DC	NUM_READ_OUTSTANDING	1: Default for 128-bit, 256-bit and 512-bit masters, a single outstanding read. 2: Default for 32-bit masters, two simultaneous outstanding reads. Can be set to 1 or 2.
M_AXI_DC M_ACE_DC	NUM_WRITE_OUTSTANDING	32: Default, 32 simultaneous outstanding writes. Can be set to 1, 2, 4, 8, 16, or 32.

Values for access permissions, memory types, quality of service and shareability domain are defined in the following table.

Table 50: **AXI Interface Signal Definitions**

Interface	Signal	Description
M_AXI_IP	C_M_AXI_IP_ARPROT	Access permission: • Unprivileged, secure instruction access (100) if input signal Non_Secure[1] = 0 • Unprivileged, non-secure instruction access (110) if input signal Non_Secure[1] = 1
M_AXI_DP	C_M_AXI_DP_ARCACHE C_M_AXI_DP_AWCACHE	Memory type, AXI4 protocol: Normal non-cacheable bufferable (0011)
	C_M_AXI_DP_ARPROT C_M_AXI_DP_AWPROT	Access permission, AXI4 and AXI4-Lite protocol: • Unprivileged, secure data access (000) if input signal Non_Secure[0] = 0 • Unprivileged, non-secure data access (010) if input signal Non_Secure[0] = 1
	C_M_AXI_DP_ARQOS C_M_AXI_DP_AWQOS	Quality of service, AXI4 protocol: • Priority 8 (1000)
M_AXI_IC	C_M_AXI_IC_ARCACHE	Memory type: • Write-back read and write-allocate (1111)
M_ACE_IC	C_M_AXI_IC_ARCACHE	Memory type, normal access: • Write-back read and write-allocate (1111) Memory type, DVM access: • Normal non-cacheable non-bufferable (0010)
	C_M_AXI_IC_ARDOMAIN	Shareability domain: • Inner shareable (01)
M_AXI_IC M_ACE_IC	C_M_AXI_IC_ARPROT	Access permission: • Unprivileged, secure instruction access (100) if input signal `Non_Secure[3]` = 0 • Unprivileged, non-secure instruction access (110) if input signal `Non_Secure[3]` = 1
	C_M_AXI_IC_ARQOS	Quality of service: • Priority 7 (0111)
M_AXI_DC	C_M_AXI_DC_ARCACHE	Memory type, normal access: • Write-back Read and Write-allocate (1111) Memory Type, exclusive access: • Normal non-cacheable non-bufferable (0010)
M_ACE_DC	C_M_AXI_DC_ARCACHE	Memory type, normal and exclusive access: • Write-back read and write-allocate (1111) Memory type, DVM access: • Normal non-cacheable non-bufferable (0010)
	C_M_AXI_DC_ARDOMAIN C_M_AXI_DC_AWDOMAIN	Shareability domain: • Inner shareable (01)

Table 50: **AXI Interface Signal Definitions** *(cont'd)*

Interface	Signal	Description
M_AXI_DC M_ACE_DC	C_M_AXI_DC_AWCACHE	Memory type, normal access: • Write-back read and write-allocate (1111) Memory type, exclusive access: • Normal non-cacheable non-bufferable (0010)
	C_M_AXI_DC_ARPROT C_M_AXI_DC_AWPROT	Access permission: • Unprivileged, secure data access (000) if input signal Non_Secure[2] = 0 • Unprivileged, non-secure data access (010) if input signal Non_Secure[2] = 1
	C_M_AXI_DC_ARQOS	Quality of service, read access: • Priority 12 ((1100)
	C_M_AXI_DC_AWQOS	Quality of service, write access: • Priority 8 (1000)

The instruction cache interface (M_AXI_IC) address width can range from 32–64 bits, depending on the value of the parameter `C_ADDR_SIZE`.

See the *AMBA AXI and ACE Protocol Specification* (Arm IHI 0022H) for details.

Stream Interfaces

The MicroBlaze V AXI4-Stream interfaces (M0_AXIS, M15_AXIS, S0_AXIS, and S15_AXIS) are implemented as 32-bit masters and slaves. See the *AMBA 4 AXI4-Stream Protocol Specification, Version 1.0* (Arm IHI 0051A) for further details.

Write Operation

A write to the stream interface is performed by MicroBlaze V using one of the PUT or PUTD custom instructions. A write operation transfers the register contents to an output interface. The transfer is completed in a single clock cycle for blocking mode writes (PUT and CPUT custom instructions) as long as the interface is not busy. If the interface is busy, the processor stalls until it becomes available. The non-blocking custom instructions (with prefix n), always complete in a single clock cycle even if the interface is busy. If the interface is busy, the write is inhibited and the C bit is set in the mstream CSR.

The control custom instructions (with prefix c) set the AXI4-Stream `TLAST` output, to 1, which is commonly used to indicate the boundary of a packet.

Read Operation

A read from the stream interface is performed by MicroBlaze V using one of the GET or GETD custom instructions. A read operation transfers the contents of an input interface to a general purpose register. The transfer is typically completed in two clock cycles for blocking mode reads as long as data is available. If data is not available, the processor stalls at this instruction until it becomes available. In the non-blocking mode (custom instructions with prefix *n*), the transfer is completed in one or two clock cycles irrespective of whether data is available. In cases where data is not available, the transfer of data does not take place and the C bit is set in the mstream CSR.

The data GET or GETD custom instructions (without prefix c) expect the AXI4-Stream `TLAST` input to be cleared to 0. Otherwise, the instructions set the FSL bit in the mstream CSR to 1. Conversely, the control GET instructions (with prefix c) expect the `TLAST` input to be set to 1; otherwise, the instructions set the FSL bit to 1. This functionality can be used to check for the boundary of a packet.

Local Memory Bus (LMB) Interface Description

The LMB is a synchronous bus used primarily to access on-chip block RAM. It uses a minimum number of control signals and a simple protocol to ensure that local block RAM are accessed in a single clock cycle. LMB signals and definitions are shown in the following table. All LMB signals are active-High.

LMB provides an optional frequency optimized protocol for the 8-stage pipeline, selected by setting the parameters `C_DLMB_PROTOCOL = 1` for the data interface and `C_ILMB_PROTOCOL = 1` for the instruction interface. When this protocol is used, all LMB input signals are delayed one clock cycle. This can improve the maximum frequency, particularly when ECC is enabled.

LMB Signal Interface

Table 51: **LMB Bus Signals**

Signal	Data Interface	Instruction Interface	Type	Description
Addr[0:N-1][1]	Data_Addr[0:N-1]	Instr_Addr[0:N-1]	O	Address bus
Byte_Enable[0:N-1][2]	Byte_Enable[0:N-1]	not used	O	Byte enables
Data_Write[0:N-1][3]	Data_Write[0:N-1]	not used	O	Write data bus
AS	D_AS	I_AS	O	Address strobe
Read_Strobe	Read_Strobe	IFetch	O	Read in progress

Table 51: **LMB Bus Signals** *(cont'd)*

Signal	Data Interface	Instruction Interface	Type	Description
Write_Strobe	Write_Strobe	not used	O	Write in progress
Data_Read[0:N-1][3]	Data_Read[0:N-1]	Instr[0:N-1]	I	Read data bus
Ready	DReady	IReady	I	Ready for next transfer
Wait	DWait	IWait	I	Wait until accepted transfer is ready
CE	DCE	ICE	I	Correctable error
UE	DUE	IUE	I	Uncorrectable error
Clk	Clk	Clk	I	Bus clock

Notes:

1. N = 32 - 64, set according to `C_ADDR_SIZE`.
2. N = 4, 8, set according to `C_LMB_DATA_SIZE` when using 64-bit MicroBlaze V.
3. N = 32, 64, set according to `C_LMB_DATA_SIZE` when using 64-bit MicroBlaze V.

Address

The address bus is an output from the core and indicates the memory address that is being accessed by the current transfer. It is valid only when AS is High. In multicycle accesses requiring more than one clock cycle to complete, `Addr[0:N-1]` is valid only in the first clock cycle of the transfer.

Byte Enables

The byte enable signals are outputs from the core and indicate which byte lanes of the data bus contain valid data. `Byte_Enable` is valid only when AS is High. In multicycle accesses requiring more than one clock cycle to complete, `Byte_Enable` is valid only in the first clock cycle of the transfer. Valid values for `Byte_Enable` are shown in the following table.

Table 52: **Valid Values for Byte Enable[0:3]**

Byte Enable	Data Byte Lanes Used			
	0:7	8:15	[16:23	24:31
0001				•
0010			•	
0100		•		
1000	•			
0011			•	•
1100	•	•		
1111	•	•	•	•

Table 53: **Valid Values for Byte Enable[0:7]**

Byte Enable	Data Byte Lanes Used							
	0:7	8:15	16:23	24:31	32:39	40:47	48:55	56:63
00000001								•
00000010							•	
00000100						•		
00001000					•			
00010000				•				
00100000			•					
01000000		•						
10000000	•							
00000011							•	•
00001100					•	•		
00110000			•	•				
11000000	•	•						
00001111					•	•	•	•
11110000	•	•	•	•				
11111111	•	•	•	•	•	•	•	•

Data Write

The write data bus is an output from the core and contains the data that is written to memory. It is valid only when AS is High. Only the byte lanes specified by `Byte_Enable` contain valid data.

Address Strobe

The address strobe is an output from the core and indicates the start of a transfer and qualifies the address bus and the byte enables. It is High only in the first clock cycle of the transfer, after which it goes Low and remains Low until the start of the next transfer.

Read Strobe

The read strobe is an output from the core and indicates that a read transfer is in progress. This signal goes High in the first clock cycle of the transfer, and can remain High until the clock cycle after `Ready` is sampled High. If a new read transfer is directly started in the next clock cycle, `Read_Strobe` remains High. The read strobe is never High at the same time as the write strobe.

Write Strobe

The write strobe is an output from the core and indicates that a write transfer is in progress. This signal goes High in the first clock cycle of the transfer, and can remain High until the clock cycle after `Ready` is sampled High. If a new write transfer is directly started in the next clock cycle, `Write_Strobe` remains High. The write strobe is never High at the same time as the read strobe.

Data Read

The read data bus is an input to the core and contains data read from memory. `Data_Read` is valid on the rising edge of the clock when `Ready` is High.

Ready

The `Ready` signal is an input to the core and indicates completion of the current transfer and that the next transfer can begin in this clock cycle. It is sampled on the rising edge of the clock. For reads, this signal indicates the `Data_Read` bus is valid, and for writes it indicates that the `Data_Write` bus has been written to local memory.

Wait

The `Wait` signal is an input to the core and indicates that the current transfer has been accepted, but not yet completed. It is sampled on the rising edge of the clock. The `Wait` signal is ignored when `Ready` is High.

CE

The `CE` signal is an input to the core and indicates that the current transfer had a correctable error. It is valid on the rising edge of the clock when `Ready` is High. For reads, this signal indicates that an error has been corrected on the `Data_Read` bus, and for byte and half word writes it indicates that the corresponding data word in local memory has been corrected before writing the new data.

UE

The `UE` signal is an input to the core and indicates that the current transfer had an uncorrectable error. It is valid on the rising edge of the clock when `Ready` is High. For reads, this signal indicates that the value of the `Data_Read` bus is erroneous, and for byte and half word writes it indicates that the corresponding data word in local memory was erroneous before writing the new data.

Clk

All operations on the LMB are synchronous to the MicroBlaze V clock.

LMB Transactions

The following diagrams provide examples of LMB bus operations.

Generic Write Operations

Figure 49: **LMB Generic Write Operation, 0 Wait States**

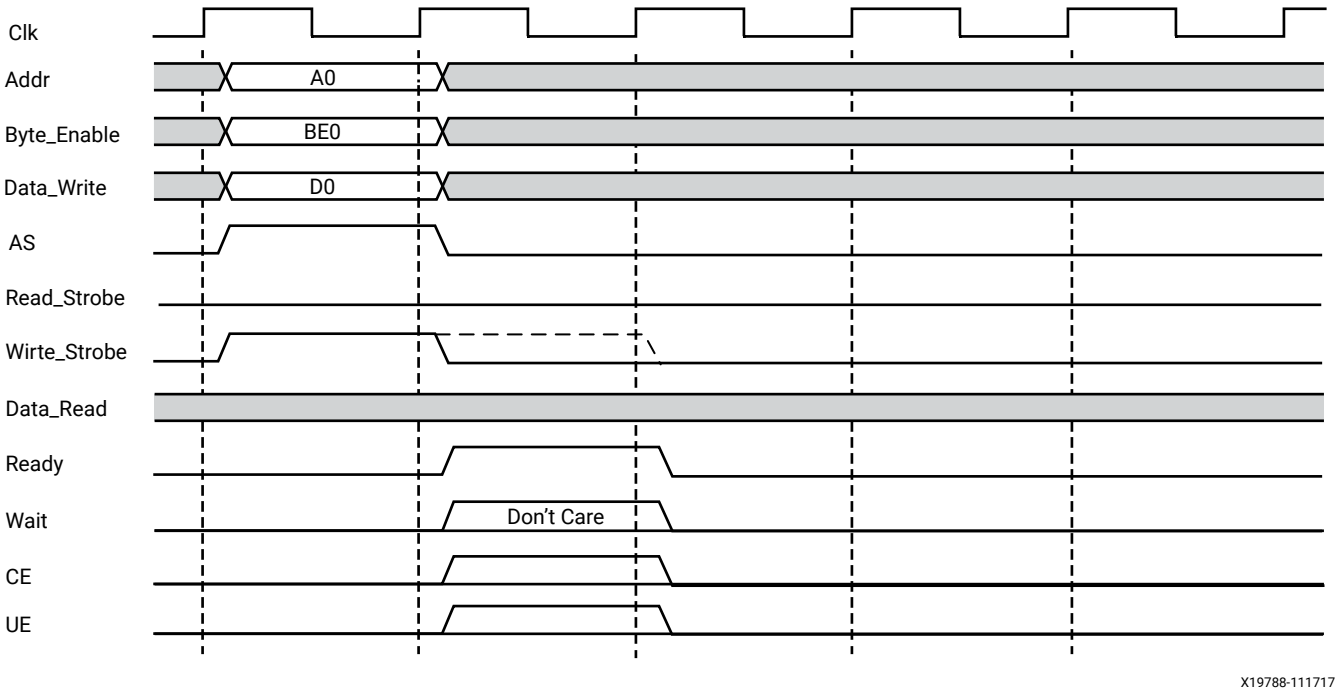
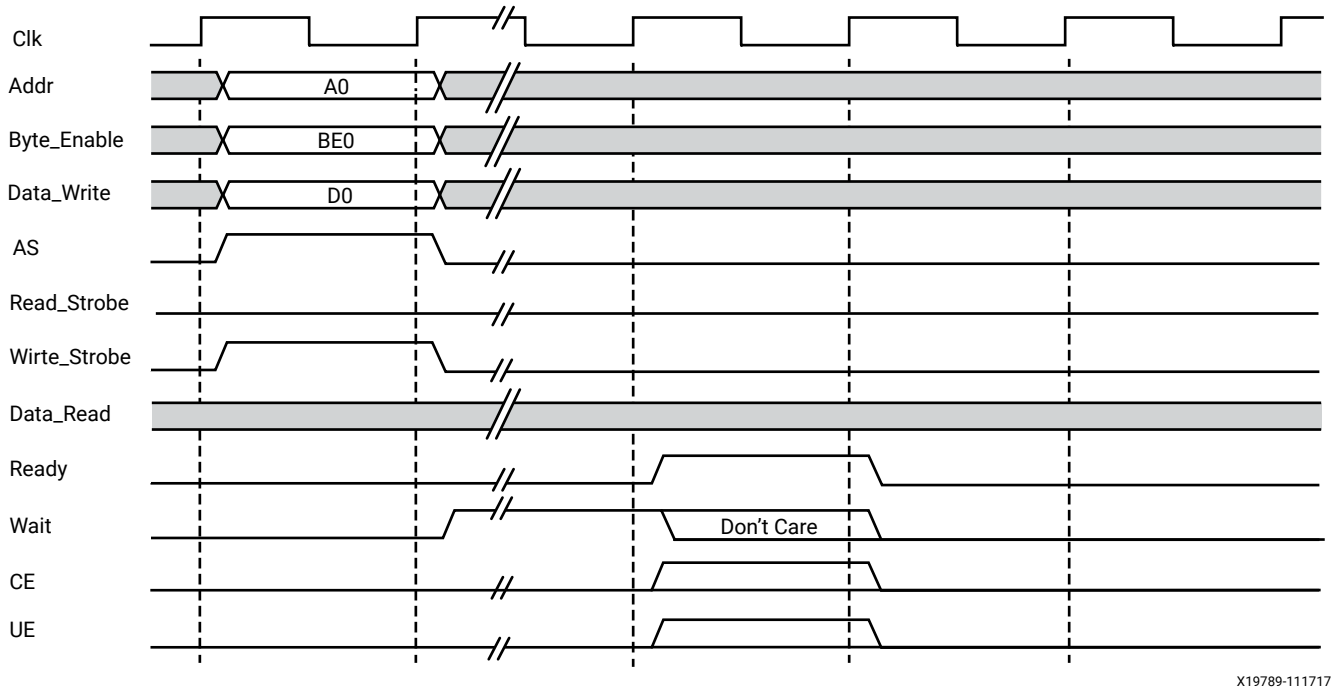


Figure 50: **LMB Generic Write Operation, N Wait States**



Generic Read Operations

Figure 51: **LMB Generic Read Operation, 0 Wait States**

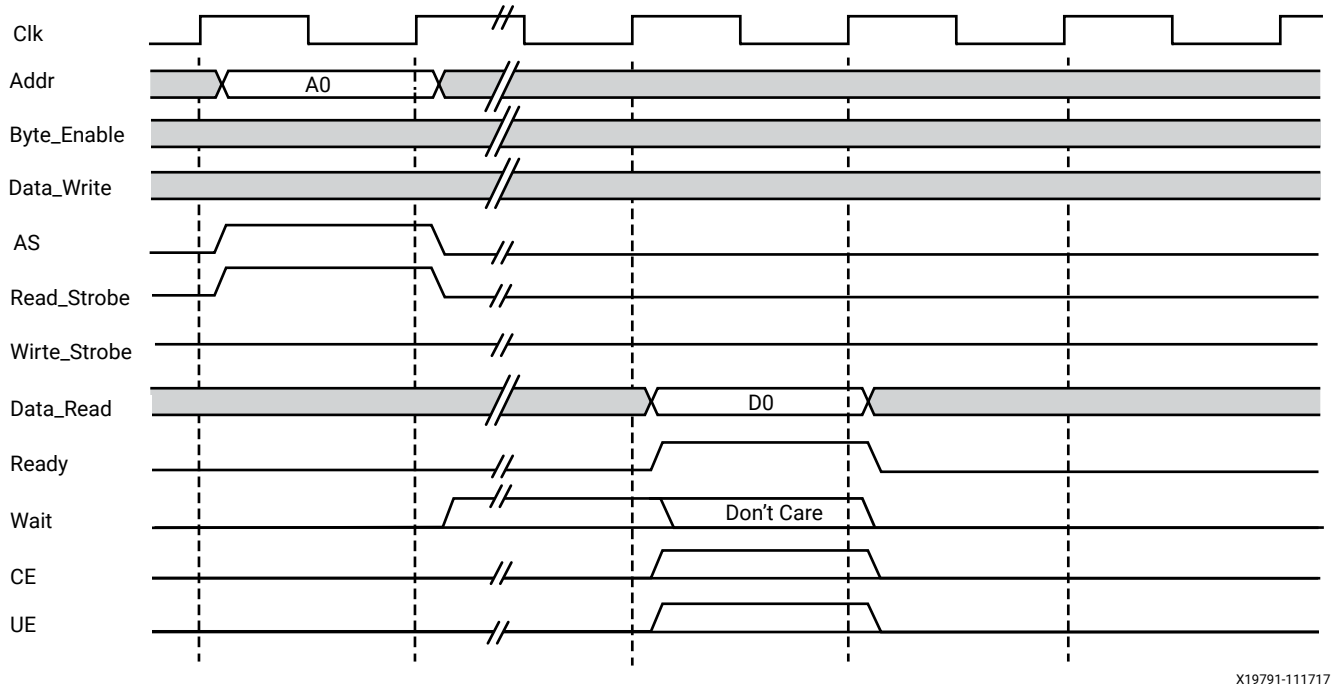
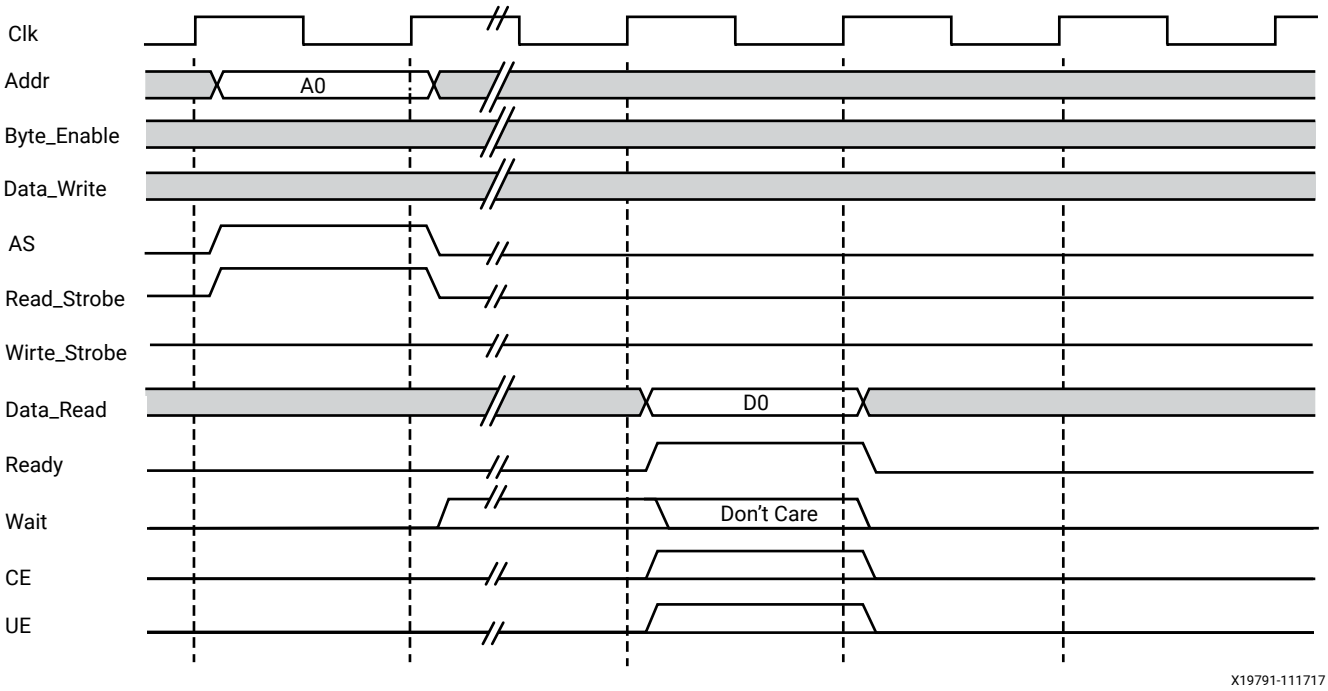
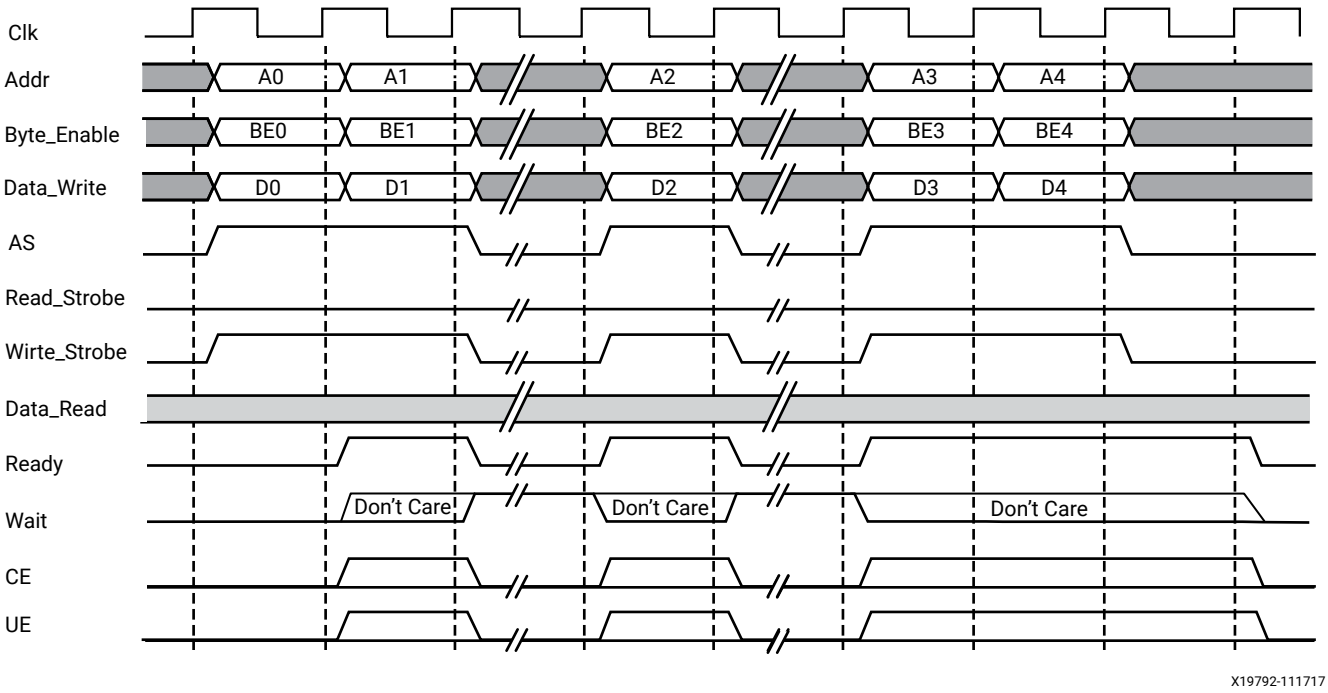


Figure 52: **LMB Generic Read Operation, N Wait States**



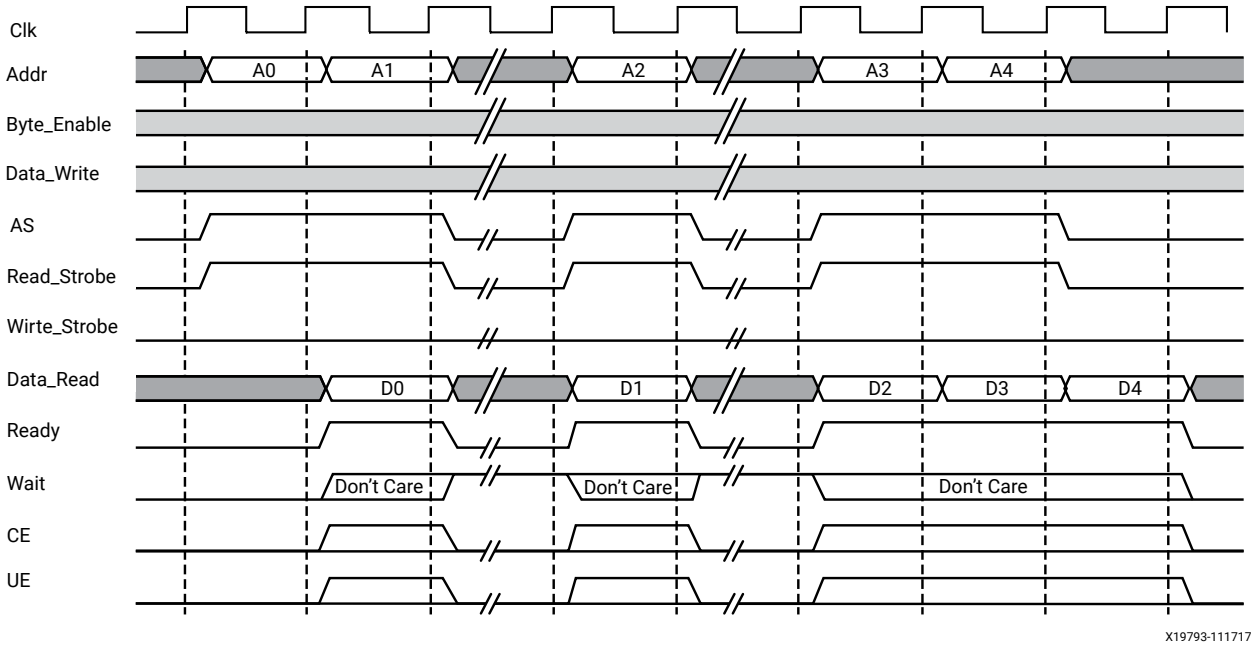
Back-to-Back Write Operation

Figure 53: **LMB Back-to-Back Write Operation**



Back-to-Back Read Operation

Figure 54: **LMB Back-to-Back Read Operation**



Back-to-Back Mixed Write/Read Operation

Figure 55: **Back-to-Back Mixed Write/Read Operation, 0 Wait States**

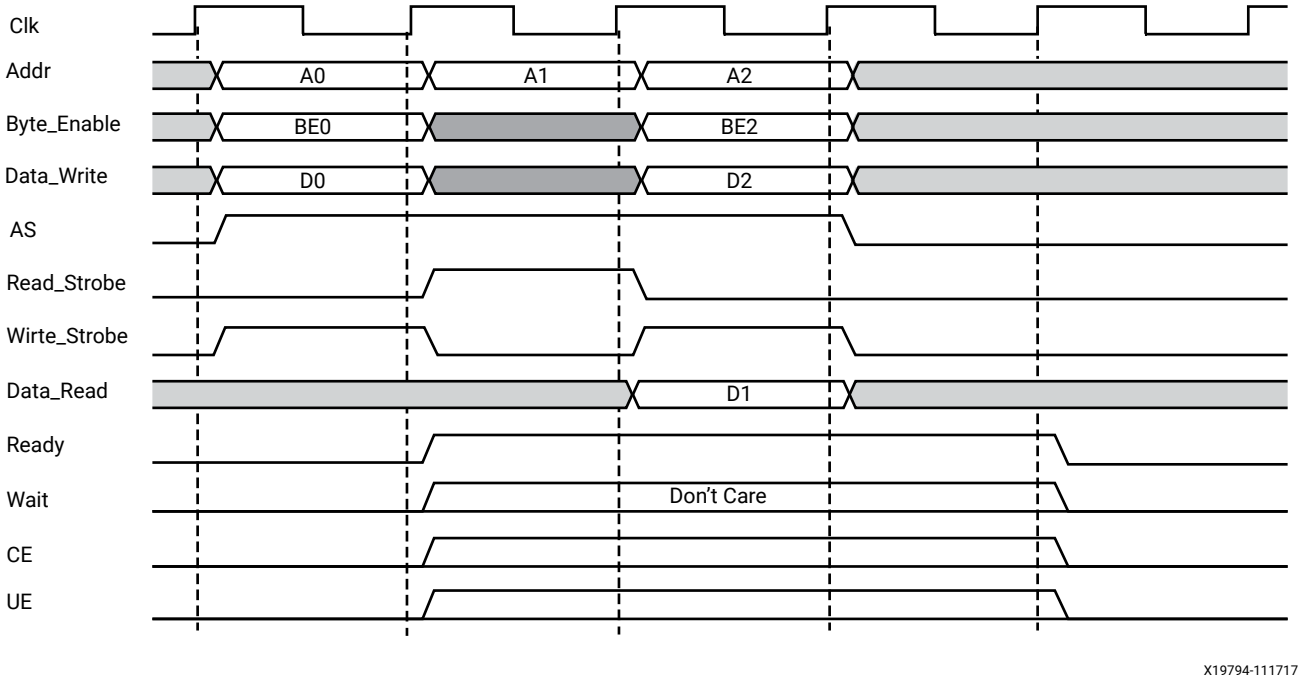
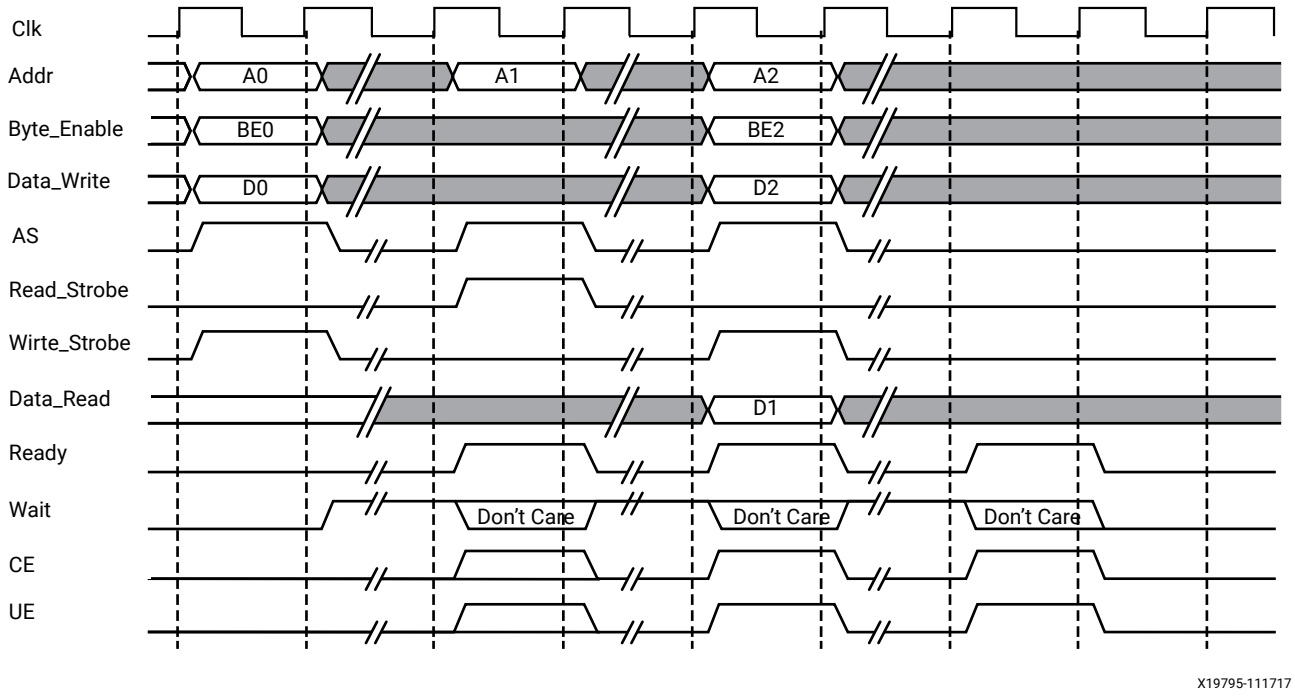


Figure 56: **Back-to-Back Mixed Write/Read Operation, N Wait States**



Read and Write Data Steering

The MicroBlaze V data-side bus interface performs the read steering and write steering required to support the following transfers:

- Byte, half word, and word transfers to word devices
- Byte and half word transfers to half word devices
- Byte transfers to byte devices

MicroBlaze V does not support transfers that are larger than the addressed device. These types of transfers require dynamic bus sizing and conversion cycles that are not supported by the bus interface. Data steering for read and write cycles are shown in the following tables.

Table 54: **Read Data Steering (Load to Register xD)**

Address	Byte Enable	Transfer Size	Register xD Data			
			xD[31:24]	xD[23:16]	xD[15:8]	xD[7:0]
11	1000	byte				Byte0
10	0100	byte				Byte1
01	0010	byte				Byte2
00	0001	byte				Byte3
10	1100	halfword			Byte0	Byte1

Table 54: **Read Data Steering (Load to Register xD)** *(cont'd)*

Address	Byte Enable	Transfer Size	Register xD Data			
			xD[31:24]	xD[23:16]	xD[15:8]	xD[7:0]
00	0011	halfword			Byte2	Byte3
00	1111	word	Byte0	Byte1	Byte2	Byte3

Table 55: **Write Data Steering (Store from Register xD)**

Address	Byte Enable	Transfer Size	Write Data Bus Bytes			
			Byte 3	Byte 2	Byte 1	Byte 0
11	1000	byte	xD[7:0]			
10	0100	byte		xD[7:0]		
01	0010	byte			xD[7:0]	
00	0001	byte				xD[7:0]
10	1100	halfword	xD[15:8]	xD[7:0]		
00	0011	halfword			xD[15:8]	xD[7:0]
00	1111	word	xD[31:24]	xD[23:16]	xD[15:8]	xD[7:0]

***Note*:** Other masters could have more restrictive requirements for byte lane placement than those allowed by MicroBlaze V. Slave devices are typically attached “left-justified” with byte devices attached to the most significant byte lane, and half word devices attached to the most significant half word lane. The MicroBlaze V steering logic fully supports this attachment method.

When using 64-bit data on DLMB or M_AXI_DP with 64-bit MicroBlaze V, the following transfers are also supported:

- byte, halfword, word, and long transfers to long devices

Data steering with 64-bit data for read cycles and 64-bit data steering for write cycles are shown in the following tables.

Table 56: **Read Data Steering (Load to Register xD)**

Address [LSB-2:LSB]	Byte_Enable [0:7]	Transfer Size	Register xD Data							
			0:7	8:15	16:23	24:31	32:39	40:47	48:55	56:63
111	10000000	byte								Byte0
110	01000000	byte								Byte1
101	00100000	byte								Byte2
100	00010000	byte								Byte3
011	00001000	byte								Byte4
010	00000100	byte								Byte5
001	00000010	byte								Byte6
000	00000001	byte								Byte7

Table 56: **Read Data Steering (Load to Register xD)** *(cont'd)*

Address [LSB-2:LSB]	Byte_Enable [0:7]	Transfer Size	Register xD Data							
			0:7	8:15	16:23	24:31	32:39	40:47	48:55	56:63
110	11000000	halfword							Byte0	Byte1
100	00110000	halfword							Byte2	Byte3
010	00001100	halfword							Byte4	Byte5
000	00000011	halfword							Byte6	Byte7
100	11110000	word					Byte0	Byte1	Byte2	Byte3
000	00001111	word					Byte4	Byte5	Byte6	Byte7
000	11111111	long	Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7

Table 57: **Write Data Steering (Store from Register xD)**

Address [LSB-2:LSB]	Byte_Enable [0:7]	Transfer Size	Write Data Bus Bytes from xD							
			Byte7	Byte6	Byte5	Byte4	Byte3	Byte2	Byte1	Byte0
111	10000000	byte	56:63							
110	01000000	byte		56:63						
101	00100000	byte			56:63					
100	00010000	byte				56:63				
011	00001000	byte					56:63			
010	00000100	byte						56:63		
001	00000010	byte							56:63	
000	00000000	byte								56:63
110	11000000	halfword	48:55	56:63						
100	00110000	halfword			48:55	56:63				
010	00001100	halfword					48:55	56:63		
000	00000011	halfword							48:55	56:63
100	11110000	word	32:39	40:47	48:55	56:63				
000	00001111	word					32:39	40:47	48:55	56:63
000	11111111	long	0:7	8:15	16:23	24:31	32:39	40:47	48:55	56:63

Lockstep Interface Description

The lockstep interface on MicroBlaze V is designed to connect a master and one or more slave MicroBlaze V instances. The lockstep signals are listed in the following table.

Table 58: **MicroBlaze V Lockstep Signals**

Signal Name	Description	VHDL Type	Direction
`Lockstep_Master_Out`	Output with signals going from master to slave MicroBlaze V. Not connected on slaves.	std_logic	output
`Lockstep_Slave_In`	Input with signals coming from master to slave MicroBlaze V. Not connected on master.	std_logic	input
`Lockstep_Out`	Output with all comparison signals from both master and slaves.	std_logic	output

The comparison signals provided by `Lockstep_Out` are listed in the following table.

Table 59: **MicroBlaze V Lockstep Comparison Signals**

Signal Name	Bus Index Range	VHDL Type
Halted	0	std_logic
IFetch	2	std_logic
I_AS	3	std_logic
Instr Addr	4 to 67	std_logic_vector
Data Addr	68 to 131	std_logic_vector
Data Write	132 to 163	std_logic_vector
D_AS	196	std_logic
Read Strobe	197	std_logic
Write Strobe	198	std_logic
Byte Enable	199 to 202	std_logic_vector
M_AXI_IP_AWID	207	std_logic
M_AXI_IP_AWADDR	208 to 271	std_logic_vector
M_AXI_IP_AWLEN	272 to 279	std_logic_vector
M_AXI_IP_AWSIZE	280 to 282	std_logic_vector
M_AXI_IP_AWBURST	283 to 284	std_logic_vector
M_AXI_IP_AWLOCK	285	std_logic
M_AXI_IP_AWCACHE	286 to 289	std_logic_vector
M_AXI_IP_AWPROT	290 to 292	std_logic_vector
M_AXI_IP_AWQOS	293 to 296	std_logic_vector
M_AXI_IP_AWVALID	297	std_logic
M_AXI_IP_WDATA	298 to 329	std_logic_vector
M_AXI_IP_WSTRB	362 to 365	std_logic_vector
M_AXI_IP_WLAST	370	std_logic
M_AXI_IP_WVALID	371	std_logic
M_AXI_IP_BREADY	372	std_logic
M_AXI_IP_ARID	373	std_logic
M_AXI_IP_ARADDR	374 to 437	std_logic_vector
M_AXI_IP_ARLEN	438 to 445	std_logic_vector

Table 59: **MicroBlaze V Lockstep Comparison Signals** *(cont'd)*

Signal Name	Bus Index Range	VHDL Type
M_AXI_IP_ARSIZE	446 to 448	std_logic_vector
M_AXI_IP_ARBURST	449 to 450	std_logic_vector
M_AXI_IP_ARLOCK	451	std_logic
M_AXI_IP_ARCACHE	452 to 455	std_logic_vector
M_AXI_IP_ARPROT	456 to 458	std_logic_vector
M_AXI_IP_ARQOS	459 to 462	std_logic_vector
M_AXI_IP_ARVALID	463	std_logic
M_AXI_IP_RREADY	464	std_logic
M_AXI_DP_AWID	465	std_logic
M_AXI_DP_AWADDR	466 to 529	std_logic_vector
M_AXI_DP_AWLEN	530 to 537	std_logic_vector
M_AXI_DP_AWSIZE	538 to 540	std_logic_vector
M_AXI_DP_AWBURST	541 to 542	std_logic_vector
M_AXI_DP_AWLOCK	543	std_logic
M_AXI_DP_AWCACHE	544 to 547	std_logic_vector
M_AXI_DP_AWPROT	548 to 550	std_logic_vector
M_AXI_DP_AWQOS	551 to 554	std_logic_vector
M_AXI_DP_AWVALID	555	std_logic
M_AXI_DP_WDATA	556 to 587	std_logic_vector
M_AXI_DP_WSTRB	620 to 623	std_logic_vector
M_AXI_DP_WLAST	628	std_logic
M_AXI_DP_WVALID	629	std_logic
M_AXI_DP_BREADY	630	std_logic
M_AXI_DP_ARID	631	std_logic
M_AXI_DP_ARADDR	632 to 695	std_logic_vector
M_AXI_DP_ARLEN	696 to 703	std_logic_vector
M_AXI_DP_ARSIZE	704 to 706	std_logic_vector
M_AXI_DP_ARBURST	707 to 708	std_logic_vector
M_AXI_DP_ARLOCK	709	std_logic
M_AXI_DP_ARCACHE	710 to 713	std_logic_vector
M_AXI_DP_ARPROT	714 to 716	std_logic_vector
M_AXI_DP_ARQOS	717 to 720	std_logic_vector
M_AXI_DP_ARVALID	721	std_logic
M_AXI_DP_RREADY	722	std_logic
Mn_AXIS_TLAST	723 + n * 35	std_logic
Mn_AXIS_TDATA	758 + n * 35 to 789 + n * 35	std_logic
Mn_AXIS_TVALID	790 + n * 35	std_logic
Sn_AXIS_TREADY	791 + n * 35	std_logic

Table 59: **MicroBlaze V Lockstep Comparison Signals** *(cont'd)*

Signal Name	Bus Index Range	VHDL Type
M_AXI_IC_AWID	1283	std_logic
M_AXI_IC_AWADDR	1284 to 1347	std_logic_vector
M_AXI_IC_AWLEN	1348 to 1355	std_logic_vector
M_AXI_IC_AWSIZE	1356 to 1358	std_logic_vector
M_AXI_IC_AWBURST	1359 to 1360	std_logic_vector
M_AXI_IC_AWLOCK	1361	std_logic
M_AXI_IC_AWCACHE	1362 to 1365	std_logic_vector
M_AXI_IC_AWPROT	1366 to 1368	std_logic_vector
M_AXI_IC_AWQOS	1369 to 1372	std_logic_vector
M_AXI_IC_AWVALID	1373	std_logic
M_AXI_IC_AWUSER	1374 to 1378	std_logic_vector
M_AXI_IC_AWDOMAIN[1]	1379 to 1380	std_logic_vector
M_AXI_IC_AWSNOOP[1]	1381 to 1383	std_logic_vector
M_AXI_IC_AWBAR	1384 to 1385	std_logic_vector
M_AXI_IC_WDATA	1386 to 1897	std_logic_vector
M_AXI_IC_WSTRB	1898 to 1961	std_logic_vector
M_AXI_IC_WLAST	1962	std_logic
M_AXI_IC_WVALID	1963	std_logic
M_AXI_IC_WUSER	1964	std_logic
M_AXI_IC_BREADY	1965	std_logic
M_AXI_IC_WACK[1]	1966	std_logic
M_AXI_IC_ARID	1967	std_logic_vector
M_AXI_IC_ARADDR	1968 to 2031	std_logic_vector
M_AXI_IC_ARLEN	2032 to 2039	std_logic_vector
M_AXI_IC_ARSIZE	2040 to 2042	std_logic_vector
M_AXI_IC_ARBURST	2043 to 2044	std_logic_vector
M_AXI_IC_ARLOCK	2045	std_logic
M_AXI_IC_ARCACHE	2046 to 2049	std_logic_vector
M_AXI_IC_ARPROT	2050 to 2052	std_logic_vector
M_AXI_IC_ARQOS	2053 to 2056	std_logic_vector
M_AXI_IC_ARVALID	2057	std_logic
M_AXI_IC_ARUSER	2058 to 2062	std_logic_vector
M_AXI_IC_ARDOMAIN[1]	2063 to 2064	std_logic_vector
M_AXI_IC_ARSNOOP[1]	2065 to 2068	std_logic_vector
M_AXI_IC_ARBAR[1]	2069 to 2070	std_logic_vector
M_AXI_IC_RREADY	2071	std_logic
M_AXI_IC_RACK[1]	2072	std_logic
M_AXI_IC_ACREADY[1]	2073	std_logic
M_AXI_IC_CRVALID[1]	2074	std_logic

Table 59: **MicroBlaze V Lockstep Comparison Signals** *(cont'd)*

Signal Name	Bus Index Range	VHDL Type
M_AXI_IC_CRRESP[1]	2075 to 2079	std_logic_vector
M_AXI_IC_CDVALID[1]	2080	std_logic
M_AXI_IC_CDLAST[1]	2081	std_logic
M_AXI_DC_AWID	2082	std_logic
M_AXI_DC_AWADDR	2083 to 2146	std_logic_vector
M_AXI_DC_AWLEN	2147 to 2154	std_logic_vector
M_AXI_DC_AWSIZE	2155 to 2157	std_logic_vector
M_AXI_DC_AWBURST	2158 to 2159	std_logic_vector
M_AXI_DC_AWLOCK	2160	std_logic
M_AXI_DC_AWCACHE	2161 to 2164	std_logic_vector
M_AXI_DC_AWPROT	2165 to 2167	std_logic_vector
M_AXI_DC_AWQOS	2168 to 2171	std_logic_vector
M_AXI_DC_AWVALID	2172	std_logic
M_AXI_DC_AWUSER	2172 to 2176	std_logic_vector
M_AXI_DC_AWDOMAIN[1]	2177 to 2178	std_logic_vector
M_AXI_DC_AWSNOOP[1]	2179 to 2182	std_logic_vector
M_AXI_DC_AWBAR[1]	2183 to 2184	std_logic_vector
M_AXI_DC_WDATA	2185 to 2696	std_logic_vector
M_AXI_DC_WSTRB	2697 to 2760	std_logic_vector
M_AXI_DC_WLAST	2761	std_logic
M_AXI_DC_WVALID	2762	std_logic
M_AXI_DC_WUSER	2863	std_logic
M_AXI_DC_BREADY	2764	std_logic
M_AXI_DC_WACK[1]	2765	std_logic
M_AXI_DC_ARID	2766	std_logic
M_AXI_DC_ARADDR	2767 to 2830	std_logic_vector
M_AXI_DC_ARLEN	2831 to 2838	std_logic_vector
M_AXI_DC_ARSIZE	2839 to 2841	std_logic_vector
M_AXI_DC_ARBURST	2842 to 2843	std_logic_vector
M_AXI_DC_ARLOCK	2844	std_logic
M_AXI_DC_ARCACHE	2845 to 2848	std_logic_vector
M_AXI_DC_ARPROT	2849 to 2851	std_logic_vector
M_AXI_DC_ARQOS	2852 to 2855	std_logic_vector
M_AXI_DC_ARVALID	2856	std_logic
M_AXI_DC_ARUSER	2857 to 2861	std_logic_vector
M_AXI_DC_ARDOMAIN[1]	2862 to 2863	std_logic_vector
M_AXI_DC_ARSNOOP[1]	2864 to 2867	std_logic_vector
M_AXI_DC_ARBAR[1]	2868 to 2869	std_logic_vector
M_AXI_DC_RREADY	2870	std_logic

Table 59: **MicroBlaze V Lockstep Comparison Signals** *(cont'd)*

Signal Name	Bus Index Range	VHDL Type
M_AXI_DC_RACK[1]	2871	std_logic
M_AXI_DC_ACREADY[1]	2872	std_logic
M_AXI_DC_CRVALID[1]	2873	std_logic
M_AXI_DC_CRRESP[1]	2874 to 2878	std_logic_vector
M_AXI_DC_CDVALID[1]	2879	std_logic
M_AXI_DC_CDLAST[1]	2880	std_logic
Trace_Instruction	2881 to 2912	std_logic_vector
Trace_Valid_Instr	2913	std_logic
Trace_PC	2914 to 2945	std_logic_vector
Trace_Reg_Write	2978	std_logic
Trace_Reg_Addr	2979 to 2983	std_logic_vector
Trace_MSR_Reg	2984 to 2998	std_logic_vector
Trace_PID_Reg	2999 to 3006	std_logic_vector
Trace_New_Reg_Value	3007 to 3038	std_logic_vector
Trace_Exception_Taken	3071	std_logic
Trace_Exception_Kind	3072 to 3076	std_logic_vector
Trace_Jump_Taken	3077	std_logic
Trace_Delay_Slot	3078	std_logic
Trace_Data_Address	3079 to 3142	std_logic_vector
Trace_Data_Write_Value	3143 to 3174	std_logic_vector
Trace_Data_Byte_Enable	3207 to 3210	std_logic_vector
Trace_Data_Access	3215	std_logic
Trace_Data_Read	3216	std_logic
Trace_Data_Write	3217	std_logic
Trace_DCache_Req	3218	std_logic
Trace_DCache_Hit	3219	std_logic
Trace_DCache_Rdy	3220	std_logic
Trace_DCache_Read	3221	std_logic
Trace_ICache_Req	3222	std_logic
Trace_ICache_Hit	3223	std_logic
Trace_ICache_Rdy	3224	std_logic
Trace_OF_PipeRun	3225	std_logic
Trace_EX_PipeRun	3226	std_logic
Trace_MEM_PipeRun	3227	std_logic
Trace_Halted	3228	std_logic
Trace_Jump_Hit	3229	std_logic
Reserved	3230 to 4095	

Notes:

1. This signal is only used when `C_INTERCONNECT` = 3 (ACE).

Debug Interface Description

The debug interface on MicroBlaze V is designed to work with the MicroBlaze Debug Module (MDM) V IP core. The MDM V is controlled by the Xilinx System Debugger (XSDB) through the JTAG port of the FPGA. The MDM V can control multiple MicroBlaze V processors at the same time. The debug signals are grouped in the DEBUG bus.

The debug interface can be grouped in the DEBUG bus, using either JTAG serial signals (by setting `C_DEBUG_INTERFACE` = 0) or the AXI4-Lite compatible parallel signals (by setting `C_DEBUG_INTERFACE` = 1). The MDM V configuration must also be set accordingly.

It is also possible to use only AXI4-Lite parallel signals (`C_DEBUG_INTERFACE` = 2) grouped in an AXI4 bus, in case the MDM V is not used. However, this configuration is not supported by the tools.

The following table lists the debug signals on MicroBlaze V.

Table 60: **MicroBlaze V Debug Signals**

Signal Name	Description	VHDL Type	Kind
Dbg_Clk	JTAG clock from MDM V	std_logic	Serial in
Dbg_TDI	JTAG TDI from MDM V	std_logic	Serial in
Dbg_TDO	JTAG TDO to MDM V	std_logic	Serial out
Dbg_Reg_En	Debug register enable from MDM V	std_logic_vector	Serial in
Dbg_Shift	JTAG BSCAN shift signal from MDM V	std_logic	Serial in
Dbg_Capture	JTAG BSCAN capture signal from MDM V	std_logic	Serial in
Dbg_Update	JTAG BSCAN update signal from MDM V	std_logic	Serial in
Debug_Rst	Reset signal from MDM V, active-High. Should be held for at least one clk clock cycle.	std_logic	Input
Dbg_Trig_In	Cross trigger event input to MDM V	std_logic_vector	Output
Dbg_Trig_Ack_In	Cross trigger event input acknowledge from MDM V	std_logic_vector	input
Dbg_Trig_Out	Cross trigger action output from MDM V	std_logic_vector	Input
Dbg_Trig_Ack_Out	Cross trigger action output acknowledge to MDM V	std_logic_vector	Output
Dbg_ARADDR	Read address from MDM V	std_logic_vector	Parallel in
Dbg_ARREADY	Read address ready to MDM V	std_logic	Parallel out
Dbg_ARVALID	Read address valid from MDM V	std_logic	Parallel in
Dbg_AWADDR	Write address from MDM V	std_logic_vector	Parallel in
Dbg_AWREADY	Write address ready to MDM V	std_logic	Parallel out
Dbg_AWVALID	Write address valid from MDM V	std_logic	Parallel in
Dbg_BREADY	Write response ready to MDM V	std_logic	Parallel out
Dbg_BRESP	Write response to MDM V	std_logic_vector	Parallel out
Dbg_BVALID	Write response valid from MDM V	std_logic	Parallel in
Dbg_RDATA	Read data to MDM V	std_logic_vector	Parallel out

Table 60: **MicroBlaze V Debug Signals** *(cont'd)*

Signal Name	Description	VHDL Type	Kind
Dbg_RREADY	Read data ready to MDM V	std_logic	Parallel out
Dbg_RRESP	Read data response to MDM V	std_logic_vector	Parallel out
Dbg_RVALID	Read data valid from MDM V	std_logic	Parallel in
Dbg_WDATA	Write data from MDM V	std_logic_vector	Parallel in
Dbg_WREADY	Write data ready to MDM V	std_logic	Parallel out
Dbg_WVALID	Write data valid from MDM V	std_logic	Parallel in
DEBUG_ACLK	Debug clock, must be same as Clk	std_logic	Parallel in
DEBUG_ARESET	Debug reset, must be same as Reset	std_logic	Parallel in

Trace Interface Description

The MicroBlaze V processor core exports a number of internal signals for trace purposes. This signal interface is not standardized and new revisions of the processor might not be backward compatible regarding signal selection or functionality. It is recommended that you not design custom logic for these signals, but rather use them using analysis IP provided by AMD Adaptive Computing. The trace signals are grouped in the `TRACE` bus. The current set of trace signals is listed in the following table.

Table 61: **MicroBlaze V Trace Signals**

Signal Name	Description	VHDL Type
Trace_Valid_Instr	Valid instruction on trace port	std_logic
Trace_Instruction[1]	Instruction code	std_logic_vector (0 to 31)
Trace_PC[1]	Program counter, where N = 32 - 64, determined by parameter C_ADDR_SIZE for 64-bit MicroBlaze V, and 32 otherwise	std_logic_vector (0 to N-1)
Trace_Reg_Write[1]	Instruction writes to the register file	std_logic
Trace_FP_Reg_Write[1, 6]	Instruction writes to the FPU register file	std_logic
Trace_Reg_Addr[1]	Destination register address	std_logic_vector (0 to 4)
Trace_PID_Reg[1]	Address Space Identifier (ASID)	std_logic_vector (0 to 7)
Trace_New_Reg_Value[1]	Destination register update value, where N = C_DATA_SIZE	std_logic_vector (0 to N-1)
Trace_Exception_Taken[1]	Instruction result in taken exception	std_logic
Trace_Exception_Kind[1, 2]	Exception type. The description for the exception type is documented below.	std_logic_vector (0 to 5)
Trace_Jump_Taken[1]	Jump instruction or branch instruction evaluated true, that is taken	std_logic
Trace_Jump_Hit[1, 5]	Branch target cache hit	std_logic
Trace_Jump_Mispredict[1, 5]	Branch target cache mispredict	std_logic
Trace_Data_Access[1]	Valid D-side memory access	std_logic

Table 61: **MicroBlaze V Trace Signals** *(cont'd)*

Signal Name	Description	VHDL Type
Trace_Data_Address[1]	Address for D-side memory access, where N = 32 - 64, determined by parameter C_ADDR_SIZE	std_logic_vector (0 to N-1)
Trace_Data_Write_Value[1]	Value for D-side memory write access, where N = C_DATA_SIZE	std_logic_vector (0 to N-1)
Trace_Data_Byte_Enable[1]	Byte enables for D-side memory access, where N = C_DATA_SIZE / 8	std_logic_vector (0 to N-1)
Trace_Data_Read[1]	D-side memory access is a read	std_logic
Trace_Data_Write[1]	D-side memory access is a write	std_logic
Trace_DCache_Req	Data memory address is within D-Cache range. Set when a memory access instruction is executed.	std_logic
Trace_DCache_Hit	Data memory address is present in D-Cache. Set simultaneously with Trace_DCache_Req when a cache hit occurs.	std_logic
Trace_DCache_Rdy	Data memory address is within D-Cache range and the access is completed. Only set following a request with Trace_DCache_Req = 1 and Trace_DCache_Hit = 0.	std_logic
Trace_DCache_Read	The D-Cache request is a read. Valid only when Trace_DCache_Req = 1.	std_logic
Trace_ICache_Req	Instruction memory address is within I-Cache range, and the cache is enabled in the Machine Status register. Set when an instruction is read into the instruction prefetch buffer.	std_logic
Trace_ICache_Hit	Instruction memory address is present in I-Cache. Set simultaneously with Trace_ICache_Req when a cache hit occurs.	std_logic
Trace_ICache_Rdy	• Instruction memory address is present in I-Cache. Set simultaneously with Trace_ICache_Req when a cache hit occurs in this case. • Instruction memory address is within I-Cache range and the access is completed. Set following a request with Trace_ICache_Req = 1 and Trace_ICache_Hit = 0 in this case.	std_logic
Trace_OF_PipeRun	Pipeline advance for Decode stage	std_logic
Trace_EX_PipeRun[3]	Pipeline advance for Execution stage	std_logic
Trace_MEM_PipeRun[3, 4]	Pipeline advance for Memory stage	std_logic
Trace_Privilege_Mode	Current privilege mode: • 00 = User mode • 01 = Supervisor mode • 11 = Machine mode	std_logic_vector(0 to 1)

Table 61: **MicroBlaze V Trace Signals** *(cont'd)*

Signal Name	Description	VHDL Type
Trace_Halted	Pipeline is halted by debug	std_logic

Notes:

1. Valid only when `Trace_Valid_Instr` = 1.
2. Valid only when `Trace_Exception_Taken` = 1.
3. Not used with the 3-stage pipeline.
4. Not used with the 4-stage pipeline. Represents the M3 pipe stage for the 8-stage pipeline.
5. Valid only with BTC enabled.
6. Valid only when FPU enabled.

The used Trace exception types are listed in the following table. All unused Trace exception types are either reserved or unimplemented.

Table 62: **Type of Trace Exception**

Trace_Exception_Kind	Description
000000	Instruction address misaligned
000001	Instruction access fault
000010	Illegal instruction
000011	Breakpoint
000100	Load address misaligned
000101	Load access fault
000110	Store/AMO address misaligned
000111	Store/AMO access fault
001000	Environment call from U-mode
001001	Environment call from S-mode
001011	Environment call from M-mode
001100	Instruction page fault
001101	Load page fault
001111	Store/AMO page fault
011000	AXI4-Stream exception
100000	Non-maskable interrupt
101011	External interrupt
110000	External Break

MicroBlaze V Core Configurability

The MicroBlaze V core has been developed to support a high degree of user configurability. This allows tailoring of the processor to meet specific cost/performance requirements.

Configuration is done using parameters that typically enable, size, or select certain processor features. For example, the instruction cache is enabled by setting the `C_USE_ICACHE` parameter. The size of the instruction cache, and the cacheable memory range, are all configurable using: `C_ICACHE_BYTE_SIZE`, `C_ICACHE_BASEADDR`, and `C_ICACHE_HIGHADDR` respectively.

Parameters valid for the latest version of MicroBlaze V are listed in the following table. The configurability is fully backward compatible.

Table 63: **Configuration Parameters**

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_FAMILY	Target family	See below	virtex7	Yes	String
C_DATA_SIZE	Data size: • 32 = rv32i • 64 = rv64i	32, 64	32		Integer
C_ADDR_SIZE	Address Size	32-64	32	NA	Integer
C_OPTIMIZATION	Select implementation optimization: • 0 = Performance • 1 = Area • 2 = Frequency • 3 = Throughput	0, 1, 2, 3	0		Integer
C_INTERCONNECT	Select interconnect: • 2 = AXI4 only • 3 = AXI4, ACE	2, 3	2		Integer
C_BASE_VECTORS[1]	Configurable base vectors	0x0 - 0xFFFFFFFF FFFFFFFF	0x0		std_logic_ vector
C_FAULT_TOLERANT	Implement fault tolerance	0, 1	0	Yes	Integer
C_ECC_USE_CE_EXCEPTION	Generate exception for correctable ECC error	0, 1	0		Integer
C_LOCKSTEP_SLAVE	Lockstep Slave	0, 1	0		Integer
C_AVOID_PRIMITIVES	Disallow FPGA primitives: • 0 = None • 1 = SRL • 2 = LUTRAM • 3 = Both	0, 1, 2, 3	0		Integer
C_ENABLE_DISCRETE_PORTS	Show discrete ports	0, 1	0		Integer
C_INSTANCE	Instance Name	Any instance name	microblaze _v	Yes	String
C_D_AXI	Data side AXI interface	0, 1	0		Integer
C_D_LMB	Data side LMB interface	0, 1	1		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_D_LMB_PROTOCOL	Data side LMB protocol	0, 1	0		Integer
C_I_AXI	Instruction side AXI interface	0, 1	0		Integer
C_I_LMB	Instruction side LMB interface	0, 1	1		Integer
C_I_LMB_PROTOCOL	Instruction side LMB protocol	0, 1	0		Integer
C_LMB_DATA_SIZE	LMB interface data size	32, 64	32		Integer
C_USE_BARREL	Barrel shifter implementation: • 1 = Performance • 2 = Area	1, 2	1		Integer
C_USE_MULDIV	Include hardware multiplier and divider: • 0 = None • 1 = Normal • 2 = Smart	0, 1, 2	0		Integer
C_USE_FPU[4]	Include hardware floating-point unit: • 0 = None • 1 = Single • 2 = Double	0, 1, 2	0		Integer
C_USE_ATOMIC	Include atomic instructions	0, 1	0		Integer
C_USE_COMPRESSION	Use compressed instructions	0, 1	0		Integer
C_USE_BITMAN_A	Use bit manipulation instructions (Zba extension)	0, 1	0		Integer
C_USE_BITMAN_B	Use bit manipulation instructions (Zbb extension)	0, 1	0		Integer
C_USE_BITMAN_C	Use bit manipulation instructions (Zbc extension)	0, 1	0		Integer
C_USE_BITMAN_S	Use bit manipulation instructions (Zbs extension)	0, 1	0		Integer
C_USE_COUNTERS	Enable base counters and timers	0, 1	1		Integer
C_MISALIGNED_EXCEPTIONS	Enable exception handling for misaligned exceptions	0, 1	1		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_ILL_INSTR_EXCEPTION	Enable exception handling for illegal instructions: • 0 = None • 1 = Basic • 2 = Complete	0, 1, 2	2		Integer
C_M_AXI_I_BUS_EXCEPTION	Enable exception handling for M_AXI_I bus error	0, 1	1		Integer
C_M_AXI_D_BUS_EXCEPTION	Enable exception handling for M_AXI_D bus error	0, 1	1		Integer
C_FSL_EXCEPTION	Enable exception handling for stream links	0,1	0		Integer
C_IMPRECISE_EXCEPTIONS	Allow imprecise exceptions	0, 1	0		Integer
C_DEBUG_ENABLED	MDM Debug interface:	0,1	1		Integer
C_NUMBER_OF_PC_BRK	Number of hardware breakpoints	0-8	1		Integer
C_NUMBER_OF_RD_ADDR_BRK	Number of read address triggers	0-4	0		Integer
C_NUMBER_OF_WR_ADDR_BRK	Number of write address triggers	0-4	0		Integer
C_DEBUG_EVENT_COUNTERS	Hardware performance monitor event counters	0 - 29	0		Integer
C_DEBUG_LATENCY_COUNTERS	Hardware performance monitor latency counters	0 - 8	0		Integer
C_DEBUG_COUNTER_WIDTH	Hardware performance monitor counter width	32, 48, 64	64		Integer
C_DEBUG_TRACE_SIZE	Trace Buffer size	0, 4096, 8192,16384, 32768,65536, 131072	0		Integer
C_DEBUG_PROFILE_SIZE	Profile Buffer size	0, 4096, 8192,16384, 32768,65536, 131072	0		Integer
C_DEBUG_EXTERNAL_TRACE	External Program Trace	0,1	0		
C_S_AXI	Slave AXI4-Lite interface	0,1	0		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_DEBUG_INTERFACE	Debug Interface: • 0 = Debug Serial • 1 = Debug Parallel • 2 = AXI4-Lite	0,1,2	0		Integer
C_ASYNC_INTERRUPT	Asynchronous Interrupt	0,1	0	Yes	Integer
C_ASYNC_WAKEUP	Asynchronous Wakeup	00,01,10,11	00	Yes	Integer
C_INTERRUPT_IS_EDGE	Level/edge interrupt	0, 1	0	Yes	Integer
C_EDGE_IS_POSITIVE	Negative/positive edge interrupt	0, 1	1	Yes	Integer
C_FSL_LINKS	Number of AXI4-Stream interfaces	0 -16	0		Integer
C_USE_EXTENDED_FSL_INSTR	Enable use of extended stream instructions	0, 1	0		Integer
C_ICACHE_BASEADDR	Instruction cache base address	0x0 - 0xFFFFFFFF FFFFFFFF	0x0		std_logic_ vector
C_ICACHE_HIGHADDR	Instruction cache high address	0x0 - 0xFFFFFFFF FFFFFFFF	0x3FFFFFFF		std_logic_ vector
C_USE_ICACHE	Instruction cache	0, 1	0		Integer
C_ICACHE_LINE_LEN	Instruction cache line length	4, 8, 16	4		Integer
C_ICACHE_FORCE_TAG_LUTRAM	Instruction cache tag always implemented with distributed RAM	0, 1	0		Integer
C_ICACHE_STREAMS	Instruction cache streams	0, 1	0		Integer
C_ICACHE_VICTIMS	Instruction cache victims	0, 2, 4, 8	0		Integer
C_ICACHE_DATA_WIDTH	Instruction cache data width: • 0 = 32 bits • 1 = Full cache line • 2 = 512 bits	0, 1, 2	0		Integer
C_ICACHE_BYTE_SIZE	Instruction cache size	64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536[2]	8192		Integer
C_DCACHE_BASEADDR	Data cache base address	0x0 - 0xFFFFFFFF FFFFFFFF	0x0		std_logic_ vector
C_DCACHE_HIGHADDR	Data cache high address	0x0 - 0xFFFFFFFF FFFFFFFF	0x3FFFFFFF		std_logic_ vector

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_USE_DCACHE	Data cache	0, 1	0		Integer
C_DCACHE_LINE_LEN	Data cache line length	4, 8, 16	4		Integer
C_DCACHE_FORCE_TAG_LUTRAM	Data cache tag always implemented with distributed RAM	0, 1	0		Integer
C_DCACHE_USE_WRITEBACK	Data cache write-back storage policy used	0, 1	0		Integer
C_DCACHE_VICTIMS	Data cache victims	0, 2, 4, 8	0		Integer
C_DCACHE_DATA_WIDTH	Data cache data width: • 0 = 32 bits • 1 = Full cache line • 2 = 512 bits	0, 1, 2	0		Integer
C_DCACHE_BYTE_SIZE	Data cache size	64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536[2]	8192		Integer
C_USE_MMU	Memory management mode: • 0 = Machine • 1 = User • 3 = Supervisor	0, 1, 3	0		Integer
C_MMU_PRIVILEGED_INSTR	Allow user mode access to stream CSR	0, 1	0		Integer
C_PMP_ENTRIES	Physical Memory Protection entries	0 - 64	0		Integer
C_PMP_GRANULARITY[3]	Physical Memory Protection granularity	0 - 14	0 - 4		Integer
C_PMP_ENHANCEMENTS	Physical Memory Protection enhancements (Smepmp)	0, 1	0		
C_USE_INTERRUPT	Enable interrupt handling: • 0 = No interrupt • 1 = Standard • 2 = Fast	0, 1, 2	1	Yes	Integer
C_USE_EXT_BRK	Enable external break handling (platform interrupt)	0, 1	0		Integer
C_USE_EXT_NM_BRK	Enable external non-maskable break (non-maskable interrupt)	0, 1	0		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_USE_SLEEP	Enable sleep, hibernate, suspend outputs: • Bit 0 = Sleep • Bit 1 = Hibernate • Bit 2 = Suspend	0-7	0		Integer
C_USE_NON_SECURE	Use corresponding non-secure input	0-15	0	Yes	Integer
C_USE_BRANCH_TARGET_CACHE	Enable branch target cache	0,1	0		Integer
C_BRANCH_TARGET_CACHE_SIZE	Branch target cache size: • 0 = Default (1024 entries) • 1 = 8 entries • 2 = 16 entries • 3 = 32 entries • 4 = 64 entries • 5 = 512 entries • 6 = 1024 entries • 7 = 2048 entries	0-7	0		Integer
C_M_AXI_DP_THREAD_ID_WIDTH	Data side AXI thread ID width	1	1		Integer
C_M_AXI_DP_DATA_WIDTH	Data side AXI data width	32	32		Integer
C_M_AXI_DP_ADDR_WIDTH	Data side AXI address width	32-64	32	Yes	Integer
C_M_AXI_DP_SUPPORTS_THREADS	Data side AXI uses threads	0	0		Integer
C_M_AXI_DP_SUPPORTS_READ	Data side AXI support for read accesses	1	1		Integer
C_M_AXI_DP_SUPPORTS_WRITE	Data side AXI support for write accesses	1	1		Integer
C_M_AXI_DP_SUPPORTS_NARROW_BURST	Data side AXI narrow burst support	0	0		Integer
C_M_AXI_DP_PROTOCOL	Data side AXI protocol	AXI4, AXI4LITE	AXI4 LITE	Yes	String
C_M_AXI_DP_EXCLUSIVE_ACCESS	Data side AXI exclusive access support	0,1	0		Integer
C_M_AXI_IP_THREAD_ID_WIDTH	Instruction side AXI thread ID width	1	1		Integer
C_M_AXI_IP_DATA_WIDTH	Instruction side AXI data width	32	32		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_M_AXI_IP_ADDR_WIDTH	Instruction side AXI address width	32-64	32	Yes	Integer
C_M_AXI_IP_SUPPORTS_THREADS	Instruction side AXI uses threads	0	0		Integer
C_M_AXI_IP_SUPPORTS_READ	Instruction side AXI support for read accesses	1	1		Integer
C_M_AXI_IP_SUPPORTS_WRITE	Instruction side AXI support for write accesses	0	0		Integer
C_M_AXI_IP_SUPPORTS_NARROW_BURST	Instruction side AXI narrow burst support	0	0		Integer
C_M_AXI_IP_PROTOCOL	Instruction side AXI protocol	AXI4LITE	AXI4 LITE		String
C_M_AXI_DC_THREAD_ID_WIDTH	Data cache AXI ID width	1	1		Integer
C_M_AXI_DC_DATA_WIDTH	Data cache AXI data width	32, 64, 128, 256, 512	32		Integer
C_M_AXI_DC_ADDR_WIDTH	Data cache AXI address width	32-64	32	Yes	Integer
C_M_AXI_DC_SUPPORTS_THREADS	Data cache AXI uses threads	0	0		Integer
C_M_AXI_DC_SUPPORTS_READ	Data cache AXI support for read accesses	1	1		Integer
C_M_AXI_DC_SUPPORTS_WRITE	Data cache AXI support for write accesses	1	1		Integer
C_M_AXI_DC_SUPPORTS_ NARROW_BURST	Data cache AXI narrow burst support	0	0		Integer
C_M_AXI_DC_SUPPORTS_USER_SIGNALS	Data cache AXI user signal support	1	1		Integer
C_M_AXI_DC_PROTOCOL	Data cache AXI protocol	AXI4	AXI4		String
C_M_AXI_DC_AWUSER_WIDTH	Data cache AXI user width	5	5		Integer
C_M_AXI_DC_ARUSER_WIDTH	Data cache AXI user width	5	5		Integer
C_M_AXI_DC_WUSER_WIDTH	Data cache AXI user width	1	1		Integer
C_M_AXI_DC_RUSER_WIDTH	Data cache AXI user width	1	1		Integer
C_M_AXI_DC_BUSER_WIDTH	Data cache AXI user width	1	1		Integer
C_M_AXI_DC_EXCLUSIVE_ACCESS	Data cache AXI exclusive access support	0,1	0		Integer
C_M_AXI_DC_USER_VALUE	Data cache AXI user value	0-31	31		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_M_AXI_IC_THREAD_ID_WIDTH	Instruction cache AXI ID width	1	1		Integer
C_M_AXI_IC_DATA_WIDTH	Instruction cache AXI data width	32, 64, 128, 256, 512	32		Integer
C_M_AXI_IC_ADDR_WIDTH	Instruction cache AXI address width	32-64	32	Yes	Integer
C_M_AXI_IC_SUPPORTS_THREADS	Instruction cache AXI uses threads	0	0		Integer
C_M_AXI_IC_SUPPORTS_READ	Instruction cache AXI support for read accesses	1	1		Integer
C_M_AXI_IC_SUPPORTS_WRITE	Instruction cache AXI support for write accesses	0	0		Integer
C_M_AXI_IC_SUPPORTS_NARROW_BURST	Instruction cache AXI narrow burst support	0	0		Integer
C_M_AXI_IC_SUPPORTS_ USER_SIGNALS	Instruction cache AXI user signal support	1	1		Integer
C_M_AXI_IC_PROTOCOL	Instruction cache AXI protocol	AXI4	AXI4		String
C_M_AXI_IC_AWUSER_WIDTH	Instruction cache AXI user width	5	5		Integer
C_M_AXI_IC_ARUSER_WIDTH	Instruction cache AXI user width	5	5		Integer
C_M_AXI_IC_WUSER_WIDTH	Instruction cache AXI user width	1	1		Integer
C_M_AXI_IC_RUSER_WIDTH	Instruction cache AXI user width	1	1		Integer
C_M_AXI_IC_BUSER_WIDTH	Instruction cache AXI user width	1	1		Integer
C_M_AXI_IC_USER_VALUE	Instruction cache AXI user value	0-31	31		Integer
C_Mn_AXIS_PROTOCOL	AXI4-Stream protocol	Generic	Generic		Integer
C_Sn_AXIS_PROTOCOL	AXI4-Stream protocol	Generic	Generic		Integer
C_Mn_AXIS_DATA_WIDTH	AXI4-Stream master data width	32	32	NA	Integer
C_Sn_AXIS_DATA_WIDTH	AXI4-Stream slave data width	32	32	NA	Integer
C_NUM_SYNC_FF_CLK	`Reset` and `Wakeup[0:1]` synchronization stages	≥0	2		Integer
C_NUM_SYNC_FF_CLK_IRQ	`Interrupt` input signal synchronization stages	≥0	1		Integer
C_NUM_SYNC_FF_CLK_DEBUG	Debug serial signal synchronization stages	≥0	2		Integer

Table 63: **Configuration Parameters** *(cont'd)*

Parameter Name	Feature/ Description	Allowable Values	Default Value	Tool Assigned	VHDL Type
C_NUM_SYNC_FF_DBG_CLK	Internal synchronization stages to `Dbg_Clk`	≥0	1		Integer

Notes:

1. The 7 least significant bits must all be 0.
2. Not all sizes are permitted in all architectures. The cache uses 0–32 RAMB primitives (0 if cache size is less than 2048).
3. With 64-bit MicroBlaze V the minimum granularity is 1. With caches enabled the minimum granularity is determined by the base-2 logarithm of the maximum cache line length.
4. When the F or D floating-point extension is enabled (C_USE_FPU > 0), the M extension (C_USE_MULDIV) must also be enabled.

Table 64: **Parameter C_FAMILY Allowable Values**

Device Family	Allowable Values
Artix devices	aartix7 artix7 artix7l qartix7 qartix7l artixuplus
Kintex 7, Kintex UltraScale, Kintex UltraScale+ devices	kintex7 kintex7l qkintex7 qkintex7l kintexu kintexuplus
Spartan devices	spartan7 spartanuplus
Virtex 7, Virtex UltraScale, Virtex UltraScale+ devices	qvirtex7 virtex7 virtexu virtexuplus virtexuplusHBM
Zynq devices	azynq zynq qzynq zynquplus zynquplusRFSOC
Versal devices	versal

The following table lists the parameters associated with each configurable feature.

Table 65: **Configurable Feature Parameters**

Feature	Associated Parameter
Processor pipeline depth	C_OPTIMIZATION
"M" Standard Extension for Integer Multiplier and Divider	C_USE_MULDIV
"A" Standard Extension for Atomic Instructions	C_USE_ATOMIC
"C" Standard Extension for Compressed Instructions	C_USE_COMPRESSION
"F" Standard Extension for Single-Precision Floating-Point	C_USE_FPU = 1
"Zba", "Zbb", "Zbc", "Zbs" Bit-Manipulation ISA Extensions	C_USE_ZBA, C_USE_ZBB, C_USE_ZBC, C_USE_ZBS
User-mode	C_USE_MMU = 1
Physical Memory Protection	C_PMP_ENTRIES > 0
Physical Memory Protection Granularity	C_PMP_GRANULARITY
Supervisor Level ISA	C_USE_MMU = 3
Page-Based Virtual Memory System	C_USE_MMU = 3
External debug support	C_DEBUG_ENABLED
Parallel debug module interface	C_DEBUG_INTERFACE = 1
Branch target cache (BTC)	C_USE_BRANCH_TARGET_CACHE
Local memory bus (LMB) data side interface	C_D_LMB

Table 65: **Configurable Feature Parameters** *(cont'd)*

Feature	Associated Parameter
Local memory bus (LMB) instruction side interface	C_I_LMB
Instruction and data caches	C_USE_ICACHE, C_USE_DCACHE
Cache line word length	C_ICACHE_LINE_LEN, C_DCACHE_LINE_LEN
LUT cache memory	C_ICACHE_BYTE_SIZE ≤ 1024, C_DCACHE_BYTE_SIZE ≤ 1024
Use write-back caching policy for D-Cache	C_DCACHE_USE_WRITEBACK
Streams for I-Cache	C_ICACHE_STREAMS
Victim handling for I-Cache	C_ICACHE_VICTIMS
Victim handling for D-Cache	C_DCACHE_VICTIMS
Force distributed RAM for cache tags	C_ICACHE_FORCE_TAG_LUTRAM, C_DCACHE_FORCE_TAG_LUTRAM
Configurable cache data widths	C_ICACHE_DATA_WIDTH, C_DCACHE_DATA_WIDTH
AXI4 (M_AXI_DP) data side interface	C_D_AXI
AXI4 (M_AXI_IP) instruction side interface	C_I_AXI
AXI4 (S_AXI) slave interface	C_S_AXI
AXI4 (M_AXI_DC) protocol for D-Cache	C_INTERCONNECT = 2
AXI4 (M_AXI_IC) protocol for I-Cache	C_INTERCONNECT = 2
ACE (M_ACE_DC) protocol for D-Cache	C_INTERCONNECT = 3
ACE (M_ACE_IC) protocol for I-Cache	C_INTERCONNECT = 3
AXI4 non-secure mode	C_USE_NON_SECURE
Lockstep support	C_LOCKSTEP_SLAVE
Configurable use of FPGA primitives	C_AVOID_PRIMITIVES
Relocatable base vectors	C_BASE_VECTORS
Pipeline pause functionality	C_ENABLE_DISCRETE_PORTS

The following table lists the parameters associated with each MicroBlaze V external interface.

Table 66: **Interface Parameters**

Interface	Associated Parameter
M_AXI_DP	C_D_AXI
DLMB	C_D_LMB
M_AXI_IP	C_I_AXI
ILMB	C_I_LMB
M0_AXIS..M15_AXIS, S0_AXIS..S15_AXIS	C_FSL_LINKS
M_AXI_DC	C_USE_DCACHE, C_INTERCONNECT = 2
M_ACE_DC	C_USE_DCACHE, C_INTERCONNECT = 3
M_AXI_IC	C_USE_ICACHE, C_INTERCONNECT = 2
M_ACE_IC	C_USE_ICACHE, C_INTERCONNECT = 3
INTERRUPT	C_USE_INTERRUPT

Table 66: **Interface Parameters** *(cont'd)*

Interface	Associated Parameter
DEBUG	C_DEBUG_ENABLED
S_AXI	C_S_AXI

Appendix A

Performance and Resource Utilization

Performance

Performance characterization of this core has been done using the margin system methodology. The details of the margin system characterization methodology are described in IP Characterization and f_{MAX} Margin System Methodology.

Maximum Frequencies

The maximum frequencies for the MicroBlaze™ V core are provided in the following table. The fastest speed grade of each family is used to generate the results in this table.

Table 67: **Maximum Frequencies**

Family	F_{max} (MHz)
AMD Virtex™ 7	322
AMD Kintex™ 7	327
AMD Artix™ 7	223
AMD Zynq™ 7000	224
AMD Spartan™ 7	196
AMD Virtex™ UltraScale™	393
AMD Kintex™ UltraScale™	384
AMD Virtex™ UltraScale+™	556
AMD Kintex™ UltraScale+™	557
AMD Zynq™ UltraScale+™	506
AMD Artix™ UltraScale+™	494
AMD Spartan™ UltraScale+™	413
AMD Versal™ Adaptive SoCs	460

Resource Utilization

The MicroBlaze V resource utilization for various parameter configurations are measured for the following devices:

- **Virtex 7 devices:** Table 68
- **Kintex 7 devices:** Table 69
- **Artix 7 devices:** Table 70
- **Zynq 7000 devices:** Table 71
- **Spartan 7 devices:** Table 72
- **Virtex UltraScale devices:** Table 73
- **Kintex UltraScale devices:** Table 74
- **Virtex UltraScale+ devices:** Table 75
- **Kintex UltraScale+ devices:** Table 76
- **Zynq UltraScale+ devices:** Table 77
- **Artix UltraScale+ devices:** Table 78
- **Spartan UltraScale+ devices:** Table 79
- **Versal devices:** Table 80

The parameter values for each of the measured configurations are shown in Table 81: Parameter Configurations. The configurations directly correspond to predefined presets and templates in the MicroBlaze V Configuration Wizard.

Table 68: **Device Utilization – Virtex 7 FPGAs (XC7VX485T ffg1761-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microelectronic Preset	2195	1135	0	4	227
Real-time Preset	4083	2672	6	4	176
Application Preset	8500	5329	16	6	147
Maximum Frequency	1205	826	0	0	322

Table 69: **Device Utilization – Kintex 7 FPGAs (XC7K325T ffg900-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2181	1135	0	4	230
Real-time Preset	4072	2671	6	4	169
Application Preset	8456	5330	16	6	166
Maximum Frequency	1207	826	0	0	327

Table 70: **Device Utilization – Artix 7 FPGAs (XC7A200T fbg676-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2236	1138	0	4	156
Real-time Preset	4106	2685	6	4	119
Application Preset	8623	5325	16	6	109
Maximum Frequency	1227	826	0	0	223

Table 71: **Device Utilization – Zynq 7000 FPGAs (XC7Z020 clg484-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2221	1145	0	4	164
Real-time Preset	4101	2670	6	4	117
Application Preset	8577	5342	16	6	115
Maximum Frequency	1234	826	0	0	224

Table 72: **Device Utilization – Spartan 7 FPGAs (XC7S25 csga225-2)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2233	1135	0	4	139
Real-time Preset	4127	2672	6	4	102
Application Preset	8596	5333	16	6	92
Maximum Frequency	1257	826	0	0	196

Table 73: **Device Utilization – Virtex UltraScale FPGAs (XCVU095 ffvd1924-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2171	1145	0	4	283
Real-time Preset	3984	2679	6	4	212
Application Preset	8393	5334	16	6	193
Maximum Frequency	1202	826	0	0	393

Table 74: **Device Utilization – Kintex UltraScale FPGAs (XCKU040 ffva1156-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2155	1135	0	4	290
Real-time Preset	3983	2671	6	4	201
Application Preset	8363	5304	16	6	209
Maximum Frequency	1188	826	0	0	384

Table 75: **Device Utilization – Virtex UltraScale+ FPGAs (XCVU3P ffvc1517-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2200	1135	0	4	403
Real-time Preset	3812	2680	6	4	267
Application Preset	8053	5326	16	6	238
Maximum Frequency	1219	826	0	0	556

Table 76: **Device Utilization – Kintex UltraScale+ FPGAs (XCKU15P ffva1156-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2228	1135	0	4	399
Real-time Preset	3815	2671	6	4	290
Application Preset	8020	5300	16	6	281
Maximum Frequency	1220	826	0	0	557

Table 77: **Device Utilization – Zynq UltraScale+ Devices (XCZU9EG ffvb1156-3)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2190	1135	0	4	384
Real-time Preset	3851	2677	6	4	261
Application Preset	8076	5343	16	6	262
Maximum Frequency	1211	826	0	0	506

Table 78: **Device Utilization – Artix UltraScale+ FPGAs (XCAU25P ffvb676-2)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2142	1135	0	4	349
Real-time Preset	3920	2669	6	4	252
Application Preset	8275	5301	16	6	252
Maximum Frequency	1216	826	0	0	494

Table 79: **Device Utilization – Spartan UltraScale+ FPGAs (XCSU35P sbvb625-2)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP48	F_{max} MHz
Microcontroller Preset	2175	1135	0	4	311
Real-time Preset	3993	2681	6	4	198
Application Preset	8436	5306	16	6	194
Maximum Frequency	1171	826	0	0	413

Table 80: **Device Utilization – Versal Adaptive SoC Devices (XCVC1920 vsva2197-3HP)**

Configuration	Device Resources				
	LUTs	FFs	BRAMs (36K)	DSP58	F_{max} MHz
Microcontroller Preset	2529	1476	0	4	363
Real-time Preset	4516	2758	4	4	251
Application Preset	9131	5217	13	6	257
Maximum Frequency	1506	1068	0	0	460

Table 81: **Parameter Configurations**

Parameter	Microcontroller Preset RV32IMC	Real-time Preset RV32IMAC	Application Preset RV32IMAFC	Maximum Frequency RV32I
C_OPTIMIZATION	1	0	0	0

Table 81: **Parameter Configurations** *(cont'd)*

Parameter	Microcontroller Preset RV32IMC	Real-time Preset RV32IMAC	Application Preset RV32IMAFC	Maximum Frequency RV32I
C_DCACHE_BYTE_SIZE	4096	8192	32768	4096
C_DCACHE_LINE_LEN	4	4	4	4
C_DCACHE_USE_WRITEBACK	0	0	0	0
C_DEBUG_ENABLED	1	1	1	0
C_M_AXI_D_BUS_EXCEPTION	0	1	1	0
C_FSL_EXCEPTION	0	0	0	0
C_FSL_LINKS	0	0	0	0
C_ICACHE_BYTE_SIZE	4096	8192	32768	4096
C_ICACHE_LINE_LEN	4	4	8	4
C_ILL_INSTR_EXCEPTION	1	1	1	0
C_M_AXI_I_BUS_EXCEPTION	0	1	1	0
C_NUMBER_OF_PC_BRK	1	2	2	1
C_NUMBER_OF_RD_ADDR_BRK	0	0	1	0
C_NUMBER_OF_WR_ADDR_BRK	0	0	1	0
C_USE_ATOMIC	0	1	1	0
C_USE_COMPRESSION	1	1	1	0
C_USE_DCACHE	0	1	1	0
C_USE_MULDIV	2	1	2	0
C_USE_BARREL	1	1	1	2
C_USE_EXTENDED_FSL_INSTR	0	0	0	0
C_USE_FPU	0	0	1	0
C_USE_ICACHE	0	1	1	0
C_USE_MMU	0	1	3	0
C_USE_BRANCH_TARGET_CACHE	0	0	1	0
C_BRANCH_TARGET_CACHE_SIZE	0	0	0	0
C_ICACHE_STREAMS	0	0	1	0
C_ICACHE_VICTIMS	0	0	8	0
C_DCACHE_VICTIMS	0	0	0	0
C_ICACHE_FORCE_TAG_LUTRAM	0	0	0	0
C_DCACHE_FORCE_TAG_LUTRAM	0	0	0	0
C_D_AXI	1	1	1	0
C_USE_INTERRUPT	1	1	1	0
C_USE_SLEEP	1	1	1	1
C_DEBUG_EVENT_COUNTERS	0	0	0	0
C_DEBUG_LATENCY_COUNTERS	0	0	0	0
C_PMP_ENTRIES	0	6	0	0

IP Characterization and f_{MAX} Margin System Methodology

This section describes the methods to determine the maximum frequency (F_{MAX}) of IP operation within a system design. The method enables realistic performance reporting for any AMD Adaptive Computing FPGA architecture. The maximum frequency of a design is the maximum frequency at which the overall system can be implemented without encountering timing issues.

F_{MAX} Margin System Methodology

It is important to determine the IP performance in the context of a user system. In the case of the MicroBlaze V characterization, the system includes the following items:

- The IP under test (MicroBlaze V processor)
- Local memory (LMB)
- One level of interconnect (AXI4, AXI4-Lite)
- Memory controller (EMC)
- On-chip block RAM controller
- Peripherals (UART, Timer, Interrupt Controller, MDM V)

Determining the F_{MAX} of an Embedded IP with these components provides a more realistic performance target.

The system above has two types of AXI Interconnect. AXI4-Lite used for peripheral command and control, and AXI4 used for memory accesses.

For F_{MAX} Margin System Analysis, the clock frequency of the system is incremented up to the maximum frequency where the system breaks with timing violations (worst case negative slack). The reported frequency is the failing frequency subtracted with this worst case negative slack.

Tool Options and Other Factors

AMD Adaptive Computing tools offer a number of options and settings that provide a trade-off between design performance, resource usage, implementation runtime, and memory footprint. The settings that produce the best results for one design might not necessarily work for another design.

For the purpose of the F_{MAX} Margin System Analysis, the IP design is characterized with default settings without specific constraints (other than the clocking constraint). This analysis is done with all different FPGA architectures and the maximum speed grade.

Appendix B

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to https://docs.amd.com.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the AMD Vivado™ IDE, select **Help → Documentation and Tutorials**.
- On Windows, click the **Start** button and select **Xilinx Design Tools → DocNav**.
- At the Linux command prompt, enter `docnav`.

***Note*:** For more information on DocNav, refer to the *Documentation Navigator User Guide* (UG968).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- Go to the Design Hubs web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see Support.

References

These documents provide supplemental material useful with this guide:

1. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA
2. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture
3. RISC-V External Debug Support, Version 1.0
4. RISC-V N-Trace (Nexus-based Trace) Specification, Version 1.0
5. RISC-V Trace Control Interface Specification, Version 1.0
6. *MicroBlaze V Processor Embedded Design User Guide* (UG1711)
7. *AMBA AXI and ACE Protocol Specification* (Arm IHI 0022H)
8. *MicroBlaze Debug Module (MDM) V LogiCORE IP Product Guide* (PG428)
9. *LMB BRAM Interface Controller LogiCORE IP Product Guide* (PG112)
10. *Soft Error Mitigation Controller LogiCORE IP Product Guide* (PG036)
11. *UltraScale Architecture Soft Error Mitigation Controller LogiCORE IP Product Guide* (PG187)
12. *Triple Modular Redundancy (TMR) LogiCORE IP Product Guide* (PG268)
13. *Vitis Unified Software Platform Documentation Landing Page* (UG1416)
14. *Vivado Design Suite User Guide: Designing with IP* (UG896)
15. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994)
16. *Vitis Unified Software Platform Documentation: Embedded Software Development* (UG1400)
17. ELF: Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification
18. *MicroBlaze Processor Reference Guide* (UG984)
19. *Device Reliability Report* (UG116)

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
07/09/2025 Version 2025.1	
General updates	• Added RV32I custom extended address instructions • Corrected AXI4-Stream custom instruction descriptions • Added PMP Enhancements (Smepmp) description • Added RISC-V N-Trace external trace • Described default halt behavior • Described LMB protocols
12/04/2024 Version 2024.2	
General updates	• Added double-precision floating point • Corrected custom instruction description • Added RISC-V N-Trace program trace • Added non-intrusive profiling
05/30/2024 Version 2024.1	
Initial release.	N/A

Please Read: Important Legal Notices

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright